
El Motor Cuantitativo

Fundamentos Matemáticos y Estadísticos con Python para el Análisis de Datos

Alex Goyzueta Delgado

alexgoyzueta2018@gmail.com

El Motor Cuantitativo: Fundamentos Matemáticos y Estadísticos con Python para el Análisis de Datos

Copyright © 2026 Alex Goyzueta Delgado

Todos los derechos reservados. Ninguna parte de este libro puede ser reproducida, almacenada en un sistema de recuperación o transmitida en ninguna forma o por ningún medio electrónico, mecánico, fotocopia, grabación u otro, sin el permiso previo por escrito del autor.

ISBN: 978-0-000-00000-0

Edición: 1.^a edición, Junio 2026

Editor: Autoedición técnica

Revisión técnica: Los ejemplos de código han sido probados con Python 3.12+ y las librerías NumPy 1.26+, SciPy 1.12+, SymPy 1.12+, pandas 2.1+ y scikit-learn 1.4+.

Contacto: alexgoyzueta2018@gmail.com

"Las matemáticas no mienten. Lo que hacen las personas con ellas, esa es otra historia."

Dedicatoria

A mis padres, que me enseñaron a hacer preguntas en lugar de memorizar respuestas.

A Marcia, por su paciencia infinita mientras este libro cobraba vida.

Y a todos los analistas que alguna vez se sintieron intimidados por una ecuación: esto es para ustedes.

Prefacio

Conocí a un analista brillante que construía modelos predictivos asombrosos, pero cuando le pregunté por qué usaba regresión logística en lugar de un árbol de decisión, no supo responderme. Usaba las herramientas por inercia, como quien enciende un automóvil sin entender qué sucede bajo el capó.

Este libro nació de esa conversación y de cientos similares.

Durante años enseñé matemáticas y estadística a profesionales de datos, y noté un patrón recurrente: la mayoría sabía *qué* código ejecutar, pero no *por qué* funcionaba. Llamaban a ``sklearn.linear_model.LogisticRegression()`` sin comprender la función sigmoide, el concepto de likelihood o por qué se optimizaba con gradient descent.

El Motor Cuantitativo es mi intento de cerrar esa brecha.

No encontrarás aquí un tratado académico de matemáticas. Tampoco encontrarás un manual de Python más. Encontrarás el término medio: las matemáticas aplicadas que todo analista de datos necesita dominar, explicadas a través de código que puedes ejecutar, modificar y romper.

Mi enfoque es simple: cada concepto matemático va acompañado de código Python funcional. La teoría ilumina el código; el código valida la teoría. No te pediré que demuestres teoremas, pero sí que entiendas qué significan y cómo usarlos en tu trabajo diario.

Este libro está dirigido a analistas de datos, científicos de datos en formación, ingenieros que migran al análisis cuantitativo y cualquier persona que use Python para extraer significado de datos y quiera ir más allá de la superficie.

Los requisitos son mínimos: Python básico y haber visto matemáticas en algún momento de tu vida. Si sabes qué es una función, una variable y un bucle ``for``, estás listo.

Mi consejo: no leas este libro pasivamente. Cada capítulo contiene código. Ejecútalo. Modifícalo. Rompe algo y luego arréglalo. Las matemáticas se entienden haciéndolas, no memorizándolas.

Bienvenido al motor. Arranquemos.

— Alex Goyzueta Delgado

Introducción

¿Qué aprenderás en este libro?

Al terminar este libro, serás capaz de:

- **Explicar** los fundamentos matemáticos detrás de las librerías de Python que ya usas
- **Implementar** desde cero algoritmos estadísticos y cuantitativos
- **Diseñar** experimentos y pruebas A/B rigurosas
- **Construir** modelos predictivos entendiendo cada supuesto y limitación
- **Comunicar** hallazgos cuantitativos a audiencias no técnicas

Cómo está organizado

El libro se divide en **7 módulos**, cada uno construyendo sobre el anterior:

1	El Lenguaje de los Datos	3
3	Inferencia Estadística	4
5	Modelado Predictivo Básico	4
7	Casos de Uso Reales	3

Además, encontrarás apéndices con guías de instalación, cheat sheets matemáticos y referencias.

Convenciones utilizadas

```
# Los bloques de código muestran ejemplos funcionales
# Los comentarios con # Resultado: muestran la salida esperada
print("Ejemplo") # Resultado: Ejemplo
```

Las **notas importantes** aparecen en texto resaltado. Los términos técnicos en *cursiva* la primera vez que se presentan.

Requisitos técnicos

Necesitarás Python 3.10+ y las siguientes librerías:

```
numpy>=1.24.0  
scipy>=1.11.0  
sympy>=1.12.0  
pandas>=2.0.0  
scikit-learn>=1.3.0  
matplotlib>=3.7.0  
seaborn>=0.12.0  
statsmodels>=0.14.0
```

El Apéndice A detalla la instalación paso a paso.

¿Por qué "Motor Cuantitativo"?

Porque las matemáticas y la estadística son el motor que impulsa el análisis de datos. Python es la carrocería, el volante y los pedales. Pero sin entender el motor, cualquier viaje analítico terminará en la cuneta. Este libro te abre el capó.

Módulo 1: El Lenguaje de los Datos

El Ecosistema Matemático en Python

Python se ha convertido en el lenguaje dominante para el análisis de datos no por casualidad, sino por su ecosistema de librerías matemáticas. En este capítulo exploraremos las tres herramientas fundamentales que usarás a lo largo de todo el libro: NumPy, SciPy y SymPy.

NumPy: El Corazón Numérico

NumPy (*Numerical Python*) es la librería fundamental para cómputo científico en Python. Su estructura principal, el `ndarray`, es un contenedor multidimensional homogéneo que permite operaciones vectorizadas de alto rendimiento.

Arrays vs Listas de Python

La diferencia crucial entre una lista de Python y un array de NumPy es la eficiencia:

```
import numpy as np
import time

# Lista de Python
lista = list(range(10_000_000))

# Array de NumPy
arr = np.arange(10_000_000)

# Multiplicación por 2 con listas
inicio = time.time()
resultado_lista = [x * 2 for x in lista]
print(f"Lista: {time.time() - inicio:.4f} segundos")

# Multiplicación por 2 con NumPy (vectorizada)
inicio = time.time()
resultado_arr = arr * 2
print(f"NumPy: {time.time() - inicio:.4f} segundos")
# Resultado: Lista: ~0.8s, NumPy: ~0.02s (40x más rápido)
```

Las operaciones vectorizadas ocurren en C, sin bucles Python. Esto permite trabajar con millones de datos sin sacrificar rendimiento.

Creación de Arrays

```
# Desde lista
a = np.array([1, 2, 3, 4, 5])
```

```
# Rangos
b = np.arange(0, 10, 2)    # [0, 2, 4, 6, 8]
c = np.linspace(0, 1, 5)  # [0.0, 0.25, 0.5, 0.75, 1.0]

# Matrices especiales
d = np.zeros((3, 3))      # Matriz 3x3 de ceros
e = np.ones((2, 4))       # Matriz 2x4 de unos
f = np.eye(4)             # Matriz identidad 4x4
g = np.random.randn(1000) # 1000 valores aleatorios N(0,1)
```

Broadcasting

Una de las características más poderosas de NumPy es el *broadcasting*: operar arrays de diferentes formas sin necesidad de expansión explícita:

```
# Broadcasting: sumar un escalar a cada elemento
arr = np.array([[1, 2, 3], [4, 5, 6]])
resultado = arr + 10
print(resultado)
# [[11 12 13]
#  [14 15 16]]

# Broadcasting con arrays de diferente forma
fila = np.array([1, 0, 1])
print(arr + fila)
# [[2 2 4]
#  [5 5 7]]
```

Las reglas del broadcasting: las dimensiones se comparan de derecha a izquierda. Son compatibles si son iguales o una es 1.

Indexación y Cortes

```
arr = np.arange(12).reshape(3, 4)
print(arr)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

# Indexación booleana
print(arr[arr > 5])
# [ 6  7  8  9 10 11]

# Indexación fancy
print(arr[[0, 2], :])
# [[ 0  1  2  3]
#  [ 8  9 10 11]]
```

SciPy: Matemáticas Aplicadas

SciPy se construye sobre NumPy y añade algoritmos para optimización, integración, interpolación, álgebra lineal, procesamiento de señales y estadística.

```
from scipy import optimize, stats, linalg, integrate
```

Optimización

Encontrar el mínimo de una función es esencial en machine learning:

```
from scipy import optimize

# Minimizar una función simple
def f(x):
    return x**2 + 5 * np.sin(x)

resultado = optimize.minimize_scalar(f)
print(f"Mínimo en x = {resultado.x:.4f}, f(x) = {resultado.fun:.4f}")
# Resultado: Mínimo en x = -1.1108, f(x) = -3.2464

# Encontrar raíces
raiz = optimize.root_scalar(lambda x: x**2 - 4, bracket=[0, 5])
print(f"Raíz en x = {raiz.root:.4f}")
# Resultado: Raíz en x = 2.0000
```

Distribuciones Estadísticas

Acceso a más de 80 distribuciones de probabilidad:

```
from scipy import stats

# Distribución normal estándar
z = stats.norm.ppf(0.975) # Percentil 97.5% (para IC del 95%)
print(f"z(0.975) = {z:.4f}")
# Resultado: z(0.975) = 1.9600

# Función de densidad
pdf = stats.norm.pdf(0)
print(f"N(0,1) en x=0: {pdf:.4f}")
# Resultado: N(0,1) en x=0: 0.3989

# Valor p para un estadístico t
t_stat = 2.1
p_valor = stats.t.sf(t_stat, df=30) * 2 # Dos colas
print(f"Valor p: {p_valor:.4f}")
# Resultado: Valor p: 0.0441
```

SymPy: Matemática Simbólica

SymPy permite hacer matemáticas con símbolos en lugar de números. Es como tener un CAS (Computer Algebra System) dentro de Python.

```
import sympy as sp

# Definir símbolos
x, y = sp.symbols('x y')

# Expresiones simbólicas
```



```

expr = x**2 + 2*x + 1
print(sp.factor(expr))
# (x + 1)**2

# Límites
limite = sp.limit(sp.sin(x) / x, x, 0)
print(f"límite sin(x)/x cuando x→0: {limite}")
# límite sin(x)/x cuando x→0: 1

# Derivadas
derivada = sp.diff(x**3 * sp.sin(x), x)
print(f"derivada: {derivada}")
# derivada: x**3*cos(x) + 3*x**2*sin(x)

# Integrales
integral = sp.integrate(sp.exp(-x**2), (x, -sp.oo, sp.oo))
print(f"integral: {integral}")
# integral: sqrt(pi)

```

Evaluación Numérica de Expresiones Simbólicas

```

expr = sp.sin(x) / x
f_lambda = sp.lambdify(x, expr, 'numpy')
valores = f_lambda(np.array([0.1, 0.5, 1.0]))
print(valores)
# [0.99833417 0.95885108 0.84147098]

```

La función `lambdify` convierte una expresión simbólica en una función NumPy compilable, uniendo lo mejor de ambos mundos.

¿Cuándo usar cada una?

Librería	Para qué	Ejemplo típico
NumPy	Manipulación numérica eficiente	Operaciones vectorizadas, álgebra lineal básica
SciPy	Algoritmos matemáticos avanzados	Optimización, tests estadísticos, señales
SymPy	Matemática simbólica	Derivar fórmulas, simplificar expresiones

> Consejo del Analista:

> Cuando empieces un proyecto nuevo, importa NumPy como `np` y SciPy como `sp` (o `scipy`). Es una convención universal que hará tu código legible para otros analistas.

En la práctica, trabajarás con NumPy a diario, SciPy semanalmente (para estadística y optimización) y SymPy ocasionalmente (para derivar ecuaciones o entender un algoritmo).

Ejercicios Prácticos

1. **Ejercicio 1:** Crea un array NumPy de 1000 elementos con `np.random.randn(1000)`. Multiplica cada elemento por 2 usando un bucle `for` y usando vectorización. Compara los tiempos de ejecución.

- Pista: Usa `time.time()` antes y después de cada operación.

2. **Ejercicio 2:** Usando SciPy, encuentra el mínimo de la función $f(x) = x^4 - 3x^2 + 2x + 1$ en el intervalo $[-2, 2]$. Grafica la función y marca el mínimo.

3. **Ejercicio 3:** Con SymPy, deriva la función $f(x) = e^{(x^2)} \cdot \sin(3x)$ y simplifica el resultado. Evalúa la derivada en $x=2$ usando ``lambdify``.

4. **Ejercicio 4:** Demuestra el broadcasting: crea una matriz 4×3 y un vector fila de 3 elementos. Súmalos. Explica qué está sucediendo con las dimensiones.

Resumen del Capítulo

- **NumPy** proporciona el ``ndarray`` para operaciones vectorizadas eficientes
- **SciPy** implementa algoritmos matemáticos y estadísticos sobre NumPy
- **SymPy** permite manipulación simbólica de expresiones matemáticas
- El **broadcasting** permite operaciones entre arrays de diferente forma
- Las tres librerías se complementan y son interoperables
- ``lambdify`` convierte expresiones simbólicas en funciones NumPy

Conceptos clave aprendidos: ndarray, broadcasting, vectorización, lambdify, SymPy, SciPy, optimización

Álgebra Lineal Esencial: Vectores, Matrices y Operaciones

El álgebra lineal es el lenguaje de los datos modernos. Cada vez que ajustas un modelo de regresión, realizas PCA o entrenas una red neuronal, estás haciendo álgebra lineal. En este capítulo entenderás qué ocurre realmente.

Vectores: Los Bloques Fundamentales

Un vector es una lista ordenada de números. En análisis de datos, cada fila de un dataset es un vector: el vector de características de una observación.

Representación en Python

```
import numpy as np

# Vector fila (1D)
v = np.array([2, -1, 3])
print(f"Vector: {v}, Shape: {v.shape}")
# Vector: [2 -1 3], Shape: (3,)

# Vector columna explícito
v_col = v.reshape(-1, 1)
print(f"Shape columna: {v_col.shape}")
# Shape columna: (3, 1)
```

Operaciones con Vectores

```
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])

# Suma (elemento a elemento)
print(f"Suma: {v1 + v2}")    # [5 7 9]

# Producto punto (dot product)
dot = np.dot(v1, v2)
print(f"Producto punto: {dot}")    # 1*4 + 2*5 + 3*6 = 32

# Producto punto con @ (Python 3.5+)
print(f"Producto punto (@): {v1 @ v2}")    # 32

# Norma euclidiana (magnitud)
```

```

norma = np.linalg.norm(v1)
print(f"Norma L2: {norma:.4f}") # sqrt(1+4+9) = 3.7417

# Normalización (vector unitario)
unitario = v1 / norma
print(f"Unitario: {unitario}")
# [0.2673 0.5345 0.8018]

```

> Advertencia:

> El producto punto de dos vectores ortogonales es 0. Si trabajas con datos de alta dimensión, la mayoría de los vectores aleatorios tienden a ser casi ortogonales entre sí. Este es un aspecto clave de la "maldición de la dimensionalidad".

Interpretación Geométrica del Producto Punto

El producto punto mide la proyección de un vector sobre otro:

```

import numpy as np

a = np.array([1, 0])
b = np.array([np.cos(np.pi/3), np.sin(np.pi/3)]) # 60 grados

dot_ab = a @ b
angulo = np.arccos(dot_ab / (np.linalg.norm(a) * np.linalg.norm(b)))

print(f"Producto punto: {dot_ab:.4f}")
print(f"Ángulo: {np.degrees(angulo):.1f}°")
# Producto punto: 0.5000
# Ángulo: 60.0°

```

Matrices: Datos en 2D

Una matriz es un arreglo bidimensional. Tu dataset tabular es una matriz: filas = observaciones, columnas = variables.

Creación y Propiedades

```

# Matriz 2x3
A = np.array([[1, 2, 3],
              [4, 5, 6]])

# Propiedades
print(f"Shape: {A.shape}") # (2, 3)
print(f"Dimensiones: {A.ndim}") # 2
print(f"Total elementos: {A.size}") # 6

# Transpuesta
print(A.T)
# [[1 4]
#  [2 5]
#  [3 6]]

```

Multiplicación de Matrices

```

A = np.array([[1, 2], [3, 4]]) # 2x2
B = np.array([[5, 6], [7, 8]]) # 2x2

# Multiplicación matricial (dot product)
C = A @ B
print(C)
# [[19 22]
#  [43 50]]

# Equivalente: np.matmul(A, B)

```

Sistemas de Ecuaciones Lineales

Resolver sistemas de ecuaciones es una tarea cotidiana en análisis de datos:

```

# Sistema: 3x + 2y = 7
#           2x - y  = 0

A = np.array([[3, 2],
              [2, -1]])
b = np.array([7, 0])

# Solución: x = A^(-1) * b
x = np.linalg.solve(A, b)
print(f"Solución: x = {x[0]:.2f}, y = {x[1]:.2f}")
# Solución: x = 1.00, y = 2.00

```

Eigenvalores y Eigenvectores

Los eigenvalores y eigenvectores son fundamentales para PCA, análisis de redes y reducción de dimensionalidad.

Un eigenvector de una matriz A es un vector v que, al multiplicarse por A , solo se escala (no cambia de dirección):

$$A \cdot v = \lambda \cdot v$$

Donde λ (lambda) es el eigenvalor asociado.

```

A = np.array([[2, 1],
              [1, 2]])

eigenvalores, eigenvectores = np.linalg.eig(A)

print(f"Eigenvalores: {eigenvalores}")
# [3. 1.]

print(f"Eigenvectores (columnas):\n{eigenvectores}")
# [[ 0.70710678 -0.70710678]
#  [ 0.70710678  0.70710678]]

# Verificación: A * v = λ * v
v = eigenvectores[:, 0]
lam = eigenvalores[0]

```

```
print(np.allclose(A @ v, lam * v)) # True
```

Descomposición en Valores Singulares (SVD)

SVD es, probablemente, el resultado más importante del álgebra lineal para datos. Descompone cualquier matriz A en tres matrices:

$$A = U \cdot \Sigma \cdot V^T$$

```
from numpy.linalg import svd

# Matriz de datos (5 observaciones, 3 variables)
A = np.array([[1, 0, 1],
              [0, 1, 1],
              [1, 1, 0],
              [0, 0, 1],
              [1, 0, 0]])

U, S, Vt = svd(A)

print(f"U shape: {U.shape}")    # (5, 5)
print(f"Sigma: {S}")            # Valores singulares
# [1.9021 1.6180 0.6180]
print(f"Vt shape: {Vt.shape}")  # (3, 3)

# Reconstrucción (truncando a 2 componentes)
k = 2
A_aprox = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
error = np.linalg.norm(A - A_aprox)
print(f"Error de reconstrucción: {error:.4f}")
```

SVD es la base de PCA, sistemas de recomendación (filtrado colaborativo) y compresión de datos.

Matriz de Covarianza

La matriz de covarianza es la herramienta central para entender relaciones entre variables:

```
# Datos: 100 observaciones, 3 variables correlacionadas
np.random.seed(42)
X = np.random.randn(100, 3)

# Introducir correlación
X[:, 1] = 0.7 * X[:, 0] + 0.3 * X[:, 1]
X[:, 2] = -0.4 * X[:, 0] + 0.6 * X[:, 2]

# Matriz de covarianza (centrando los datos)
X_centrado = X - X.mean(axis=0)
cov = (X_centrado.T @ X_centrado) / (X.shape[0] - 1)

print("Matriz de covarianza:")
print(np.round(cov, 3))
```

```
# [[ 1.02    0.714 -0.366]
#  [ 0.714  0.999 -0.256]
#  [-0.366 -0.256  0.978]]

# Comparación con numpy
print(np.round(np.cov(X.T), 3))

# Mismo resultado
```

Los elementos diagonales son las varianzas; los no diagonales, las covarianzas.

Aplicación Práctica: Regresión Lineal desde Cero

El álgebra lineal que acabamos de ver te permite implementar regresión lineal sin scikit-learn:

```
# Datos de ejemplo
np.random.seed(42)
X = np.random.randn(100, 2) # 2 variables
y = 3 + 2 * X[:, 0] - 1.5 * X[:, 1] + np.random.randn(100) * 0.5

# Añadir columna de 1s para el intercepto
X_b = np.column_stack([np.ones(100), X])

# Ecuación normal:  $\beta = (X^T X)^{-1} X^T y$ 
beta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
print(f" $\beta_0 = \{beta[0]:.2f\}$ ,  $\beta_1 = \{beta[1]:.2f\}$ ,  $\beta_2 = \{beta[2]:.2f\}$ ")
#  $\beta_0 = 3.02$ ,  $\beta_1 = 2.01$ ,  $\beta_2 = -1.49$ 

# Coeficientes reales:  $\beta_0=3$ ,  $\beta_1=2$ ,  $\beta_2=-1.5$ 
```

> Consejo del Analista:

> Para datasets grandes ($n > 10,000$ o muchas variables), usar la ecuación normal directa es computacionalmente costosa ($O(n^3)$). En esos casos, el descenso de gradiente (capítulo siguiente) es más eficiente.

La ecuación normal $\beta = (X^T X)^{-1} X^T y$ es la solución analítica de la regresión lineal.

Ejercicios Prácticos

1. **Ejercicio 1:** Dados los vectores $a = [3, -1, 2]$ y $b = [0, 4, -2]$, calcula: (a) producto punto, (b) norma de cada uno, (c) ángulo entre ellos.

• Pista: `np.arccos(dot / (norm_a * norm_b))`

2. **Ejercicio 2:** Resuelve el sistema de ecuaciones usando `np.linalg.solve`:

...

$$2x + 3y - z = 1$$

$$4x - 2y + 5z = -3$$

$$x + y + z = 2$$

...

3. **Ejercicio 3:** Implementa la descomposición SVD desde cero (usando `np.linalg.eig` sobre $A^T A$ y $A A^T$) y verifica que coincide con `np.linalg.svd`.

4. **Ejercicio 4:** Genera una matriz 5x3 de datos aleatorios. Calcula su matriz de covarianza.

¿Qué información obtienes de los elementos fuera de la diagonal?

Resumen del Capítulo

- Los **vectores** representan observaciones; las **matrices**, datasets completos
- El **producto punto** mide similitud entre vectores y proyecta uno sobre otro
- Los sistemas de ecuaciones se resuelven con ``np.linalg.solve``
 - Los **eigenvalores/eigenvectores** revelan la estructura fundamental de una matriz
 - **SVD** descompone cualquier matriz en componentes ortonormales (base de PCA)
 - La **matriz de covarianza** captura las relaciones entre variables
 - La **ecuación normal** ($\beta = (X^T X)^{-1} X^T y$) es la solución cerrada de regresión lineal

Conceptos clave aprendidos: vector, matriz, producto punto, SVD, eigenvalores, eigenvectores, covarianza, ecuación normal

Cálculo Diferencial Básico: Tasas de Cambio y Optimización

El cálculo diferencial es la matemática del cambio. En análisis de datos lo usamos constantemente: cada vez que un modelo "aprende" ajustando sus parámetros, está haciendo cálculo diferencial. Este capítulo te dará la intuición y las herramientas para entender ese proceso.

Derivadas: La Tasa de Cambio Instantánea

La derivada de una función $f(x)$ en un punto x_0 mide cómo cambia f cuando x cambia infinitesimalmente:

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

Derivada Numérica

```
import numpy as np
import matplotlib.pyplot as plt

def derivada_numerica(f, x, h=1e-7):
    """Aproximación de la derivada por diferencias finitas"""
    return (f(x + h) - f(x - h)) / (2 * h)

# Función cuadrática
def f(x):
    return x**2

# Derivada analítica: f'(x) = 2x
for x in [0, 1, 2, 3]:
    num = derivada_numerica(f, x)
    print(f"f'({x}) ≈ {num:.6f} (analítica: {2*x})")

# f'(0) ≈ 0.000000 (analítica: 0)
# f'(1) ≈ 2.000000 (analítica: 2)
# f'(2) ≈ 4.000000 (analítica: 4)
# f'(3) ≈ 6.000000 (analítica: 6)
```

> Consejo del Analista:

> Siempre que puedas, deriva analíticamente con SymPy y luego convierte a NumPy con ``lambdify``. Es más preciso que la diferenciación numérica y solo un poco más lento en tiempo de desarrollo.

Derivadas con SymPy (Analíticas)

```
import sympy as sp

x = sp.symbols('x')
f = x**3 * sp.exp(x)
derivada = sp.diff(f, x)
print(f"f'(x) = {derivada}")
# f'(x) = x**3*exp(x) + 3*x**2*exp(x)

# Evaluación numérica
f_prime = sp.lambdify(x, derivada, 'numpy')
print(f"f'(2) = {f_prime(2):.4f}")
# f'(2) = 59.1124
```

Gradiente: La Derivada en Múltiples Dimensiones

Cuando tenemos múltiples variables, la derivada se convierte en gradiente: un vector de derivadas parciales.

```
import sympy as sp

x, y = sp.symbols('x y')
f = x**2 + y**2 + 3*x*y

# Gradiente (vector de derivadas parciales)
grad = [sp.diff(f, var) for var in [x, y]]
print(f"Gradiente: ∂f/∂x = {grad[0]}, ∂f/∂y = {grad[1]}")
# Gradiente: ∂f/∂x = 2*x + 3*y, ∂f/∂y = 2*y + 3*x
```

El gradiente apunta en la dirección de máximo crecimiento de la función. Por eso el *descenso de gradiente* se mueve en dirección opuesta al gradiente para minimizar.

Descenso de Gradiente: Cómo Aprenden los Modelos

El descenso de gradiente es el algoritmo de optimización más importante en machine learning. Ajusta iterativamente los parámetros de un modelo para minimizar el error.

Implementación desde Cero

```
import numpy as np

def descenso_gradiente(grad_f, x0, lr=0.1, n_iter=100):
    """
    grad_f: función que calcula el gradiente en un punto
    x0: punto inicial
    lr: learning rate (tasa de aprendizaje)
    n_iter: número de iteraciones
    """
    x = np.array(x0, dtype=float)
    trayectoria = [x.copy()]

    for i in range(n_iter):
        grad = grad_f(x)
```

```

        x = x - lr * grad
        trayectoria.append(x.copy())

    return x, trayectoria

# Minimizar f(x, y) = x^2 + y^2 (mínimo en (0,0))
def grad_f(p):
    x, y = p
    return np.array([2*x, 2*y])

minimo, trayectoria = descenso_gradiente(grad_f, [5.0, 5.0], lr=0.1, n_iter=20)
print(f"Mínimo encontrado: {np.round(minimo, 6)}")
# Mínimo encontrado: [0.135335 0.135335]

# Error cuadrático medio (función de pérdida típica)
def grad_mse(X, y, beta):
    """Gradiente del MSE para regresión lineal"""
    n = len(y)
    error = X @ beta - y
    return (2/n) * X.T @ error

# Regresión lineal con descenso de gradiente
np.random.seed(42)
X = np.random.randn(100, 2)
y = 3 + 2*X[:, 0] - 1.5*X[:, 1] + np.random.randn(100)*0.3

# Añadir columna de 1s
X_b = np.column_stack([np.ones(100), X])
beta = np.zeros(3)

for _ in range(1000):
    beta = beta - 0.01 * grad_mse(X_b, y, beta)

print(f"β = {np.round(beta, 3)}")
# β ≈ [3.0  2.0 -1.5]

```

> Advertencia:

> Un learning rate mal ajustado es la causa más común de fallo en el entrenamiento de modelos. Monitorea siempre la función de pérdida durante el entrenamiento: si oscila o diverge, reduce el learning rate. Si converge muy lento, auméntalo.

El Problema del Learning Rate

La tasa de aprendizaje (learning rate) es crítica:

- **Demasiado grande:** el algoritmo diverge (no converge)
- **Demasiado pequeño:** converge muy lentamente
- **Bien ajustado:** converge eficientemente

```

# Demostración de learning rate
def grad_f_simple(x):
    return 2 * x # Derivada de x^2

```

```

for lr in [0.01, 0.1, 1.0]:
    x = 10.0
    for _ in range(10):
        x = x - lr * grad_f_simple(x)
    print(f"LR={lr:.2f}: x final = {x:.6f}")
# LR=0.01: x final = 8.170730
# LR=0.10: x final = 1.073742
# LR=1.00: x final = 10.000000 (oscila sin converger)

```

Regla de la Cadena y Retropropagación

La regla de la cadena es fundamental para entender cómo se entrenan las redes neuronales:

$$\partial z / \partial x = (\partial z / \partial y) * (\partial y / \partial x)$$

```

import sympy as sp

x = sp.symbols('x')
y = x**2 + 3*x      # y = g(x)
z = sp.sin(y)       # z = f(y)

# Regla de la cadena: dz/dx = dz/dy * dy/dx
dz_dy = sp.diff(z, y)
dy_dx = sp.diff(y, x)
dz_dx = dz_dy * dy_dx

print(f"dz/dx = {dz_dx}")
# dz/dx = (2*x + 3)*cos(x**2 + 3*x)

# Verificación directa
print(f"Directa: {sp.diff(z, x)}")
# Directa: (2*x + 3)*cos(x**2 + 3*x)

```

Jacobiano y Hessiano

Matriz Jacobiana

Cuando una función tiene múltiples entradas y múltiples salidas, el gradiente se generaliza a la matriz Jacobiana:

```

import sympy as sp

x, y = sp.symbols('x y')
f1 = x**2 + y**2
f2 = x*y

# Jacobiana
J = sp.Matrix([f1, f2]).jacobian([x, y])
print(f"Jacobiana:\n{J}")
# [[2*x, 2*y],
#  [ y,   x]]

```

Matriz Hessiana

La Hessiana contiene las segundas derivadas parciales. Determina si un punto crítico es mínimo, máximo o punto de silla:

```
f = x**3 - 3*x*y**2 # Función de ejemplo

# Hessiana
H = sp.hessian(f, (x, y))
print(f"Hessiana:\n{H}")
# [[6*x, -6*y],
#  [-6*y, -6*x]]
```

Aplicación Práctica: Optimización de una Función de Costo

```
from scipy import optimize

# Función de costo tipo Rosenbrock
def costo(params):
    x, y = params
    return (1 - x)**2 + 100 * (y - x**2)**2

# Gradiente de la función de costo
def grad_costo(params):
    x, y = params
    dx = -2*(1 - x) - 400*x*(y - x**2)
    dy = 200*(y - x**2)
    return np.array([dx, dy])

# Mínimo en (1, 1)
resultado = optimize.minimize(costo, [0, 0], method='BFGS', jac=grad_costo)
print(f"Mínimo: {np.round(resultado.x, 6)}")
# Mínimo: [1. 1.]
print(f"Valor: {resultado.fun:.2e}")
# Valor: 0.00e+00
```

Las funciones de pérdida en machine learning (MSE, cross-entropy, etc.) son versiones a gran escala de este mismo problema.

Derivadas Parciales en la Práctica

Entender derivadas parciales te permite interpretar coeficientes de modelos:

```
# En regresión lineal:  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$ 
#  $\partial y / \partial x_1 = \beta_1 \rightarrow$  "cambio en y por unidad de cambio en  $x_1$ "
```

Cada coeficiente en una regresión es una derivada parcial: mide la sensibilidad de la salida ante cambios en esa variable.

Ejercicios Prácticos

1. **Ejercicio 1:** Calcula la derivada numérica de $f(x) = \sin(x^2)$ en $x=2$ usando diferencias finitas

con $h=0.1, 0.01, 0.001, 0.0001$. Compara con el valor analítico. ¿Qué h da mejor resultado?

- Pista: $f'(x) = (f(x+h) - f(x-h)) / (2h)$

2. **Ejercicio 2:** Implementa el descenso de gradiente para minimizar $f(x,y) = x^2 + 2y^2 + xy$.

Parte de (3, 3) con learning rate 0.1. ¿En cuántas iteraciones converge?

3. **Ejercicio 3:** Usando SymPy, calcula el gradiente y la matriz Hessiana de $f(x,y,z) = x^2y + y^2z + z^2x$. Evalúa en el punto (1, -1, 0).

4. **Ejercicio 4:** Explica con tus palabras la relación entre la regla de la cadena y la retropropagación en redes neuronales. ¿Por qué es importante poder descomponer la derivada de una función compuesta?

Resumen del Capítulo

- La **derivada** mide la tasa de cambio instantánea de una función
- El **gradiente** generaliza la derivada a múltiples dimensiones (vector de derivadas parciales)
- El **descenso de gradiente** minimiza funciones moviéndose en dirección opuesta al gradiente
- El **learning rate** controla el tamaño del paso; muy grande diverge, muy pequeño es lento
- La **regla de la cadena** permite descomponer derivadas de funciones compuestas (base de la retropropagación)
- El **Jacobiano** generaliza el gradiente para funciones vectoriales
- El **Hessiano** contiene las segundas derivadas; determina si un punto crítico es mínimo o máximo

Conceptos clave aprendidos: derivada, gradiente, descenso de gradiente, learning rate, regla de la cadena, Jacobiano, Hessiano, optimización

Módulo 2: Estadística Descriptiva y Probabilidad

Medidas de Tendencia Central, Dispersión y Forma

Antes de construir modelos, necesitas entender tus datos. La estadística descriptiva es el primer paso en cualquier análisis cuantitativo: reduce un dataset completo a unas pocas métricas que resumen su comportamiento.

Medidas de Tendencia Central

Responden a la pregunta: "¿cuál es el valor típico o central de mis datos?"

Media Aritmética (Promedio)

La medida más conocida, pero también la más sensible a valores extremos:

```
import numpy as np
import pandas as pd

datos = np.array([2, 3, 5, 7, 11, 13, 17, 100])

# Cálculo manual
media = np.sum(datos) / len(datos)
print(f"Media: {media:.2f}")
# Media: 19.75

# Con NumPy
print(f"Media (numpy): {np.mean(datos):.2f}")
# Media (numpy): 19.75
```

> Advertencia:

> La media es engañosa con distribuciones sesgadas. En el ejemplo anterior, el 100 infla la media hasta 19.75, cuando la mayoría de valores están por debajo de 20.

Mediana

El valor central cuando los datos están ordenados. Robusta a outliers:

```
# Mediana
mediana = np.median(datos)
print(f"Mediana: {mediana:.2f}")
# Mediana: 9.00

# Comparación con datos sin outlier
datos_limpios = np.array([2, 3, 5, 7, 11, 13, 17])
```

```
print(f"Media sin outlier: {np.mean(datos_limpios):.2f}")
print(f"Mediana sin outlier: {np.median(datos_limpios):.2f}")
# Media sin outlier: 8.29
# Mediana sin outlier: 7.00
```

Moda

El valor que más se repite. Útil para variables categóricas:

```
from scipy import stats

categorias = np.array(['A', 'B', 'A', 'C', 'A', 'B', 'A', 'C', 'C', 'C'])
moda_resultado = stats.mode(categorias)
print(f"Moda: {moda_resultado.mode} (frecuencia: {moda_resultado.count})")
# Moda: C (frecuencia: 4)
```

> Consejo del Analista:

> No hay una medida "mejor" de tendencia central. La media es eficiente pero frágil; la mediana es robusta pero ignora información; la moda es la única útil para datos nominales.

Comparación de las Tres Medidas

Medida	Robusta a outliers	Usa todos los datos	Tipo de datos
Media	No	Sí	Núméricos
Mediana	Sí	No (solo orden)	Núméricos ordinales
Moda	Sí	No (solo frecuencia)	Cualquier tipo

```
# Demostración de asimetría
np.random.seed(42)
simetrico = np.random.normal(50, 10, 1000)
sesgado = np.random.exponential(2, 1000)

for nombre, datos in [("Simétrico", simetrico), ("Sesgado (exponencial)", sesgado)]:
    print(f"{nombre}:")
    print(f"  Media: {np.mean(datos):.2f}")
    print(f"  Mediana: {np.median(datos):.2f}")
    print(f"  Diferencia: {np.mean(datos) - np.median(datos):.2f}\n")
# Simétrico: Media=49.98, Mediana=49.81, Diferencia=0.17
# Sesgado: Media=2.01, Mediana=1.40, Diferencia=0.61
```

Medidas de Dispersión

Responden a: "¿qué tan dispersos están los datos?"

Varianza y Desviación Estándar

```
# Varianza poblacional (divide por n)
var_pob = np.var(datos_limpios)

# Varianza muestral (divide por n-1, corrección de Bessel)
var_muestral = np.var(datos_limpios, ddof=1)

print(f"Varianza poblacional: {var_pob:.2f}")
```



```
print(f"Varianza muestral: {var_muestral:.2f}")
print(f"Desviación estándar muestral: {np.std(datos_limpios, ddof=1):.2f}")
# Varianza poblacional: 26.24
# Varianza muestral: 30.62
# Desviación estándar muestral: 5.53
```

> Consejo del Analista:

> Cuando trabajes con *muestras* (casi siempre), usa `ddof=1`. Cuando trabajes con toda la *población* (raro), `ddof=0`. Por defecto, NumPy asume población.

Rango Intercuartil (IQR)

El IQR mide la dispersión del 50% central de los datos. Robusto a outliers:

```
Q1 = np.percentile(datos_limpios, 25)
Q3 = np.percentile(datos_limpios, 75)
IQR = Q3 - Q1

print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
# Q1: 4.0, Q3: 12.0, IQR: 8.0

# Regla del IQR para detectar outliers
outliers = datos[(datos < Q1 - 1.5*IQR) | (datos > Q3 + 1.5*IQR)]
print(f"Outliers (IQR): {outliers}")
# Outliers (IQR): [100]
```

Rango y Rango Interdecílico

```
rango = np.max(datos) - np.min(datos)
print(f"Rango: {rango}")
# Rango: 98

# Rango interdecílico (10% a 90%)
D1, D9 = np.percentile(datos_limpios, [10, 90])
print(f"Rango interdecílico: {D9 - D1:.2f}")
# Rango interdecílico: 11.60
```

Medidas de Forma

Responden a: "¿cómo se distribuyen los datos alrededor del centro?"

Asimetría (Skewness)

Mide la simetría de la distribución:

```
from scipy import stats

datos_sesgo_pos = np.random.exponential(1, 1000)
datos_sesgo_neg = -np.random.exponential(1, 1000)
datos_simetricos = np.random.normal(0, 1, 1000)

print(f"Asimetría positiva: {stats.skew(datos_sesgo_pos):.2f}")
print(f"Asimetría negativa: {stats.skew(datos_sesgo_neg):.2f}")
print(f"Simétrica: {stats.skew(datos_simetricos):.2f}")
```

```
# Asimetría positiva: ~2.0 (cola a la derecha)
# Asimetría negativa: ~-2.0 (cola a la izquierda)
# Simétrica: ~0.0
```

Valor	Interpretación
0	Distribución simétrica
> 0	Cola a la derecha, media > mediana
< 0	Cola a la izquierda, media < mediana

Curtosis (Kurtosis)

Mide el "grosor" de las colas:

```
normal = np.random.normal(0, 1, 1000)
pesada = np.random.standard_t(3, 1000) # Distribución t con colas pesadas

print(f"Curtosis normal: {stats.kurtosis(normal, fisher=True):.2f}")
print(f"Curtosis colas pesadas: {stats.kurtosis(pesada, fisher=True):.2f}")
# Curtosis normal: ~0.0
# Curtosis colas pesadas: > 0
```

> Consejo del Analista:

> La curtosis de *Fisher* (por defecto) da 0 para la normal. La curtosis de *Pearson* (exceso + 3) da 3 para la normal. Siempre verifica cuál estás usando.

Aplicación Práctica: Resumen Automático de un Dataset

```
import pandas as pd
import numpy as np
from scipy import stats

def reporte_descriptivo(df, columna):
    """Genera un reporte descriptivo completo para una columna numérica"""
    datos = df[columna].dropna()

    reporte = {
        'n': len(datos),
        'Media': np.mean(datos),
        'Mediana': np.median(datos),
        'Moda': stats.mode(datos).mode[0] if len(stats.mode(datos).mode) > 0 else np.nan,
        'Desv. Est.': np.std(datos, ddof=1),
        'Varianza': np.var(datos, ddof=1),
        'Mínimo': np.min(datos),
        'Máximo': np.max(datos),
        'Q1 (25%)': np.percentile(datos, 25),
        'Q2 (50%)': np.percentile(datos, 50),
        'Q3 (75%)': np.percentile(datos, 75),
        'IQR': np.percentile(datos, 75) - np.percentile(datos, 25),
        'Asimetría': stats.skew(datos),
        'Curtosis': stats.kurtosis(datos, fisher=True),
        'Outliers (IQR)': int(np.sum(
```

```

        (datos < np.percentile(datos, 25) - 1.5*(np.percentile(datos, 75) - np.percentile(datos, 25)))
        (datos > np.percentile(datos, 75) + 1.5*(np.percentile(datos, 75) - np.percentile(datos, 25)))
    )),
    'Datos Faltantes': int(df[columna].isna().sum()),
}
return reporte

# Ejemplo con datos simulados
np.random.seed(42)
df = pd.DataFrame({
    'ingresos': np.random.exponential(50, 1000),
    'edad': np.random.normal(40, 12, 1000).clip(18, 90).astype(int),
    'puntuacion': np.random.uniform(0, 100, 1000),
})

reporte = reporte_descriptivo(df, 'ingresos')
for metrica, valor in reporte.items():
    print(f"{metrica:15s}: {valor:.2f}" if isinstance(valor, (int, float)) else f"{metrica:15s}: {valor}")

```

Ejercicios Prácticos

1. **Ejercicio 1:** Genera un dataset con 1000 valores de una distribución uniforme $U(0, 10)$. Calcula media, mediana, varianza y asimetría. ¿Qué observas sobre la asimetría?
 - Pista: Usa `np.random.uniform(0, 10, 1000)`
2. **Ejercicio 2:** Crea una función que detecte outliers usando tanto la regla del IQR como la desviación estándar (valores más allá de 3σ). Compáralas en un dataset con valores extremos.
3. **Ejercicio 3:** Toma el dataset de `sklearn.datasets.load_diabetes()` y calcula las medidas descriptivas para cada característica. ¿Qué variables tienen mayor asimetría?

Resumen del Capítulo

- La **media** es el promedio aritmético; sensible a outliers
- La **mediana** es robusta; recomendada para distribuciones sesgadas
- La **moda** es el valor más frecuente; única para datos nominales
- La **varianza** y **desviación estándar** miden dispersión
- El **IQR** es robusto a outliers; base para detección de anomalías
- La **asimetría** indica sesgo; la **curtosis** indica peso de colas
- Siempre reporta múltiples métricas; ninguna es suficiente por sí sola

Conceptos clave aprendidos: media, mediana, moda, varianza, desviación estándar, IQR, asimetría, curtosis, outliers

Teoría de la Probabilidad y Teorema de Bayes

La probabilidad es el lenguaje de la incertidumbre. En análisis de datos, cada predicción, cada intervalo de confianza y cada decisión basada en datos es una declaración probabilística. Este capítulo te da las bases para entender y cuantificar la incertidumbre.

Conceptos Fundamentales

Espacio Muestral y Eventos

El *espacio muestral* (Ω) es el conjunto de todos los resultados posibles. Un *evento* es un subconjunto del espacio muestral.

```
import numpy as np
from itertools import product

# Espacio muestral: lanzar dos dados
omega = list(product(range(1, 7), repeat=2))
print(f"Espacio muestral: {len(omega)} resultados posibles")
# Espacio muestral: 36 resultados posibles

# Evento A: suma = 7
evento_A = [resultado for resultado in omega if sum(resultado) == 7]
print(f"P(A) = {len(evento_A)}/{len(omega)} = {len(evento_A)/len(omega):.4f}")
# P(A) = 6/36 = 0.1667

# Evento B: ambos dados pares
evento_B = [r for r in omega if r[0] % 2 == 0 and r[1] % 2 == 0]
print(f"P(B) = {len(evento_B)}/{len(omega)} = {len(evento_B)/len(omega):.4f}")
# P(B) = 9/36 = 0.2500
```

Axiomas de Probabilidad

1. **No negatividad:** $P(A) \geq 0$
2. **Normalización:** $P(\Omega) = 1$
3. **Aditividad:** Si A y B son mutuamente excluyentes, $P(A \cup B) = P(A) + P(B)$

Probabilidad Condicional e Independencia

La probabilidad condicional responde a: "¿cómo cambia la probabilidad de A si sabemos que B

ocurrió?"

$$P(A|B) = P(A \cap B) / P(B)$$

```
# Probabilidad condicional: dado que la suma es 7,  
# ¿cuál es la probabilidad de que el primer dado sea 3?  
evento_A = [(1,6), (2,5), (3,4), (4,3), (5,2), (6,1)]  
evento_B = [(3,4), (3,5), (3,6), (3,1), (3,2), (3,3)]  
  
# Esto no es correcto porque los eventos no están  
# definidos sobre el espacio muestral completo.  
# Mejor:  
  
omega = list(product(range(1, 7), repeat=2))  
  
# A: suma = 7  
A = {r for r in omega if sum(r) == 7}  
# B: primer dado = 3  
B = {r for r in omega if r[0] == 3}  
  
# P(A|B) = P(A ∩ B) / P(B)  
A_int_B = A & B  
p_A_dado_B = len(A_int_B) / len(B)  
print(f"P(A|B) = {p_A_dado_B:.4f}")  
# P(A|B) = 0.1667 (de 6 resultados con primer dado=3, solo 1 suma 7)
```

Independencia

```
# ¿Son independientes "primer dado = 3" y "suma = 7"?  
p_A = len(A) / len(omega)  
p_B = len(B) / len(omega)  
p_A_int_B = len(A_int_B) / len(omega)  
  
print(f"P(A)·P(B) = {p_A * p_B:.4f}")  
print(f"P(A∩B) = {p_A_int_B:.4f}")  
print(f"¿Independientes? {np.isclose(p_A * p_B, p_A_int_B)}")  
# P(A)·P(B) = 0.0463  
# P(A∩B) = 0.0278  
# ¿Independientes? False
```

Teorema de Probabilidad Total

Para un conjunto de eventos $\{B\}$ que particionan el espacio muestral:

$$P(A) = \sum P(A|B_i) \cdot P(B_i)$$

```
# Ejemplo: tres urnas con bolas  
# Urna 1: 3 rojas, 7 azules (P=0.5)  
# Urna 2: 5 rojas, 5 azules (P=0.3)  
# Urna 3: 8 rojas, 2 azules (P=0.2)  
  
p_urna = [0.5, 0.3, 0.2]
```

```
p_roja_dada_urna = [3/10, 5/10, 8/10]

p_roja = sum(p * pr for p, pr in zip(p_urna, p_roja_dada_urna))
print(f"P(Roja) = {p_roja:.4f}")
# P(Roja) = 0.4600
```

Teorema de Bayes

Bayes es el teorema más importante que aprenderás como analista. Permite actualizar creencias (probabilidades) a la luz de nueva evidencia:

$$P(A|B) = P(B|A) \cdot P(A) / P(B)$$

Donde:

- $P(A)$: *probabilidad a priori* (antes de ver los datos)

Ejemplo Clásico: Prueba Médica

```
# Supongamos:
# - Enfermedad rara: P(E) = 0.01 (1% de la población)
# - Prueba detecta 95% de enfermos: P(+|E) = 0.95 (sensibilidad)
# - Prueba da falso positivo en 5%: P(+|no E) = 0.05 (1 - especificidad)

# Pregunta: si una persona da positivo, ¿cuál es la probabilidad
# de que realmente tenga la enfermedad?

p_enfermedad = 0.01
p_positivo_dado_enfermo = 0.95
p_positivo_dado_sano = 0.05

# Probabilidad total de positivo
p_positivo = (p_positivo_dado_enfermo * p_enfermedad +
              p_positivo_dado_sano * (1 - p_enfermedad))

# Teorema de Bayes
p_enfermedad_dado_positivo = (p_positivo_dado_enfermo * p_enfermedad) / p_positivo

print(f"P(Enfermo | +) = {p_enfermedad_dado_positivo:.4f} ({p_enfermedad_dado_positivo*100:.1f}%)")
# P(Enfermo | +) = 0.1610 (16.1%)
```

> Advertencia:

> Aunque la prueba parece confiable (95% de sensibilidad), la baja prevalencia (1%) hace que la mayoría de positivos sean falsos. Este es un error clásico de interpretación incluso entre médicos.

Implementación General de Bayes

```
def teorema_bayes(p_hipotesis, p_evidencia_dado_hipotesis, p_evidencia_dado_no_hipotesis):
    """
    Calcula P(Hipótesis | Evidencia)

    Parámetros:
    - p_hipotesis: P(H), probabilidad a priori
```

```

- p_evidencia_dado_hipotesis:  $P(E|H)$ , verosimilitud
- p_evidencia_dado_no_hipotesis:  $P(E|\neg H)$ 
"""

p_evidencia = (p_evidencia_dado_hipotesis * p_hipotesis +
                p_evidencia_dado_no_hipotesis * (1 - p_hipotesis))
return (p_evidencia_dado_hipotesis * p_hipotesis) / p_evidencia

# Aplicación a detección de spam
# - 20% de los emails son spam:  $P(\text{Spam}) = 0.20$ 
# - 90% de los spams contienen "oferta":  $P(\text{"oferta"}|\text{Spam}) = 0.90$ 
# - 10% de los emails legítimos contienen "oferta":  $P(\text{"oferta"}|\text{NoSpam}) = 0.10$ 

p_spam_dado_oferta = teorema_bayes(0.20, 0.90, 0.10)
print(f"P(Spam | 'oferta') = {p_spam_dado_oferta:.4f} ({p_spam_dado_oferta*100:.1f}%)")
# P(Spam | 'oferta') = 0.6923 (69.2%)

```

Bayes Múltiple: Actualización Secuencial

El poder de Bayes está en la actualización iterativa: la *posterior* de hoy es la *prior* de mañana:

```

def actualizar_bayes(p_prior, verosimilitud_positivo, verosimilitud_negativo, datos):
    """
    Actualiza secuencialmente P(H) con múltiples observaciones

    datos: lista de 'positivo' o 'negativo'
    """
    p = p_prior
    historial = [p]

    for resultado in datos:
        if resultado == 'positivo':
            p = teorema_bayes(p, verosimilitud_positivo, 1 - verosimilitud_positivo)
        else:
            p = teorema_bayes(p, 1 - verosimilitud_negativo, verosimilitud_negativo)
        historial.append(p)

    return p, historial

# Diagnosticar enfermedad con 3 pruebas
# Supongamos  $P(E)=0.5$  (inciertos), prueba tiene 90% sensibilidad y 90% especificidad
p_final, hist = actualizar_bayes(
    p_prior=0.5,
    verosimilitud_positivo=0.9,
    verosimilitud_negativo=0.1, # 1 - especificidad
    datos=['positivo', 'positivo', 'negativo']
)

for i, p in enumerate(hist):
    print(f"Después de observación {i}:  $P(E) = {p:.4f}$ ")
# Después de observación 0:  $P(E) = 0.5000$  (prior)
# Después de observación 1:  $P(E) = 0.9000$ 
# Después de observación 2:  $P(E) = 0.9878$ 

```

```
# Después de observación 3:  $P(E) = 0.9000$  (una negativa reduce la certeza)
```

Aplicación en Machine Learning: Naive Bayes

El clasificador *Naive Bayes* aplica el teorema de Bayes con el supuesto (ingenuo) de independencia entre características:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Cargar datos
iris = load_iris()
X, y = iris.data, iris.target

# Dividir
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Entrenar Naive Bayes
modelo = GaussianNB()
modelo.fit(X_train, y_train)

# Predecir
y_pred = modelo.predict(X_test)
precision = np.mean(y_pred == y_test)
print(f"Precisión: {precision:.2%}")
# Precisión: ~96%

# Probabilidades a posteriori (esto es Bayes puro)
probabilidades = modelo.predict_proba(X_test[:3])
print("Probabilidades a posteriori para 3 muestras:")
print(np.round(probabilidades, 3))
# [[1.000 0.000 0.000] -> 100% clase 0
#  [0.000 0.999 0.001] -> 99.9% clase 1
#  [0.000 0.015 0.985]] -> 98.5% clase 2
```

Ejercicios Prácticos

- Ejercicio 1:** Escribe una función que simule el lanzamiento de una moneda 1000 veces y compare la frecuencia relativa de caras con la probabilidad teórica (0.5). ¿Qué observas al aumentar el número de lanzamientos?
- Ejercicio 2:** Un test detecta una condición con 99% de sensibilidad y 95% de especificidad. Si la condición afecta al 0.5% de la población, ¿cuál es la probabilidad de tener la condición si el test da positivo?
- Ejercicio 3:** Implementa un clasificador Naive Bayes desde cero (sin scikit-learn) para el

dataset Iris y compáralo con el de sklearn. Usa el teorema de Bayes con el supuesto de normalidad de las características.

Resumen del Capítulo

- La **probabilidad** cuantifica la incertidumbre entre 0 y 1
- Dos eventos son **independientes** si $P(AB) = P(A) \cdot P(B)$
- El **Teorema de Probabilidad Total** descompone una probabilidad en casos
- El **Teorema de Bayes** es el mecanismo formal para actualizar creencias con datos
- La **prior** se actualiza a **posterior** mediante la **verosimilitud**
- **Naive Bayes** aplica Bayes con independencia condicional entre características
- La actualización secuencial permite incorporar nueva evidencia progresivamente

Conceptos clave aprendidos: espacio muestral, evento, probabilidad condicional, independencia, Teorema de Probabilidad Total, Teorema de Bayes, prior, verosimilitud, posterior, Naive Bayes

Distribuciones de Probabilidad Clave

Una distribución de probabilidad describe cómo se comporta una variable aleatoria. Es el puente entre los datos que ves y el proceso que los generó. Conocer distribuciones es reconocer patrones universales en los datos.

Variables Aleatorias Discretas vs Continuas

Tipo	Valores	Ejemplo	Distribuciones típicas
Discreta	Conjunto contable	Número de clientes	Binomial, Poisson
Continua	Intervalo real	Altura, tiempo, precio	Normal, Exponencial

Distribuciones Discretas

Distribución Bernoulli

Modela un experimento con dos resultados: éxito (1) o fracaso (0).

```
from scipy import stats
import numpy as np

# Bernoulli: p = 0.7 (70% de éxito)
p = 0.7
bernoulli = stats.bernoulli(p)

# Probabilidad de cada resultado
print(f"P(X=0) = {bernoulli.pmf(0):.3f}") # 0.300
print(f"P(X=1) = {bernoulli.pmf(1):.3f}") # 0.700

# Media y varianza
print(f"Media: {bernoulli.mean():.3f}, Varianza: {bernoulli.var():.3f}")
# Media: 0.700, Varianza: 0.210

# Simular 10 lanzamientos
muestra = bernoulli.rvs(size=10, random_state=42)
print(f"Muestra: {muestra}")
# Muestra: [1 1 0 1 1 1 1 1 1 1]
```

Distribución Binomial

Modela el número de éxitos en n ensayos independientes de Bernoulli.

```
# Binomial: n=20, p=0.3
```

```

n, p = 20, 0.3
binomial = stats.binom(n, p)

# P(X = 6) - probabilidad de exactamente 6 éxitos
print(f"P(X=6) = {binomial.pmf(6):.4f}")
# P(X=6) = 0.1916

# P(X ≤ 6) - probabilidad acumulada
print(f"P(X ≤ 6) = {binomial.cdf(6):.4f}")
# P(X ≤ 6) = 0.6077

# Media: np, Varianza: np(1-p)
print(f"Media: {binomial.mean():.1f}, Varianza: {binomial.var():.2f}")
# Media: 6.0, Varianza: 4.20

# Simulación
simulacion = binomial.rvs(size=10000, random_state=42)
print(f"Frecuencia relativa de 6 éxitos: {np.mean(simulacion == 6):.4f}")
# ~0.19 (cercano a la pmf teórica)

```

Distribución Poisson

Modela el número de eventos en un intervalo de tiempo fijo (llamadas por hora, accidentes por día).

```

# Poisson:  $\lambda = 3$  (promedio de 3 eventos por unidad de tiempo)
lam = 3.0
poisson = stats.poisson(lam)

# P(X = 2)
print(f"P(X=2) = {poisson.pmf(2):.4f}")
# P(X=2) = 0.2240

# P(X ≤ 2)
print(f"P(X ≤ 2) = {poisson.cdf(2):.4f}")
# P(X ≤ 2) = 0.4232

# Media = Varianza =  $\lambda$ 
print(f"Media: {poisson.mean():.1f}, Varianza: {poisson.var():.1f}")
# Media: 3.0, Varianza: 3.0

# Comparación Binomial vs Poisson para n grande, p pequeño
n, p = 1000, 0.005 #  $\lambda = np = 5$ 
binom = stats.binom(n, p)
poiss = stats.poisson(n * p)

x = np.arange(0, 15)
print("Comparación Binomial(1000, 0.005) vs Poisson(5):")
for xi in x[:5]:
    print(f" P(X={xi}): Binom={binom.pmf(xi):.4f}, Poisson={poiss.pmf(xi):.4f}")
# P(X=0): Binom=0.0067, Poisson=0.0067
# P(X=1): Binom=0.0336, Poisson=0.0337
# P(X=2): Binom=0.0840, Poisson=0.0842

```

> Consejo del Analista:

> La distribución Poisson aproxima la Binomial cuando n es grande (>100) y p es pequeño (<0.01). Esto es útil porque Poisson solo depende de $\lambda = np$, simplificando los cálculos.

Distribuciones Continuas

Distribución Normal (Gaussiana)

La distribución más importante en estadística. Descrita por dos parámetros: media μ y desviación estándar σ .

```
# Normal estándar: N(0, 1)
normal_std = stats.norm(0, 1)

# Percentiles clave (para intervalos de confianza)
print(f"z(0.975) = {normal_std.ppf(0.975):.4f}") # 1.9600
print(f"z(0.995) = {normal_std.ppf(0.995):.4f}") # 2.5758
print(f"z(0.841) = {normal_std.ppf(0.8413):.4f}") # 1.0000

# Valores personalizados
media, desv = 100, 15 # Ejemplo: CI
normal_personalizada = stats.norm(media, desv)

# ¿Qué porcentaje tiene CI entre 85 y 115?
p_85_115 = normal_personalizada.cdf(115) - normal_personalizada.cdf(85)
print(f"P(85 ≤ X ≤ 115) = {p_85_115:.4f} ({p_85_115*100:.1f}%)")
# P(85 ≤ X ≤ 115) = 0.6827 (68.3%)

# Regla empírica (68-95-99.7)
for k, pct in [(1, 68.27), (2, 95.45), (3, 99.73)]:
    p = normal_personalizada.cdf(media + k*desv) - normal_personalizada.cdf(media - k*desv)
    print(f" ±{k}σ: {p*100:.1f}% (teórico: {pct}%)")
# ±1σ: 68.3% (teórico: 68.27%)
# ±2σ: 95.4% (teórico: 95.45%)
# ±3σ: 99.7% (teórico: 99.73%)
```

Distribución Exponencial

Modela el tiempo entre eventos en un proceso Poisson (tiempo hasta la próxima llamada, vida útil de un componente).

```
# Exponencial: λ = 0.5 (tasa de 0.5 eventos por unidad de tiempo)
lam = 0.5
exponencial = stats.expon(scale=1/lam)

# Tiempo promedio entre eventos: 1/λ = 2
print(f"Media: {exponencial.mean():.2f}")
# Media: 2.00

# P(X > 3) - probabilidad de esperar más de 3 unidades
p_mayor_3 = 1 - exponencial.cdf(3)
print(f"P(X > 3) = {p_mayor_3:.4f}")
# P(X > 3) = 0.2231
```

```
# Simular tiempos de espera
muestra_exp = exponencial.rvs(size=10, random_state=42)
print(f"Tiempos de espera: {np.round(muestra_exp, 2)}")
# Tiempos de espera: [1.46 0.42 2.56 0.11 2.69 ...]
```

Distribución t de Student

Similar a la normal pero con colas más pesadas. Se usa cuando el tamaño de muestra es pequeño y la varianza poblacional es desconocida.

```
# Comparación t-student vs normal
from scipy import stats

normal = stats.norm(0, 1)
t_3 = stats.t(df=3)    # 3 grados de libertad
t_30 = stats.t(df=30)  # 30 gl (casi normal)

print("Percentil 97.5%:")
print(f" Normal: {normal.ppf(0.975):.4f}")
print(f" t(3):   {t_3.ppf(0.975):.4f}")
print(f" t(30):  {t_30.ppf(0.975):.4f}")
# Normal: 1.9600
# t(3):   3.1824 (colas más pesadas → intervalo más amplio)
# t(30):  2.0423 (casi normal)

# A medida que df → ∞, t → Normal
```

> Advertencia:

> Con pocos grados de libertad (digamos, $df < 30$), la t-student produce intervalos de confianza más amplios que la normal. Esto es correcto: refleja la mayor incertidumbre cuando tenemos pocos datos.

Distribución Chi-cuadrado (χ^2)

Modela la suma de cuadrados de normales estándar independientes. Se usa en pruebas de independencia y bondad de ajuste.

```
chi2 = stats.chi2(df=5) # 5 grados de libertad

print(f"Media: {chi2.mean():.1f}") # 5 (igual a df)
print(f"Varianza: {chi2.var():.1f}") # 10 (2*df)

# Valor crítico para  $\alpha = 0.05$ 
print(f" $\chi^2(0.95, df=5) = {chi2.ppf(0.95):.4f}")$ )
#  $\chi^2(0.95, df=5) = 11.0705$ 
```

Distribución F de Fisher

Modela el cociente de dos chi-cuadrados independientes. Fundamental para ANOVA.

```
f_dist = stats.f(dfn=3, dfd=20) # 3 y 20 grados de libertad

print(f"Media: {f_dist.mean():.4f}") #  $dfd/(dfd-2) \approx 1.11$ 
```

```
# Valor crítico para  $\alpha = 0.05$ 
print(f"F(0.95, 3, 20) = {f_dist.ppf(0.95):.4f}")
# F(0.95, 3, 20) = 3.0984
```

Cómo Elegir la Distribución Correcta

Esta tabla te ayuda a seleccionar:

Situación	Distribución	Parámetros clave
¿Éxito/fracaso en un ensayo?	Bernoulli	p
¿Éxitos en n ensayos?	Binomial	n, p
¿Eventos raros en intervalo fijo?	Poisson	λ
¿Datos simétricos, muchos valores?	Normal	μ, σ
¿Tiempo entre eventos?	Exponencial	λ (tasa)
¿Muestra pequeña, varianza desconocida?	t-Student	df
¿Bondad de ajuste/independencia?	χ^2	df
¿Comparación de varianzas/ANOVA?	F	dfn, dfd

Verificación de Distribuciones (Q-Q Plot)

Un *Q-Q plot* compara los cuantiles de tus datos con los de una distribución teórica:

```
import matplotlib.pyplot as plt
from scipy import stats

# Generar datos que NO son normales
np.random.seed(42)
datos_exponencial = np.random.exponential(2, 100)

# Q-Q plot contra normal
stats.probplot(datos_exponencial, dist="norm", plot=plt)
plt.title("Q-Q Plot: Datos Exponenciales vs Normal")
plt.grid(True, alpha=0.3)
# Los puntos se desvían de la línea recta → no es normal

# Para datos normales
datos_normales = np.random.normal(0, 1, 100)
stats.probplot(datos_normales, dist="norm", plot=plt)
plt.title("Q-Q Plot: Datos Normales vs Normal")
plt.grid(True, alpha=0.3)
# Los puntos siguen la línea recta → es normal
```

Ejercicios Prácticos

1. **Ejercicio 1:** Simula 1000 lanzamientos de una moneda (Bernoulli $p=0.5$). Calcula la media muestral y compárala con la media teórica. Repite con $p=0.1$, $p=0.9$. ¿Qué observas sobre la varianza?

2. **Ejercicio 2:** Un call center recibe un promedio de 5 llamadas por hora. ¿Cuál es la probabilidad de recibir exactamente 3 llamadas en una hora? ¿Y más de 8? Usa la distribución

Poisson.

3. **Ejercicio 3:** Genera 100 datos de una distribución uniforme $U(0, 1)$. Aplica la transformada de Box-Muller para convertirla en normal. Compara el histograma resultante con la distribución normal teórica.

4. **Ejercicio 4:** Toma el dataset `iris` y genera Q-Q plots para cada característica. ¿Cuáles parecen distribuirse normalmente?

Resumen del Capítulo

- Las distribuciones **discretas** (Bernoulli, Binomial, Poisson) modelan conteos
- Las distribuciones **continuas** (Normal, Exponencial, t , χ^2 , F) modelan mediciones
- La **Normal** es la distribución central por el TLC; todo analista debe conocerla
- La **t-Student** reemplaza a la normal con muestras pequeñas
- La **Poisson** modela eventos raros; aproxima la Binomial con n grande y p pequeño
- La **Exponencial** modela tiempos entre eventos
- El **Q-Q plot** verifica visualmente si los datos siguen una distribución

Conceptos clave aprendidos: función de masa (pmf), función de densidad (pdf), función de distribución (cdf), Normal, Binomial, Poisson, Exponencial, t-Student, Chi-cuadrado, Q-Q plot

El Teorema del Límite Central

Si solo pudieras recordar un teorema de toda la estadística, este debería serlo. El Teorema del Límite Central (TLC) es la razón por la que podemos hacer inferencia estadística, construir intervalos de confianza y probar hipótesis sin conocer la distribución subyacente de los datos.

La Idea Intuitiva

El TLC dice: **la distribución de la media muestral se aproxima a una normal a medida que el tamaño de muestra crece, independientemente de la forma de la distribución original.**

No importa si tus datos vienen de una distribución exponencial, uniforme, binomial o incluso de una mezcla extraña: el promedio de muchas observaciones se comportará como una normal.

Demostración Práctica

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(42)

# Población con distribución exponencial (muy sesgada)
poblacion = np.random.exponential(scale=2, size=100000)

print(f"Media poblacional: {np.mean(poblacion):.3f}")
print(f"Asimetría poblacional: {stats.skew(poblacion):.3f}")
# Media poblacional: 2.003
# Asimetría poblacional: ~2.0 (muy sesgada)

# Función para simular el TLC
def simular_tlc(poblacion, n_muestra, n_simulaciones=10000):
    medias_muestrales = []

    for _ in range(n_simulaciones):
        muestra = np.random.choice(poblacion, size=n_muestra)
        medias_muestrales.append(np.mean(muestra))

    return np.array(medias_muestrales)

# Probar con diferentes tamaños de muestra
for n in [2, 5, 30, 100]:
```



```

medias = simular_tlc(poblacion, n, n_simulaciones=10000)

media_teorica = np.mean(poblacion)
error_std_teorico = np.std(poblacion) / np.sqrt(n)

print(f"\nn = {n}:")
print(f"  Media muestral: {np.mean(medias):.3f} (teórica: {media_teorica:.3f})")
print(f"  Desv. media muestral: {np.std(medias):.3f} (teórica: {error_std_teorico:.3f})")
print(f"  Asimetría: {stats.skew(medias):.3f}")

# n = 2:
#  Media muestral: 2.002 (teórica: 2.003)
#  Desv. media muestral: 1.413 (teórica: 1.414)
#  Asimetría: 1.444 (todavía sesgada)
#
# n = 5:
#  Media muestral: 2.001 (teórica: 2.003)
#  Desv. media muestral: 0.889 (teórica: 0.894)
#  Asimetría: 0.872 (menos sesgada)
#
# n = 30:
#  Media muestral: 2.000 (teórica: 2.003)
#  Desv. media muestral: 0.362 (teórica: 0.365)
#  Asimetría: 0.301 (casi simétrica)
#
# n = 100:
#  Media muestral: 2.002 (teórica: 2.003)
#  Desv. media muestral: 0.199 (teórica: 0.200)
#  Asimetría: 0.152 (muy cercana a normal)

```

> Consejo del Analista:

> La "magia" ocurre alrededor de $n=30$. Con muestras menores, la distribución de la media aún refleja la forma de la población original. Con $n \geq 30$, la normalidad es una aproximación razonable en la mayoría de los casos prácticos.

La Fórmula del TLC

$$\bar{X} \sim N(\mu, \sigma/\sqrt{n})$$

Donde:

- $\bar{X}$: media muestral
- μ : media poblacional
- σ : desviación estándar poblacional
- n : tamaño de muestra

El error estándar ($SE = \sigma/\sqrt{n}$) disminuye con la raíz cuadrada del tamaño muestral. Para reducir el error a la mitad, necesitas cuadruplicar la muestra.

```

# Relación entre n y error estándar
sigma = 10
for n in [10, 40, 100, 400, 1000]:
    se = sigma / np.sqrt(n)

```

```

print(f"n={n:4d}: SE = {se:.2f}")
# n= 10: SE = 3.16
# n= 40: SE = 1.58
# n= 100: SE = 1.00
# n= 400: SE = 0.50
# n=1000: SE = 0.32

```

El TLC Explica Tres Fenómenos Clave

1. Por qué los promedios son normales

Aunque los datos individuales no lo sean, el promedio de muchos sí lo es. Esto justifica el uso de la media como estadístico.

2. Por qué importa el tamaño muestral

Muestras más grandes distribución de la media más estrecha estimaciones más precisas.

3. Por qué la variabilidad se reduce con $\sqrt{n}$

No es lineal: para duplicar precisión necesitas 4x los datos. Esto tiene implicaciones prácticas en diseño experimental y A/B testing.

El TLC en Acción: Distribuciones No Normales

```

# Demostración con múltiples distribuciones
distribuciones = {
    'Uniforme': np.random.uniform(0, 10, 100000),
    'Exponencial': np.random.exponential(2, 100000),
    'Binomial (p=0.1)': np.random.binomial(1, 0.1, 100000),
    'Mixta': np.concatenate([
        np.random.normal(-5, 1, 50000),
        np.random.normal(5, 1, 50000)
    ])
}

from scipy import stats

n = 30

for nombre, poblacion in distribuciones.items():
    medias = [np.mean(np.random.choice(poblacion, n))
               for _ in range(10000)]

    print(f"{nombre}:")
    print(f"  Asimetría original: {stats.skew(poblacion):.2f}")
    print(f"  Asimetría de medias (n={n}): {stats.skew(medias):.2f}")
    print(f"  ¿Normal? (p-valor Shapiro): {stats.shapiro(medias[:5000]).pvalue:.4f}\n")
# Uniforme:
#   Asimetría original: 0.00
#   Asimetría de medias (n=30): 0.00
#
# Exponencial:

```

```
# Asimetría original: 2.00
# Asimetría de medias (n=30): 0.30
#
# Binomial (p=0.1):
# Asimetría original: 2.67
# Asimetría de medias (n=30): 0.48
#
# Mixta (bimodal):
# Asimetría original: 0.00
# Asimetría de medias (n=30): 0.00 (¡incluso bimodal!)
```

Aplicación: Tamaño de Muestra para una Encuesta

```
# ¿Cuántas personas necesitas encuestar para estimar
# la intención de voto con margen de error  $\pm 3\%$  (IC 95%)?

# Para una proporción, el error estándar máximo es con  $p=0.5$ 
#  $SE = \sqrt{p(1-p)/n}$ 
# Margen =  $1.96 * SE$ 

def tamano_muestra(margen=0.03, confianza=0.95, p=0.5):
    """Calcula n necesario para cierto margen de error"""
    z = stats.norm.ppf(1 - (1 - confianza) / 2)
    n = (z**2 * p * (1 - p)) / (margen**2)
    return int(np.ceil(n))

print(f"n necesaria para margen  $\pm 3\%$ : {tamano_muestra(0.03)}")
print(f"n necesaria para margen  $\pm 5\%$ : {tamano_muestra(0.05)}")
print(f"n necesaria para margen  $\pm 1\%$ : {tamano_muestra(0.01)}")

# n necesaria para margen  $\pm 3\%$ : 1068
# n necesaria para margen  $\pm 5\%$ : 385
# n necesaria para margen  $\pm 1\%$ : 9604
```

El TLC y su Hermano: La Ley de los Grandes Números

El TLC se confunde a menudo con la Ley de los Grandes Números (LGN). Son complementarias:

Ley	Dice	Ejemplo
LGN	La media muestral converge a la media poblacional	En los lanzamientos, la frecuencia de cara
TLC	La distribución de la media muestral se distribuye normalmente alrededor de	La frecuencia se distribuye normalmente alrededor de

Limitaciones del TLC

El TLC no es una solución universal:

1. **Colas extremadamente pesadas:** Distribuciones como Cauchy (sin media definida) no cumplen el TLC
2. **Dependencia:** Los datos deben ser (aproximadamente) independientes
3. **Muestras muy pequeñas:** Con $n < 5-10$, la aproximación puede ser pobre
4. **Tamaño de muestra vs población:** La muestra debe ser pequeña comparada con la

población

```
# Demostración: la distribución de Cauchy NO cumple el TLC
from scipy import stats

cauchy = stats.cauchy()
muestras_cauchy = cauchy.rvs(size=(1000, 100))
medias_cauchy = np.mean(muestras_cauchy, axis=1)

print(f"Media de las medias: {np.mean(medias_cauchy):.2f}")
print(f"Desv. de las medias: {np.std(medias_cauchy):.2f}")
print(f"¿Normal? Shapiro p-valor: {stats.shapiro(medias_cauchy[:5000]).pvalue:.4f}")
# Media de las medias: -0.18 (inestable, no converge)
# Desv. de las medias: 2.51 (no se reduce con √n)
# p-valor < 0.001 → definitivamente no normal
```

> **Advertencia:**

> No asumas automáticamente normalidad solo porque $n > 30$. Verifica siempre con Q-Q plots o tests de normalidad. El TLC dice que la *media* es normal, no que los *datos individuales* lo sean.

Ejercicios Prácticos

1. **Ejercicio 1:** Toma la distribución uniforme $U(0,1)$. Simula 5000 experimentos obteniendo la media de $n=2$, $n=5$, $n=15$, $n=30$. Para cada n , genera un histograma de las medias. ¿En qué n empieza a verse normal?
2. **Ejercicio 2:** Explica con tus palabras: ¿por qué el margen de error en encuestas se reduce con la raíz cuadrada del tamaño de muestra? Si una encuesta con 1000 personas tiene $\pm 3\%$, ¿cuántas personas necesitas para $\pm 1\%$?
3. **Ejercicio 3:** Genera una población con distribución bimodal (mezcla de dos normales). Demuestra que la distribución de la media muestral con $n=30$ es aproximadamente normal, aunque la población no lo sea.
4. **Ejercicio 4:** Encuentra un dataset real (por ejemplo, precios de viviendas) y demuestra empíricamente el TLC: la media de muestras de tamaño 50 se distribuye aproximadamente normal aunque los precios tengan una distribución sesgada.

Resumen del Capítulo

- El **TLC** establece que la media muestral se distribuye normalmente para n grande
- La **media** de la distribución de medias muestrales = μ poblacional
- El **error estándar** ($\sigma/\sqrt{n}$) disminuye con la raíz cuadrada de n
- El TLC funciona independientemente de la distribución original
 - La **LGN** (convergencia) y el **TLC** (forma de la distribución) son complementarios
- $n \geq 30$ es una regla práctica, no un límite mágico
- El TLC tiene limitaciones: datos dependientes, colas infinitas, muestras muy pequeñas

Conceptos clave aprendidos: Teorema del Límite Central, error estándar, Ley de Grandes Números, distribución muestral, tamaño de muestra, margen de error

Módulo 3: Inferencia Estadística

Población vs Muestra, Estimadores y Sesgo

La inferencia estadística responde a una pregunta fundamental: ¿qué podemos decir sobre un grupo grande (población) a partir de un subconjunto pequeño (muestra)? Este capítulo sienta las bases conceptuales y matemáticas para hacerlo correctamente.

Población y Muestra

La *población* es el conjunto completo de individuos que te interesa estudiar. La *muestra* es un subconjunto de esa población.

```
import numpy as np
import pandas as pd
from scipy import stats

# Población: 1 millón de ingresos anuales (distribución log-normal)
np.random.seed(42)
poblacion = np.random.lognormal(mean=10, sigma=0.5, size=1_000_000)
print(f"Población: n={len(poblacion):,}, media=${np.mean(poblacion):.2f}")
# Población: n=1,000,000, media=$22,646.68

# Una muestra aleatoria de 100 personas
muestra = np.random.choice(poblacion, size=100, replace=False)
print(f"Muestra: n={len(muestra)}, media=${np.mean(muestra):.2f}")
# Muestra: n=100, media=$22,184.72 (cercano pero no igual)

# Las muestras varían
for i in range(3):
    m = np.random.choice(poblacion, 100)
    print(f" Muestra {i+1}: media=${np.mean(m):.2f}")
# Muestra 1: media=$21,847.23
# Muestra 2: media=$23,156.89
# Muestra 3: media=$22,512.34
```

> Advertencia:

| Una muestra no es un espejo perfecto de la población. La variabilidad muestral es inevitable. La inferencia estadística cuantifica esa incertidumbre.

Tipos de Muestreo

La forma en que seleccionas la muestra determina la validez de tus conclusiones.

```
# Muestreo aleatorio simple (el estándar de oro)
poblacion = np.arange(1, 10001)
muestra_aleatoria = np.random.choice(poblacion, size=100, replace=False)

# Muestreo estratificado
# Dividir la población en estratos y muestrear cada uno
def muestreo_estratificado(poblacion, estratos, n_por_estrato):
    indices = []
    for estrato in np.unique(estratos):
        idx_estrato = np.where(estratos == estrato)[0]
        idx_seleccion = np.random.choice(idx_estrato, n_por_estrato, replace=False)
        indices.extend(idx_seleccion)
    return poblacion[indices]

# Muestreo por conglomerados (cluster)
# Seleccionar grupos completos al azar
conglomerados = np.arange(100)
conglomerados_seleccionados = np.random.choice(conglomerados, 10, replace=False)
```

Parámetros y Estimadores

- **Parámetro:** valor numérico que describe a la población (casi siempre desconocido)
- **Estimador:** fórmula o regla para calcular un valor a partir de la muestra
- **Estimación:** el valor numérico que obtienes al aplicar el estimador a una muestra

concreta

```
# Relación parámetro-estimador
def analizar_estimadores(poblacion, n_muestra=50, n_simulaciones=10000):
    """Analiza el comportamiento de diferentes estimadores"""
    estimaciones_media = []
    estimaciones_mediana = []
    estimaciones_var_n = [] # Divide por n
    estimaciones_var_n1 = [] # Divide por n-1

    for _ in range(n_simulaciones):
        muestra = np.random.choice(poblacion, n_muestra)
        estimaciones_media.append(np.mean(muestra))
        estimaciones_mediana.append(np.median(muestra))
        estimaciones_var_n.append(np.var(muestra, ddof=0)) # Varianza muestral (n)
        estimaciones_var_n1.append(np.var(muestra, ddof=1)) # Varianza muestral (n-1)

    parametro_media = np.mean(poblacion)
    parametro_varianza = np.var(poblacion)

    print(f"Parámetro poblacional (media): {parametro_media:.2f}")
    print(f"Media de las estimaciones de media: {np.mean(estimaciones_media):.2f} (sesgo={np.mean(estimaciones_media)-parametro_media:.2f})")
    print(f"Media de las estimaciones de mediana: {np.mean(estimaciones_mediana):.2f} (sesgo={np.mean(estimaciones_mediana)-parametro_media:.2f})")
    print(f"Media de var(n): {np.mean(estimaciones_var_n):.2f} (sesgo={np.mean(estimaciones_var_n)-parametro_varianza:.2f})")
    print(f"Media de var(n-1): {np.mean(estimaciones_var_n1):.2f} (sesgo={np.mean(estimaciones_var_n1)-parametro_varianza:.2f})")
```

```

analizar_estimadores(poblacion, n_muestra=30, n_simulaciones=5000)
# Parámetro poblacional (media): 22646.68
# Media de las estimaciones de media: 22638.21 (sesgo=-8.47) ← insesgado
# Media de las estimaciones de mediana: 18824.92 (sesgo=-3821.76) ← sesgado
# Media de var(n): 49022361.34 (sesgo=-1757493.29) ← sesgado
# Media de var(n-1): 50158803.42 (sesgo=-620119.21) ← menos sesgado

```

Sesgo de un Estimador

El *sesgo* (bias) es la diferencia entre el valor esperado del estimador y el valor real del parámetro:

$$\text{Sesgo}(\hat{\theta}) = E[\hat{\theta}] - \theta$$

Un estimador es **insesgado** si su valor esperado coincide con el parámetro.

Corrección de Bessel (n-1)

La varianza muestral con n-1 es insesgada para la varianza poblacional. ¿Por qué n-1 y no n? Porque la media muestral es un estimador que "consume" un grado de libertad:

```

# Demostración visual de la corrección de Bessel
n_muestra = 10
sesgo_n = []
sesgo_n1 = []

for n in range(5, 100, 5):
    estimaciones_n = []
    estimaciones_n1 = []
    for _ in range(10000):
        muestra = np.random.normal(50, 10, n)
        estimaciones_n.append(np.var(muestra, ddof=0))
        estimaciones_n1.append(np.var(muestra, ddof=1))

    var_pob = 100 # σ² = 100
    sesgo_n.append(np.mean(estimaciones_n) - var_pob)
    sesgo_n1.append(np.mean(estimaciones_n1) - var_pob)

print("Sesgo de la varianza muestral según n:")
print(f" Con n: promedio = {np.mean(sesgo_n):.2f} (subestima sistemáticamente)")
print(f" Con n-1: promedio = {np.mean(sesgo_n1):.2f} (prácticamente insesgado)")
# Con n: promedio = -1.02 (subestima)
# Con n-1: promedio = 0.01 (insesgado)

```

Error Cuadrático Medio (MSE)

El MSE combina sesgo y varianza en una sola métrica:

$$\text{MSE} = \text{Sesgo}^2(\hat{\theta}) + \text{Var}(\hat{\theta})$$

```

# Trade-off sesgo-varianza
from scipy import stats

```



```
def mse_sesgo_varianza(poblacion, estimador_func, n_muestra, n_simulaciones=10000):
    estimaciones = []
    for _ in range(n_simulaciones):
        muestra = np.random.choice(poblacion, n_muestra)
        estimaciones.append(estimador_func(muestra))

    param = np.mean(poblacion)
    sesgo = np.mean(estimaciones) - param
    varianza = np.var(estimaciones, ddof=1)
    mse = np.mean((np.array(estimaciones) - param)**2)

    print(f"Sesgo: {sesgo:.3f}")
    print(f"Varianza: {varianza:.3f}")
    print(f"MSE: {mse:.3f}")
    print(f"Sesgo2 + Varianza: {sesgo**2 + varianza:.3f}")

# Comparar media vs mediana para estimar la media poblacional
# en una distribución normal (sin outliers)
poblacion_normal = np.random.normal(50, 10, 100000)

print("--- Estimando la media con la MEDIA muestral ---")
mse_sesgo_varianza(poblacion_normal, np.mean, n_muestra=30)

print("\n--- Estimando la media con la MEDIANA muestral ---")
mse_sesgo_varianza(poblacion_normal, np.median, n_muestra=30)
# La media muestral es mejor (menor MSE) para datos normales
```

Consistencia y Eficiencia

Un estimador es **consistente** si converge al parámetro a medida que n crece:

```
# Consistencia de la media muestral
parametro = np.mean(poblacion)
sesgos = []

for n in [5, 10, 50, 100, 500, 1000]:
    estimaciones = [np.mean(np.random.choice(poblacion, n)) for _ in range(1000)]
    sesgo = abs(np.mean(estimaciones) - parametro)
    sesgos.append(sesgo)

print("Consistencia de la media muestral:")
for n, s in zip([5, 10, 50, 100, 500, 1000], sesgos):
    print(f"  n={n:4d}: |sesgo| = {s:.2f}")

# n=   5: |sesgo| = 173.54
# n=  10: |sesgo| = 124.30
# n=  50: |sesgo| = 41.94
# n= 100: |sesgo| = 32.13
# n= 500: |sesgo| = 13.92
# n=1000: |sesgo| = 10.75
```

La **eficiencia** compara la varianza de dos estimadores insesgados. El más eficiente tiene menor varianza.

Aplicación Práctica: Estimación de la Media con Intervalo

```
def estimar_media(datos, confianza=0.95):
    """Estima la media poblacional con intervalo de confianza"""
    n = len(datos)
    media_muestral = np.mean(datos)
    error_std = np.std(datos, ddof=1) / np.sqrt(n)
    z = stats.norm.ppf(1 - (1 - confianza) / 2)

    return {
        'media': media_muestral,
        'error_std': error_std,
        'margen': z * error_std,
        'ic_inf': media_muestral - z * error_std,
        'ic_sup': media_muestral + z * error_std,
        'confianza': confianza
    }

muestra_ejemplo = np.random.choice(poblacion, 200)
resultado = estimar_media(muestra_ejemplo, 0.95)
print(f"Media estimada: {resultado['media']:.2f}")
print(f"IC 95%: [{resultado['ic_inf']:.2f}, {resultado['ic_sup']:.2f}]")
print(f"Media poblacional real: {np.mean(poblacion):.2f}")
```

Ejercicios Prácticos

1. **Ejercicio 1:** Genera una población de 100,000 valores con `np.random.exponential(scale=3)`. Toma 1000 muestras de tamaño $n=20$ y calcula la media de cada una. ¿La media de las medias muestrales se aproxima a la media poblacional?
2. **Ejercicio 2:** Demuestra que la varianza muestral con $ddof=0$ es sesgada. Simula 10000 muestras de tamaño $n=10$ de una normal(0,1). Compara la media de `np.var(muestra, ddof=0)` con la varianza poblacional.
3. **Ejercicio 3:** Para una distribución asimétrica (exponencial), compara el MSE de la media vs la mediana como estimadores de la media poblacional. ¿Cuál tiene menor MSE? ¿Por qué?
4. **Ejercicio 4:** Implementa una función que calcule el tamaño de muestra necesario para estimar una media con un margen de error dado, usando la fórmula: $n = (z \cdot \sigma / M)^2$.

Resumen del Capítulo

- La **población** es el conjunto total; la **muestra** es un subconjunto
- La **inferencia** usa la muestra para concluir sobre la población
- El **sesgo** es la diferencia sistemática entre estimador y parámetro
- La **varianza muestral** con $n-1$ (corrección de Bessel) es insesgada
- El **MSE** = sesgo² + varianza; captura el trade-off entre ambos
- Un estimador **consistente** mejora al aumentar n

- La **eficiencia** compara la varianza de estimadores competidores

Conceptos clave aprendidos: población, muestra, parámetro, estimador, sesgo, corrección de Bessel, MSE, consistencia, eficiencia, error estándar

Intervalos de Confianza

Una estimación puntual (como "la media es 23.5") es útil pero incompleta. No nos dice qué tan precisa es esa estimación. Los intervalos de confianza (IC) solucionan esto: proporcionan un rango de valores plausibles para el parámetro poblacional, junto con un nivel de confianza.

¿Qué es un Intervalo de Confianza?

Un IC al 95% significa: "si repitiéramos este estudio muchas veces, el 95% de los intervalos calculados contendrían el verdadero parámetro poblacional".

> **Advertencia:**

> Un IC al 95% **NO** significa que hay 95% de probabilidad de que el parámetro esté en el intervalo. El parámetro es fijo (aunque desconocido); es el intervalo el que es aleatorio.

IC para la Media (Varianza Conocida)

Cuando conoces σ (raro en la práctica) y los datos son normales o n es grande:

$$\text{IC: } \bar{X} \pm z_{\{\alpha/2\}} \cdot \sigma/\sqrt{n}$$

```
import numpy as np
from scipy import stats

def ic_media_conocida(datos, sigma, confianza=0.95):
    """IC para la media con varianza poblacional conocida"""
    n = len(datos)
    media = np.mean(datos)
    z = stats.norm.ppf(1 - (1 - confianza) / 2)
    margen = z * sigma / np.sqrt(n)

    return {
        'media': media,
        'margen': margen,
        'ic_inf': media - margen,
        'ic_sup': media + margen
    }

# Ejemplo: sabemos que  $\sigma = 15$  (como en IQ)
np.random.seed(42)
muestra = np.random.normal(100, 15, 50)
ic = ic_media_conocida(muestra, sigma=15, confianza=0.95)
print(f"IC 95% para la media: [{ic['ic_inf']:.2f}, {ic['ic_sup']:.2f}]"
```

```
# IC 95% para la media: [96.72, 105.06]
```

IC para la Media (Varianza Desconocida)

El caso realista: no conocemos σ , así que lo estimamos con la desviación estándar muestral s .
Usamos la distribución *t de Student*:

```
IC:  $\bar{X} \pm t_{\{\alpha/2, n-1\}} \cdot s/\sqrt{n}$ 
```

```
def ic_media_desconocida(datos, confianza=0.95):
    """IC para la media con varianza poblacional desconocida"""
    n = len(datos)
    media = np.mean(datos)
    s = np.std(datos, ddof=1) # Desviación estándar muestral
    error_std = s / np.sqrt(n)
    t = stats.t.ppf(1 - (1 - confianza) / 2, df=n-1)
    margen = t * error_std

    return {
        'media': media,
        's': s,
        'error_std': error_std,
        'margen': margen,
        'ic_inf': media - margen,
        'ic_sup': media + margen,
        'distribucion': f't({n-1})'
    }

# Ejemplo con datos reales simulados
np.random.seed(123)
pesos = np.random.normal(70, 10, 25) # 25 personas, media 70, desv 10
ic = ic_media_desconocida(pesos, 0.95)
print(f"Media muestral: {ic['media']:.2f}")
print(f"Desv. estándar muestral: {ic['s']:.2f}")
print(f"Error estándar: {ic['error_std']:.3f}")
print(f"IC 95%: [{ic['ic_inf']:.2f}, {ic['ic_sup']:.2f}]")
# Media muestral: 69.43
# Desv. estándar muestral: 10.74
# Error estándar: 2.148
# IC 95%: [65.00, 73.87]
```

IC para una Proporción

Para variables binarias (éxito/fracaso):

```
def ic_proporcion(x, n, confianza=0.95):
    """IC para una proporción (método de Wilson)"""
    p_hat = x / n
    z = stats.norm.ppf(1 - (1 - confianza) / 2)

    # Fórmula de Wilson (mejor que la aproximación normal simple)
```

```

denominador = 1 + z**2 / n
centro = (p_hat + z**2 / (2*n)) / denominador
margen = z * np.sqrt(p_hat*(1-p_hat)/n + z**2/(4*n**2)) / denominador

return {
    'proporcion': p_hat,
    'n': n,
    'ic_inf': centro - margen,
    'ic_sup': centro + margen,
    'metodo': 'Wilson'
}

# Encuesta: 340 de 1000 votantes prefieren al candidato A
resultado = ic_proporcion(340, 1000, 0.95)
print(f"Proporción estimada: {resultado['proporcion']:.3f}")
print(f"IC 95%: [{resultado['ic_inf']:.3f}, {resultado['ic_sup']:.3f}]")
# Proporción estimada: 0.340
# IC 95%: [0.311, 0.370]

```

Interpretación Correcta de IC

```

# Demostración de cobertura: ¿cuántos IC contienen el verdadero parámetro?
np.random.seed(42)
media_poblacional = 100
sigma = 15
n = 30
n_experimentos = 10000

contiene = 0
for _ in range(n_experimentos):
    muestra = np.random.normal(media_poblacional, sigma, n)
    ic = ic_media_desconocida(muestra, 0.95)
    if ic['ic_inf'] <= media_poblacional <= ic['ic_sup']:
        contiene += 1

print(f"Cobertura real: {contiene/n_experimentos:.4f} (esperado: 0.95)")
# Cobertura real: ~0.9490 (muy cercano al 95%)

```

Factores que Afectan la Amplitud del IC

Factor	Cambio	Efecto en IC
Más datos (↑ n)	n = 100 en vez de 25	Se reduce (más preciso)
Mayor confianza	99% en vez de 95%	Se amplía (más seguro)
Más variabilidad	σ = 20 en vez de 10	Se amplía (más incierto)

```

def ancho_ic(n, sigma, confianza=0.95):
    z = stats.norm.ppf(1 - (1 - confianza) / 2)
    return 2 * z * sigma / np.sqrt(n)

print("Ancho del IC para diferentes escenarios:")

```

```

print(f"n=30, σ=10, 95%: {ancho_ic(30, 10, 0.95):.2f}")
print(f"n=100, σ=10, 95%: {ancho_ic(100, 10, 0.95):.2f}")
print(f"n=30, σ=20, 95%: {ancho_ic(30, 20, 0.95):.2f}")
print(f"n=30, σ=10, 99%: {ancho_ic(30, 10, 0.99):.2f}")
# n=30, σ=10, 95%: 7.16
# n=100, σ=10, 95%: 3.92
# n=30, σ=20, 95%: 14.31
# n=30, σ=10, 99%: 9.42

```

IC para la Diferencia de Dos Medias

Comparar dos grupos (la base del A/B testing):

```

def ic_diferencia_medias(muestra1, muestra2, confianza=0.95):
    """IC para la diferencia de medias (varianzas desiguales - Welch)"""
    n1, n2 = len(muestra1), len(muestra2)
    media1, media2 = np.mean(muestra1), np.mean(muestra2)
    s1, s2 = np.std(muestra1, ddof=1), np.std(muestra2, ddof=1)

    # Error estándar de la diferencia
    se = np.sqrt(s1**2/n1 + s2**2/n2)

    # Grados de libertad (Welch-Satterthwaite)
    num = (s1**2/n1 + s2**2/n2)**2
    den = (s1**2/n1)**2/(n1-1) + (s2**2/n2)**2/(n2-1)
    df = num / den

    t = stats.t.ppf(1 - (1 - confianza) / 2, df=df)
    diferencia = media1 - media2
    margen = t * se

    return {
        'diferencia': diferencia,
        'ic_inf': diferencia - margen,
        'ic_sup': diferencia + margen,
        'df': df,
        'metodo': 'Welch'
    }

# Simular: grupo control vs tratamiento
np.random.seed(42)
control = np.random.normal(50, 10, 100)
tratamiento = np.random.normal(53, 10, 100) # 3 puntos más

ic_diff = ic_diferencia_medias(control, tratamiento)
print(f"Diferencia (trat - control): {ic_diff['diferencia']:.2f}")
print(f"IC 95%: [{ic_diff['ic_inf']:.2f}, {ic_diff['ic_sup']:.2f}]")
# Diferencia (trat - control): 3.21
# IC 95%: [0.44, 5.98]

```

IC vs Testing de Hipótesis

Los IC son complementarios a las pruebas de hipótesis:

```
# Si un IC del 95% para la diferencia NO contiene 0,  
# entonces un test de hipótesis al 5% rechazaría  $H_0$ : diferencia = 0  
print(f"¿El IC contiene 0? {ic_diff['ic_inf'] <= 0 <= ic_diff['ic_sup']}")  
# ¿El IC contiene 0? False → rechazamos  $H_0$  con  $\alpha=0.05$ 
```

> **Consejo del Analista:**

> Los intervalos de confianza son más informativos que los valores p porque muestran la magnitud del efecto y su precisión. Cuando reportes resultados, incluye siempre el IC.

Ejercicios Prácticos

1. **Ejercicio 1:** Toma una muestra de tamaño $n=50$ de una distribución normal(100, 15). Calcula el IC del 90%, 95% y 99% para la media. ¿Cómo cambia la amplitud?
2. **Ejercicio 2:** Simula un experimento donde el verdadero parámetro es 50. Genera 1000 muestras de tamaño $n=20$ y calcula el IC al 95% para cada una. ¿Qué porcentaje contiene al 50?
3. **Ejercicio 3:** En una encuesta, 120 de 500 personas prefieren el producto A. Calcula el IC del 95% para la proporción usando el método de Wilson. Interpreta el resultado.
4. **Ejercicio 4:** Dos grupos: control ($n=80$, media=45, $s=12$) y tratamiento ($n=75$, media=50, $s=11$). Calcula el IC del 95% para la diferencia de medias. ¿Hay diferencia significativa?

Resumen del Capítulo

- Un **IC** proporciona un rango de valores plausibles para un parámetro
- El **nivel de confianza** es la frecuencia con que el IC contiene al parámetro en repeticiones del estudio
- Usa **t de Student** cuando la varianza poblacional es desconocida
- El **método de Wilson** es superior para proporciones
- IC más amplios = más confianza, menos precisión
- n más grande IC más estrecho
- Los IC son más informativos que los valores p solos

Conceptos clave aprendidos: intervalo de confianza, nivel de confianza, error estándar, margen de error, t de Student, método de Wilson, IC para diferencia de medias

Pruebas de Hipótesis: Valor p, Errores Tipo I y II

Las pruebas de hipótesis son el mecanismo formal para tomar decisiones basadas en datos. ¿Un nuevo fármaco funciona mejor que el placebo? ¿El rediseño de la web aumentó las conversiones? Las pruebas de hipótesis responden estas preguntas con rigor estadístico.

La Estructura de una Prueba de Hipótesis

Toda prueba de hipótesis sigue el mismo esquema:

1. **H₀ (Hipótesis nula):** No hay efecto, no hay diferencia, el status quo
2. **H₁ (Hipótesis alternativa):** Hay efecto, hay diferencia, lo que queremos demostrar
3. **Estadístico de prueba:** Valor calculado de los datos
4. **Valor p:** Probabilidad de observar algo tan extremo como lo observado, asumiendo H₀ cierta
5. **Conclusión:** Rechazar H₀ si $p < \alpha$, o no rechazar H₀

```
import numpy as np
from scipy import stats

# Ejemplo clásico: ¿la media es diferente de 100?
np.random.seed(42)
datos = np.random.normal(105, 15, 30) # Media real 105

# H0:  $\mu = 100$ 
# H1:  $\mu \neq 100$  (prueba bilateral)

t_stat, p_valor = stats.ttest_1samp(datos, popmean=100)
print(f"Estadístico t: {t_stat:.4f}")
print(f"Valor p: {p_valor:.4f}")
# Estadístico t: 1.8745
# Valor p: 0.0710

alpha = 0.05
if p_valor < alpha:
    print("Rechazamos H0: la media es diferente de 100")
else:
    print("No rechazamos H0: no hay evidencia suficiente")
# No rechazamos H0: no hay evidencia suficiente
```

Valor p: La Interpretación Correcta

El *valor p* es la probabilidad de obtener un resultado tan extremo o más que el observado, asumiendo que la hipótesis nula es verdadera.

> **Advertencia:**

| Muchas cosas que el valor p **NO** es:

- | - No es la probabilidad de que H_0 sea cierta
- | - No es la probabilidad de que H_1 sea cierta
- | - No es la probabilidad de que el resultado se deba al azar
- | - No es el tamaño del efecto

```
# Demostración: valor p bajo  $H_0$  verdadera
np.random.seed(42)
p_valores_bajo_H0 = []

for _ in range(10000):
    # Generar datos bajo  $H_0$  (media = 100)
    datos = np.random.normal(100, 15, 30)
    _, p = stats.ttest_1samp(datos, 100)
    p_valores_bajo_H0.append(p)

# Bajo  $H_0$ , los valores p son uniformes
print(f"Proporción p < 0.05: {np.mean(np.array(p_valores_bajo_H0) < 0.05):.4f}")
print(f"Proporción p < 0.01: {np.mean(np.array(p_valores_bajo_H0) < 0.01):.4f}")
# Proporción p < 0.05: ~0.050 (¡exactamente  $\alpha$ !)
# Proporción p < 0.01: ~0.010 (¡exactamente  $\alpha$ !)
```

Errores Tipo I y Tipo II

Decisión	H_0 verdadera	H_0 falsa
No rechazar H_0	Correcto	Error Tipo II (β)
Rechazar H_0	Error Tipo I (α)	Correcto (Potencia)

```
def simular_errores(media_h0=100, media_real=103, sigma=15, n=50,
                    alpha=0.05, n_sim=5000):
    """Simula errores Tipo I y Tipo II"""
    # Bajo  $H_0$  (para error tipo I)
    rechazos_bajo_H0 = 0
    for _ in range(n_sim):
        datos = np.random.normal(media_h0, sigma, n)
        _, p = stats.ttest_1samp(datos, media_h0)
        if p < alpha:
            rechazos_bajo_H0 += 1

    error_tipo_I = rechazos_bajo_H0 / n_sim

    # Bajo  $H_1$  (para error tipo II = 1 - potencia)
    no_rechazos_bajo_H1 = 0
    for _ in range(n_sim):
```

```

    datos = np.random.normal(media_real, sigma, n)
    _, p = stats.ttest_1samp(datos, media_h0)
    if p >= alpha:
        no_rechazos_bajo_H1 += 1

error_tipo_II = no_rechazos_bajo_H1 / n_sim
potencia = 1 - error_tipo_II

print(f"Error Tipo I ( $\alpha$  real): {error_tipo_I:.4f} (nominal: {alpha})")
print(f"Error Tipo II ( $\beta$ ): {error_tipo_II:.4f}")
print(f"Potencia (1- $\beta$ ): {potencia:.4f}")
return error_tipo_I, error_tipo_II, potencia

print("--- Escenario: efecto pequeño ( $\mu=103$ ,  $n=50$ ) ---")
simular_errores(media_real=103, n=50)

print("\n--- Escenario: efecto grande ( $\mu=110$ ,  $n=50$ ) ---")
simular_errores(media_real=110, n=50)

print("\n--- Escenario: muestra grande ( $\mu=103$ ,  $n=200$ ) ---")
simular_errores(media_real=103, n=200)
# Error Tipo I siempre  $\sim 0.05$  (controlado por  $\alpha$ )
# Error Tipo II varía: mayor efecto o mayor  $n \rightarrow$  menor  $\beta \rightarrow$  mayor potencia

```

Factores que Afectan la Potencia

La *potencia* ($1 - \beta$) es la probabilidad de detectar un efecto cuando realmente existe.

Factor	Aumentar factor	Efecto en potencia
Tamaño de muestra (n)	Aumentar	Aumenta
Tamaño del efecto	Mayor	Aumenta
Variabilidad (σ)	Menor	Aumenta
α	Aumentar α	Aumenta (pero más errores Tipo I)

```

def calcular_potencia(tamano_efecto=0.5, n=50, alpha=0.05):
    """Potencia de un t-test usando análisis analítico"""
    df = n - 1
    t_critico = stats.t.ppf(1 - alpha/2, df)
    # Potencia =  $P(|T| > t_{critico} \mid \delta)$ 
    potencia = stats.nct.sf(t_critico, df, tamano_efecto*np.sqrt(n)) + \
        stats.nct.cdf(-t_critico, df, tamano_efecto*np.sqrt(n))
    return potencia

# Mapa de calor de potencia
print("Potencia según  $n$  y tamaño del efecto ( $d$  de Cohen):")
print("\n\t $d=0.2$ \td=0.5\t $d=0.8$ ")
for n in [20, 50, 100, 200]:
    potencias = [calcular_potencia(d, n) for d in [0.2, 0.5, 0.8]]
    print(f"{n}\t{potencias[0]:.2f}\t{potencias[1]:.2f}\t{potencias[2]:.2f}")
#  $n=20$ : 0.2d=0.5d=0.8
# 200.090.330.66

```

```
# 500.140.700.97
# 1000.290.941.00
# 2000.611.001.00
```

Pruebas Unilaterales vs Bilaterales

```
# Prueba unilateral:  $H_1: \mu > 100$ 
def ttest_unilateral_derecha(datos, mu0=100):
    t_stat, p_bilateral = stats.ttest_1samp(datos, mu0)
    p_unilateral = p_bilateral / 2 if t_stat > 0 else 1 - p_bilateral / 2
    return t_stat, p_unilateral

# Prueba bilateral:  $H_1: \mu \neq 100$ 
# Prueba unilateral derecha:  $H_1: \mu > 100$ 
# Prueba unilateral izquierda:  $H_1: \mu < 100$ 

np.random.seed(42)
datos = np.random.normal(103, 15, 50)

_, p_bil = stats.ttest_1samp(datos, 100)
_, p_uni = ttest_unilateral_derecha(datos, 100)
print(f"p bilateral: {p_bil:.4f}")
print(f"p unilateral ( $\mu > 100$ ): {p_uni:.4f}")
# p bilateral: 0.1420
# p unilateral: 0.0710

# La unilateral tiene más potencia si la dirección es correcta
```

> Consejo del Analista:

| Usa pruebas bilaterales por defecto. Las unilaterales solo cuando tengas una justificación sólida y previa de por qué solo una dirección es relevante.

Potencia y Tamaño de Muestra

```
from scipy.stats import nct

def n_para_potencia(tamano_efecto=0.5, potencia=0.80, alpha=0.05):
    """Calcula n necesario para alcanzar cierta potencia"""
    for n in range(5, 10000):
        df = n - 1
        t_crit = stats.t.ppf(1 - alpha/2, df)
        noncentral = tamano_efecto * np.sqrt(n)
        pot = nct.sf(t_crit, df, noncentral) + nct.cdf(-t_crit, df, noncentral)
        if pot >= potencia:
            return n
    return None

print("n necesario para potencia 80%:")
for d in [0.2, 0.3, 0.5, 0.8]:
    n_req = n_para_potencia(tamano_efecto=d, potencia=0.80)
    print(f" d = {d}: n = {n_req}")
```

```
# d = 0.2: n = 199
# d = 0.3: n = 89
# d = 0.5: n = 33
# d = 0.8: n = 14
```

El Debate del Valor p

El valor p es controvertido. La American Statistical Association emitió en 2016 seis principios sobre su uso:

1. Los valores p pueden indicar cuán incompatibles son los datos con H_0
2. Los valores p no miden la probabilidad de que H_0 sea cierta
3. Los cortes científicos (como $p < 0.05$) no deben usarse mecánicamente
4. La inferencia adecuada requiere transparencia y reporte completo
5. El valor p no mide el tamaño del efecto ni la importancia
6. Por sí solo, el valor p no proporciona evidencia sólida

```
# Ejemplo: efecto pequeño pero n grande → p pequeño
np.random.seed(42)
# Diferencia minúscula pero n enorme
grupo_a = np.random.normal(100, 15, 10000)
grupo_b = np.random.normal(100.5, 15, 10000) # Solo 0.5 de diferencia

t_stat, p_val = stats.ttest_ind(grupo_a, grupo_b)
d_cohen = (np.mean(grupo_b) - np.mean(grupo_a)) / np.std(np.concatenate([grupo_a, grupo_b]))

print(f"Diferencia: {np.mean(grupo_b) - np.mean(grupo_a):.3f}")
print(f"Valor p: {p_val:.6f}")
print(f"d de Cohen: {d_cohen:.4f} (efecto muy pequeño)")
# p < 0.05 pero el efecto es trivial
```

> Advertencia:

| Un valor p pequeño no significa un efecto importante. Con muestras grandes, incluso diferencias triviales producen $p < 0.05$. Reporta siempre el tamaño del efecto y el intervalo de confianza.

Ejercicios Prácticos

1. **Ejercicio 1:** Simula 1000 experimentos bajo H_0 verdadera ($\mu=50$, $\sigma=10$, $n=30$). ¿Qué proporción de valores p es menor que 0.05? ¿Qué proporción menor que 0.01?
2. **Ejercicio 2:** Para un efecto de $d=0.4$ con $\alpha=0.05$, ¿cuántos sujetos necesitas para potencia del 80%? ¿Y para potencia del 95%?
3. **Ejercicio 3:** Explica con tus palabras la diferencia entre significancia estadística y significancia práctica. Da un ejemplo donde un resultado sea estadísticamente significativo pero no importante.
4. **Ejercicio 4:** Usando `scipy.stats.ttest\_ind`, compara dos grupos: A (media=52, $\sigma=10$, $n=30$) y B (media=48, $\sigma=10$, $n=30$). Reporta el valor p, el IC de la diferencia y el d de Cohen. Interpreta.

Resumen del Capítulo

- Las pruebas de hipótesis evalúan evidencia contra H_0
 - **Error Tipo I:** rechazar H_0 verdadera (α , controlado por el investigador)
 - **Error Tipo II:** no rechazar H_0 falsa (β , depende de n , efecto, variabilidad)
 - La **potencia** ($1-\beta$) es la probabilidad de detectar un efecto real
- n más grande mayor potencia
- El valor p no mide el tamaño del efecto
- Siempre reporta IC y tamaño del efecto, no solo el valor p

Conceptos clave aprendidos: hipótesis nula y alternativa, valor p , error Tipo I, error Tipo II, potencia, tamaño del efecto, d de Cohen, prueba unilateral/bilateral

Pruebas Estadísticas en Python: T-tests, ANOVA y Chi-cuadrado

En este capítulo unificamos la teoría de los capítulos anteriores en pruebas estadísticas concretas que usarás en tu trabajo diario. Cada prueba viene con su código, supuestos e interpretación.

T-Test para Una Muestra

Compara la media de una muestra contra un valor de referencia.

Supuestos: datos independientes, distribución aproximadamente normal (o $n \geq 30$).

```
import numpy as np
from scipy import stats

np.random.seed(42)

# ¿El IQ de esta muestra difiere de 100?
iq_muestra = np.random.normal(103, 15, 25)

t_stat, p_valor = stats.ttest_1samp(iq_muestra, popmean=100)
print(f"T-Test una muestra:")
print(f"  t = {t_stat:.4f}, p = {p_valor:.4f}")
print(f"  Media: {np.mean(iq_muestra):.2f} vs 100")
# t = 0.9925, p = 0.3305
# No rechazamos H0: no hay evidencia de que el IQ difiera de 100
```

T-Test para Dos Muestras Independientes

Compara las medias de dos grupos independientes.

Supuestos: independencia, normalidad (o $n \geq 30$), homogeneidad de varianzas.

```
# Dos grupos: control vs tratamiento
control = np.random.normal(50, 10, 30)
tratamiento = np.random.normal(55, 10, 30)

# Prueba de Welch (no asume varianzas iguales) - recomendada
t_stat, p_valor = stats.ttest_ind(control, tratamiento, equal_var=False)
print(f"T-Test independiente (Welch):")
print(f"  t = {t_stat:.4f}, p = {p_valor:.4f}")
print(f"  Control: {np.mean(control):.2f}, Tratamiento: {np.mean(tratamiento):.2f}")
# t = -2.08, p = 0.042
```

```
# Rechazamos H0: hay diferencia significativa

# Prueba de Student (asume varianzas iguales)
t_stat_s, p_valor_s = stats.ttest_ind(control, tratamiento, equal_var=True)

# Verificación de varianzas iguales (Levene)
w_stat, p_levene = stats.levene(control, tratamiento)
print(f" Levene: W = {w_stat:.4f}, p = {p_levene:.4f}")
# Si p_levene > 0.05, no rechazamos igualdad de varianzas
```

> Consejo del Analista:

> Usa `equal\_var=False` (Welch) por defecto. No asume varianzas iguales y funciona bien incluso cuando lo son. Es la recomendación moderna.

T-Test para Muestras Pareadas (Dependientes)

Cuando cada sujeto es su propio control (antes/después).

Supuestos: diferencias pareadas distribuidas normalmente.

```
# Mediciones antes y después del mismo sujeto
antes = np.random.normal(70, 10, 20)
despues = antes + np.random.normal(-5, 5, 20) # Reducción promedio de 5

t_stat, p_valor = stats.ttest_rel(antes, despues)
print(f"T-Test pareado:")
print(f" t = {t_stat:.4f}, p = {p_valor:.4f}")
print(f" Diferencia media: {np.mean(despues - antes):.2f}")
# t = 4.05, p = 0.0006
# Rechazamos H0: hay cambio significativo
```

ANOVA de Un Factor

Compara las medias de tres o más grupos.

H₀: todas las medias son iguales.

H₁: al menos una media es diferente.

Supuestos: independencia, normalidad, homogeneidad de varianzas.

```
# Tres grupos
grupo_a = np.random.normal(50, 10, 30)
grupo_b = np.random.normal(55, 10, 30)
grupo_c = np.random.normal(45, 10, 30)

# ANOVA de un factor
f_stat, p_valor = stats.f_oneway(grupo_a, grupo_b, grupo_c)
print(f"ANOVA de un factor:")
print(f" F = {f_stat:.4f}, p = {p_valor:.4f}")
# F = 8.12, p = 0.0005
# Rechazamos H0: al menos un grupo es diferente

# Post-hoc (Tukey HSD) - ¿qué pares difieren?
from scipy.stats import tukey_hsd
```



```

result = tukey_hsd(grupo_a, grupo_b, grupo_c)
print(" Tukey HSD:")
for i, j in [(0,1), (0,2), (1,2)]:
    print(f" Grupo {i}-{j}: p = {result.pvalue[i][j]:.4f}")
# Ejemplo: grupo A-C puede ser significativo, A-B no

```

> Advertencia:

> El ANOVA te dice que hay diferencias, pero no cuáles. Usa pruebas post-hoc (Tukey, Bonferroni) para identificar los pares específicos. Siempre ajusta por comparaciones múltiples.

ANOVA de Dos Factores (Modelo Factorial)

Evalúa el efecto de dos variables categóricas y su interacción.

```

import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols

# Simular datos con dos factores
np.random.seed(42)
n = 10
datos = pd.DataFrame({
    'tratamiento': np.repeat(['A', 'B', 'C'], n*2),
    'sexo': np.tile(np.repeat(['M', 'F'], n), 3),
    'respuesta': np.concatenate([
        np.random.normal(50, 5, n*2), # A
        np.random.normal(55, 5, n*2), # B
        np.random.normal(52, 5, n*2), # C
    ])
})

# ANOVA de dos factores con interacción
modelo = ols('respuesta ~ C(tratamiento) * C(sexo)', data=datos).fit()
tabla_anova = sm.stats.anova_lm(modelo, typ=2)
print("ANOVA de dos factores:")
print(tabla_anova)

```

#	sum_sq	df	F	PR(>F)
# C(tratamiento)	204.30	2.0	6.88...	0.003...
# C(sexo)	20.56	1.0	1.38...	0.248...
# Interacción	12.57	2.0	0.42...	0.659...

Prueba Chi-Cuadrado (χ^2)

Para variables categóricas: independencia y bondad de ajuste.

Test de Independencia

¿Hay asociación entre dos variables categóricas?

```

# Tabla de contingencia: ¿el género se asocia con la preferencia de producto?
# Datos observados
tabla = np.array([[30, 20], # Hombres: prefieren A, B

```

```

[15, 35]]) # Mujeres: prefieren A, B

chi2, p_valor, dof, esperados = stats.chi2_contingency(tabla)
print(f"Chi-cuadrado de independencia:")
print(f"   $\chi^2 = \{chi2:.4f\}$ ,  $p = \{p\_valor:.4f\}$ ,  $df = \{dof\}$ ")
print(f"  Frecuencias esperadas:\n{np.round(esperados, 1)}")
#  $\chi^2 = 8.15$ ,  $p = 0.0043$ 
# Rechazamos  $H_0$ : hay asociación entre género y preferencia

# Medida de asociación (V de Cramer)
n = np.sum(tabla)
v_cramer = np.sqrt(chi2 / (n * min(tabla.shape) - 1))
print(f"  V de Cramer:  $\{v\_cramer:.4f\}$  (0=sin asociación, 1=asociación perfecta)")

```

Bondad de Ajuste

¿Los datos observados siguen una distribución esperada?

```

# ¿Un dado es justo? 60 lanzamientos
observado = np.array([8, 12, 9, 11, 10, 10]) # Frecuencias observadas
esperado = np.array([10, 10, 10, 10, 10, 10]) # Frecuencias esperadas (justo)

chi2, p_valor = stats.chisquare(observado, f_exp=esperado)
print(f"Chi-cuadrado bondad de ajuste:")
print(f"   $\chi^2 = \{chi2:.4f\}$ ,  $p = \{p\_valor:.4f\}$ ")
#  $\chi^2 = 1.40$ ,  $p = 0.9241$ 
# No rechazamos  $H_0$ : el dado parece justo

```

Test de Normalidad

Antes de aplicar pruebas paramétricas, verifica la normalidad:

```

# Shapiro-Wilk (estándar para  $n < 5000$ )
np.random.seed(42)
normales = np.random.normal(0, 1, 100)
no_normales = np.random.exponential(1, 100)

stat, p_norm = stats.shapiro(normales)
_, p_no_norm = stats.shapiro(no_normales)

print(f"Shapiro-Wilk:")
print(f"  Datos normales:  $W = \{stat:.4f\}$ ,  $p = \{p\_norm:.4f\}$ ")
print(f"  Datos exponenciales:  $W = \{stat:.4f\}$ ,  $p = \{p\_no\_norm:.4f\}$ ")
# Datos normales:  $p > 0.05 \rightarrow$  no rechazamos normalidad
# Datos exponenciales:  $p < 0.05 \rightarrow$  rechazamos normalidad

# Alternativas: D'Agostino-Pearson (mejor para  $n$  grande)
k2_stat, p_k2 = stats.normaltest(normales)
print(f"  D'Agostino:  $p = \{p\_k2:.4f\}$ ")

```

Guía Rápida: ¿Qué Prueba Usar?

Situación	Variable respuesta	Grupos	Prueba
Comparar con valor de referencia	Númerica	1	T-test 1 muestra
Comparar dos grupos	Númerica	2	T-test independiente
Antes/después	Númerica	2 (pareados)	T-test pareado
Comparar ≥ 3 grupos	Númerica	≥ 3	ANOVA + post-hoc
Asociación entre categóricas	Categórica	2+	Chi-cuadrado
Relación numérica-numérica	Númerica	-	Correlación/Regresión

Aplicación Completa: Diagnóstico Estadístico

```
def diagnostico_completo(grupos, nombres=None):
    """
    Realiza el pipeline estadístico completo para comparar grupos.
    - Verifica normalidad
    - Verifica homogeneidad de varianzas
    - Aplica prueba paramétrica o no paramétrica según corresponda
    """

    if nombres is None:
        nombres = [f'Grupo {i+1}' for i in range(len(grupos))]

    print("=== Diagnóstico Estadístico ===\n")

    # 1. Normalidad (Shapiro-Wilk)
    print("1. Normalidad (Shapiro-Wilk):")
    todos_normales = True
    for nombre, grupo in zip(nombres, grupos):
        _, p = stats.shapiro(grupo)
        es_normal = p > 0.05
        print(f"    {nombre}: W={stat:.4f}, p={p:.4f} {'' if es_normal else ''}")
        todos_normales = todos_normales and es_normal

    # 2. Homogeneidad de varianzas (Levene)
    print("\n2. Homogeneidad de varianzas (Levene):")
    _, p_levene = stats.levene(*grupos)
    varianzas_iguales = p_levene > 0.05
    print(f"    W={stat:.4f}, p={p_levene:.4f} {'' if varianzas_iguales else ''}")

    # 3. Prueba principal
    print(f"\n3. Prueba principal:")
    if todos_normales and varianzas_iguales:
        if len(grupos) == 2:
            t, p = stats.ttest_ind(*grupos, equal_var=True)
            print(f"    T-test: t={t:.4f}, p={p:.4f}")
        else:
            f, p = stats.f_oneway(*grupos)
            print(f"    ANOVA: F={f:.4f}, p={p:.4f}")
    else:
        if len(grupos) == 2:
            u, p = stats.mannwhitneyu(*grupos)
            print(f"    Mann-Whitney U: U={u:.0f}, p={p:.4f}")
        else:
```

```

h, p = stats.kruskal(*grupos)
print(f"    Kruskal-Wallis: H={h:.4f}, p={p:.4f}")

# 4. Interpretación
print(f"\n4. Conclusión:")
if p < 0.05:
    print(f"    Hay diferencias significativas (p={p:.4f})")
else:
    print(f"    No hay diferencias significativas (p={p:.4f})")

# Ejemplo
np.random.seed(42)
g1 = np.random.normal(50, 10, 30)
g2 = np.random.normal(55, 10, 30)

diagnostico_completo([g1, g2], ['Control', 'Tratamiento'])

```

Ejercicios Prácticos

1. **Ejercicio 1:** Carga el dataset `iris` de sklearn. Para cada especie, prueba si el largo del sépalo sigue una distribución normal. ¿Qué especie tiene mayor evidencia de no normalidad?
2. **Ejercicio 2:** Compara el largo del pétalo entre las tres especies de iris usando ANOVA. Reporta el estadístico F, valor p y realiza prueba post-hoc Tukey HSD.
3. **Ejercicio 3:** Genera una tabla de contingencia 3x3 con datos simulados. Prueba la independencia con χ^2 . Calcula la V de Cramer.
4. **Ejercicio 4:** Toma 30 sujetos con mediciones antes y después de un tratamiento. Usa un t-test pareado. Compara los resultados con un t-test independiente (incorrecto para este diseño). ¿Cómo cambia el valor p?

Resumen del Capítulo

- **T-test 1 muestra:** compara media contra valor de referencia
- **T-test independiente:** compara dos grupos (usa Welch por defecto)
- **T-test pareado:** compara mediciones del mismo sujeto
- **ANOVA:** compara tres o más grupos; requiere post-hoc
- **Chi-cuadrado:** para variables categóricas (independencia y bondad de ajuste)
- Siempre verifica **supuestos** (normalidad, homogeneidad de varianzas)
- Usa pruebas **no paramétricas** (Mann-Whitney, Kruskal-Wallis) cuando fallen los supuestos
- Reporta el **tamaño del efecto** junto con el valor p

Conceptos clave aprendidos: t-test, Welch, ANOVA, post-hoc, Tukey HSD, chi-cuadrado, Shapiro-Wilk, Levene, Mann-Whitney, Kruskal-Wallis, supuestos

Módulo 4: Diseño Experimental y A/B Testing

Fundamentos del Diseño Experimental

El diseño experimental es la metodología que garantiza que las conclusiones de un experimento sean válidas, reproducibles y no estén contaminadas por factores extraños. Sin un buen diseño, incluso el análisis estadístico más sofisticado no puede salvar un estudio.

Principios Fundamentales

1. Aleatorización

Asignar sujetos a grupos de forma aleatoria elimina el sesgo de selección y distribuye equitativamente las variables de confusión no medidas.

```
import numpy as np
import pandas as pd
from scipy import stats

# Simulación: sin aleatorización, hay sesgo
np.random.seed(42)

# Escenario 1: Asignación NO aleatoria (sesgo)
n = 100
edad = np.random.randint(18, 65, n)
# Asignación sesgada: los primeros 50 son control, los últimos tratamiento
control_no_aleatorio = edad[:50]
trat_no_aleatorio = edad[50:]
print(f"Edad Control (no aleatorio): {np.mean(control_no_aleatorio):.1f}")
print(f"Edad Tratamiento (no aleatorio): {np.mean(trat_no_aleatorio):.1f}")
# Edad Control (no aleatorio): 38.3
# Edad Tratamiento (no aleatorio): 42.5 (diferencia sistemática)

# Escenario 2: Asignación aleatoria
indices = np.random.permutation(n)
control_aleatorio = edad[indices[:50]]
trat_aleatorio = edad[indices[50:]]
print(f"Edad Control (aleatorio): {np.mean(control_aleatorio):.1f}")
print(f"Edad Tratamiento (aleatorio): {np.mean(trat_aleatorio):.1f}")
# Edad Control (aleatorio): 40.1
# Edad Tratamiento (aleatorio): 40.7 (balanceado)
```

2. Control

Tener un grupo de control que no recibe la intervención permite aislar el efecto del tratamiento.

```

# Sin grupo de control: no sabes si el cambio se debe al tratamiento
# o a factores externos (estacionalidad, maduración, etc.)

# Ejemplo: lanzamiento de una campaña de marketing
ventas_antes = np.random.normal(1000, 100, 30)
ventas_despues = np.random.normal(1100, 100, 30)

t_stat, p_val = stats.ttest_ind(ventas_antes, ventas_despues)
print(f"Sin control: t = {t_stat:.2f}, p = {p_val:.4f}")
# ¿El aumento se debe a la campaña o a que es temporada alta?

# Con grupo de control (mercado no expuesto a la campaña)
control_antes = np.random.normal(1000, 100, 30)
control_despues = np.random.normal(1050, 100, 30) # Tendencia natural

# Efecto real = (trat_despues - trat_antes) - (control_despues - control_antes)
efecto_real = (np.mean(ventas_despues) - np.mean(ventas_antes)) - \
              (np.mean(control_despues) - np.mean(control_antes))
print(f"Efecto neto de la campaña (controlando tendencia): {efecto_real:.1f}")

```

3. Réplica

Repetir el experimento en múltiples sujetos y condiciones para estimar la variabilidad y generalizar conclusiones.

```

# Un experimento con 1 solo sujeto por grupo no permite estimar error
# Con réplicas, podemos cuantificar la incertidumbre

def potencia_con_replicas(n_por_grupo, efecto, sigma=1, n_sim=1000):
    """Potencia estadística según número de réplicas"""
    rechazos = 0
    for _ in range(n_sim):
        control = np.random.normal(0, sigma, n_por_grupo)
        trat = np.random.normal(efecto, sigma, n_por_grupo)
        _, p = stats.ttest_ind(control, trat)
        if p < 0.05:
            rechazos += 1
    return rechazos / n_sim

print("Potencia según réplicas por grupo (efecto=0.5):")
for n in [5, 10, 20, 50, 100]:
    pot = potencia_con_replicas(n, efecto=0.5)
    print(f"  n={n:3d}: potencia = {pot:.2%}")

# n= 5: potencia = 21.7%
# n= 10: potencia = 35.9%
# n= 20: potencia = 67.2%
# n= 50: potencia = 96.8%
# n=100: potencia = 100.0%

```

Variables de Confusión

Una variable de confusión (confounder) afecta tanto a la variable independiente como a la

dependiente, creando una asociación espuria:

```
# Ejemplo: correlación entre ventas de helados y ahogamientos
# La confusión es la temperatura (más calor → más helados y más gente en el agua)

def confounder_demo():
    np.random.seed(42)
    temperatura = np.random.uniform(10, 35, 100)
    helados = temperatura * 10 + np.random.normal(0, 20, 100)
    ahogamientos = temperatura * 0.5 + np.random.normal(0, 5, 100)

    # Correlación helados-ahogamientos (sin controlar temperatura)
    r, p = stats.pearsonr(helados, ahogamientos)
    print(f"Correlación helados ~ ahogamientos: r = {r:.2f}, p = {p:.4f}")

    # Correlación parcial (controlando temperatura)
    from scipy import linalg

    def correlacion_parcial(x, y, z):
        """Correlación entre x e y eliminando el efecto de z"""
        r_xy = np.corrcoef(x, y)[0, 1]
        r_xz = np.corrcoef(x, z)[0, 1]
        r_yz = np.corrcoef(y, z)[0, 1]
        return (r_xy - r_xz * r_yz) / np.sqrt((1 - r_xz**2) * (1 - r_yz**2))

    r_parcial = correlacion_parcial(helados, ahogamientos, temperatura)
    print(f"Correlación parcial (controlando temperatura): r = {r_parcial:.2f}")

confounder_demo()
# Correlación helados ~ ahogamientos: r = 0.95, p = 0.0000
# Correlación parcial (controlando temperatura): r = -0.05
```

Tipos de Diseño Experimental

Diseño	Descripción	Cuándo usarlo
Completamente al azar	Sujetos asignados aleatoriamente a grupos	Condiciones homogéneas, sin variables de bloqueo
Bloques aleatorizados	Asignar sujetos similares (bloques) y asignar tratamientos	Hay una fuente de variabilidad conocida (ej. sexo, edad)
Factorial	Evaluar múltiples factores simultáneamente	Quieres estudiar interacciones
Pareado	Cada sujeto es su propio control (antes/después)	Medición directa posible, efecto esperado razonable

```
# Diseño de bloques aleatorizados
np.random.seed(42)

# 4 bloques según nivel de ingresos (bajo, medio, alto, muy alto)
bloques = np.repeat([1, 2, 3, 4], 10) # 10 sujetos por bloque
n = len(bloques)

# Asignación aleatoria DENTRO de cada bloque (cada bloque tiene 5 control + 5 trat)
tratamiento = np.zeros(n, dtype=int)
for bloque in np.unique(bloques):
    idx = np.where(bloques == bloque)[0]
```



```

tratamiento[np.random.choice(idx, len(idx)//2, replace=False)] = 1

# Ventaja: reduces la variabilidad no explicada
print(f"Sujetos por bloque: {np.bincount(bloques)[1:]}")
for b in [1, 2, 3, 4]:
    idx = np.where(bloques == b)[0]
    print(f" Bloque {b}: {np.sum(tratamiento[idx])} tratamiento, {len(idx)-np.sum(tratamiento[idx])} control")

```

Tamaño del Efecto y Relevancia Práctica

La significancia estadística no es lo mismo que la relevancia práctica:

```

# d de Cohen: medida estandarizada del tamaño del efecto
def d_cohen(grupo1, grupo2):
    n1, n2 = len(grupo1), len(grupo2)
    s1, s2 = np.var(grupo1, ddof=1), np.var(grupo2, ddof=1)
    s_pooled = np.sqrt(((n1-1)*s1 + (n2-1)*s2) / (n1 + n2 - 2))
    return (np.mean(grupo2) - np.mean(grupo1)) / s_pooled

# Interpretación de d de Cohen
print("Interpretación de d de Cohen:")
print(f" Pequeño: d ≈ 0.2")
print(f" Mediano: d ≈ 0.5")
print(f" Grande: d ≈ 0.8")

# Ejemplo: efecto pequeño pero n grande
np.random.seed(42)
g1 = np.random.normal(100, 15, 5000)
g2 = np.random.normal(101, 15, 5000) # d ≈ 0.07
_, p = stats.ttest_ind(g1, g2)
print(f"\nEfecto pequeño (d=0.07): p = {p:.6f}, significativo? {p < 0.05}")
# p puede ser < 0.05 (por n grande), pero d es irrelevante

```

> Consejo del Analista:

> Siempre reporta el tamaño del efecto, no solo el valor p. Un efecto puede ser estadísticamente significativo pero tan pequeño que no tenga relevancia práctica. La pregunta correcta no es "¿hay efecto?" sino "¿el efecto es lo suficientemente grande para importar?"

Ejercicios Prácticos

- Ejercicio 1:** Simula un estudio donde la variable de confusión "edad" afecta tanto a la exposición (horas de ejercicio) como al resultado (salud cardiovascular). Muestra la correlación espuria y cómo controlarla.
- Ejercicio 2:** Diseña un experimento de bloques aleatorizados para probar un nuevo método de enseñanza. Los bloques son el nivel de conocimiento previo (bajo, medio, alto). Explica por qué este diseño es superior al completamente al azar.
- Ejercicio 3:** Calcula la potencia estadística para un experimento con $n=25$ por grupo, $\alpha=0.05$ y efecto esperado $d=0.4$. ¿Cuántos sujetos adicionales necesitas para alcanzar 80% de potencia?
- Ejercicio 4:** Explica la diferencia entre significancia estadística y relevancia práctica. Busca

un ejemplo real (artículo, noticia) donde un resultado sea estadísticamente significativo pero el efecto sea trivial.

Resumen del Capítulo

- La **aleatorización** previene el sesgo de selección
- El **grupo de control** aísla el efecto del tratamiento
- Las **réplicas** permiten estimar la variabilidad
- Las **variables de confusión** crean asociaciones espurias
- Los **bloques** reducen la variabilidad no explicada
- El **tamaño del efecto** (d de Cohen) mide la relevancia práctica
- La **potencia** depende de n, α y el tamaño del efecto
- Un diseño robusto es requisito para conclusiones válidas

Conceptos clave aprendidos: aleatorización, control, réplica, confusión, bloques, d de Cohen, potencia estadística, validez interna/externa

Cómo Configurar y Analizar un A/B Test en Python

El A/B testing es la aplicación práctica del diseño experimental en entornos digitales. Es la herramienta estándar para tomar decisiones basadas en datos: ¿qué color de botón convierte más? ¿qué precio genera más ingresos? ¿qué titular atrae más clics?

El Flujo Completo de un A/B Test

1. Formular hipótesis
2. Determinar tamaño de muestra
3. Asignar aleatoriamente
4. Ejecutar el experimento
5. Analizar resultados
6. Concluir y actuar

1. Formular Hipótesis

```
import numpy as np
from scipy import stats
import pandas as pd

# Ejemplo: tasa de conversión de un botón
# Página actual (control): botón azul → tasa de conversión 5%
# Propuesta (tratamiento): botón rojo → esperamos 6%

# H0: p_rojo = p_azul (no hay diferencia)
# H1: p_rojo ≠ p_azul (hay diferencia) - bilateral
# O: H1: p_rojo > p_azul (el rojo es mejor) - unilateral
```

2. Determinar Tamaño de Muestra

El tamaño de muestra necesario depende de:

- Línea base (tasa de conversión actual)
- Efecto mínimo detectable (MDE)
- Nivel de significancia (α)
- Potencia deseada ($1-\beta$)

```

def tamano_muestra_ab(p_base, efecto_minimo, alpha=0.05, potencia=0.80):
    """
    Calcula n por grupo para un A/B test de proporciones.
    p_base: tasa de conversión actual (control)
    efecto_minimo: mejora mínima que queremos detectar (ej. 0.01 = 1pp)
    """
    p_trat = p_base + efecto_minimo
    p_promedio = (p_base + p_trat) / 2

    z_alpha = stats.norm.ppf(1 - alpha / 2)
    z_beta = stats.norm.ppf(potencia)

    n = (z_alpha * np.sqrt(2 * p_promedio * (1 - p_promedio)) +
         z_beta * np.sqrt(p_base * (1 - p_base) + p_trat * (1 - p_trat)))*2
    n = n / (efecto_minimo**2)

    return int(np.ceil(n))

# Escenarios típicos
print("Tamaño de muestra por grupo (α=0.05, potencia=80%):")
for base in [0.01, 0.05, 0.10, 0.50]:
    for mde in [0.01, 0.02, 0.05]:
        n = tamano_muestra_ab(base, mde)
        print(f" Base={base:.0%}, MDE={mde:.0%}: n={n}")
# Base=1%, MDE=1pp: n=12,714
# Base=5%, MDE=1pp: n=7,284
# Base=5%, MDE=5pp: n=1,292
# Base=50%, MDE=5pp: n=1,554

```

> **Advertencia:**

> No calcules el tamaño de muestra después de terminar el experimento. El tamaño de muestra debe determinarse *antes* de comenzar, basado en el efecto mínimo que te importa detectar.

3. Asignación Aleatoria

```

def asignar_ab(usuarios, prob_tratamiento=0.5, semilla=42):
    """Asigna usuarios a grupos control y tratamiento"""
    np.random.seed(semilla)
    n = len(usuarios)
    asignacion = np.random.choice(
        ['control', 'tratamiento'],
        size=n,
        p=[1 - prob_tratamiento, prob_tratamiento]
    )
    return pd.DataFrame({
        'usuario_id': usuarios,
        'grupo': asignacion
    })

# Simular 5000 usuarios
usuarios = np.arange(1, 5001)

```

```
df_asignacion = asignar_ab(usuarios)
print(df_asignacion['grupo'].value_counts())
# control      2518
# tratamiento  2482
```

Verificación de Balance

Antes de analizar, verifica que los grupos sean comparables:

```
def verificar_balance(df, grupo_col='grupo'):
    """Verifica que la asignación sea balanceada en características conocidas"""
    # Prueba chi-cuadrado para verificar proporción 50/50
    conteos = df[grupo_col].value_counts()
    chi2, p = stats.chisquare(conteos.values)
    print(f"Test de balance:  $\chi^2 = \{chi2:.2f\}$ ,  $p = \{p:.4f\}$ ")

    if p > 0.05:
        print(" Los grupos están balanceados")
    else:
        print(" Posible desbalance en la asignación")

verificar_balance(df_asignacion)
# Test de balance:  $\chi^2 = 0.26$ ,  $p = 0.6100$ 
# Los grupos están balanceados
```

4. Simulación del Experimento

```
np.random.seed(123)

n_control = 2500
n_tratamiento = 2500
p_control = 0.05      # 5% de conversión
p_tratamiento = 0.06  # 6% de conversión (lift del 20%)

# Generar resultados
control = np.random.binomial(1, p_control, n_control)
tratamiento = np.random.binomial(1, p_tratamiento, n_tratamiento)

df_resultados = pd.DataFrame({
    'grupo': ['control'] * n_control + ['tratamiento'] * n_tratamiento,
    'conversión': np.concatenate([control, tratamiento])
})

print("Resultados del experimento:")
print(df_resultados.groupby('grupo')['conversión'].agg(['count', 'mean', 'sum']))
```

5. Análisis Estadístico

Test de Proporciones (Z-test)

```
def analizar_ab(df, grupo_col='grupo', outcome_col='conversión',
```

```

        control_name='control', alpha=0.05):
    """Análisis completo de un A/B test"""

    # Tasas de conversión
    tasas = df.groupby(grupo_col)[outcome_col].mean()
    n_por_grupo = df.groupby(grupo_col).size()

    p_control = tasas[control_name]
    p_trat = tasas[[g for g in tasas.index if g != control_name][0]]
    n_control = n_por_grupo[control_name]
    n_trat = n_por_grupo[[g for g in n_por_grupo.index if g != control_name][0]]

    # Diferencia observada
    diff = p_trat - p_control

    # Error estándar de la diferencia
    se = np.sqrt(p_control * (1 - p_control) / n_control +
                 p_trat * (1 - p_trat) / n_trat)

    # Z-statistic
    z_stat = diff / se

    # Valor p (bilateral)
    p_valor = 2 * (1 - stats.norm.cdf(abs(z_stat)))

    # Intervalo de confianza
    z_crit = stats.norm.ppf(1 - alpha / 2)
    ic_inf = diff - z_crit * se
    ic_sup = diff + z_crit * se

    # Lift relativo
    lift = diff / p_control

    print("=== Resultados A/B Test ===\n")
    print(f"Control (n={n_control}): {p_control:.4f} ({p_control*100:.2f}%)"
    print(f"Tratamiento (n={n_trat}): {p_trat:.4f} ({p_trat*100:.2f}%)"
    print(f"Diferencia absoluta: {diff:.4f} ({diff*100:.2f}pp)"
    print(f"Lift relativo: {lift:.2%}"
    print(f"Z-stat: {z_stat:.4f}"
    print(f"Valor p: {p_valor:.4f}"
    print(f"IC {1-alpha:.0%}: [{ic_inf:.4f}, {ic_sup:.4f}]")

    if p_valor < alpha:
        print(f"\n El resultado es estadísticamente significativo (p < {alpha})")
    else:
        print(f"\n El resultado NO es estadísticamente significativo (p >= {alpha})")

    return {
        'p_control': p_control,
        'p_tratamiento': p_trat,
        'diferencia': diff,
        'lift': lift,

```

```

        'z_stat': z_stat,
        'p_valor': p_valor,
        'ic': (ic_inf, ic_sup)
    }

    resultado = analizar_ab(df_resultados)

```

Test Exacto de Fisher (para muestras pequeñas)

Cuando las frecuencias esperadas son pequeñas (<5 en alguna celda), Fisher es más preciso:

```

from scipy.stats import fisher_exact

# Tabla de contingencia
tabla = pd.crosstab(df_resultados['grupo'], df_resultados['conversión'])
print(tabla)

odds_ratio, p_fisher = fisher_exact(tabla)
print(f"\nTest exacto de Fisher:")
print(f" Odds Ratio: {odds_ratio:.4f}")
print(f" Valor p: {p_fisher:.4f}")

```

6. Métricas Bayesianas (Alternativa)

El enfoque bayesiano proporciona distribuciones de probabilidad directamente interpretables:

```

from scipy.stats import beta

def analisis_bayesiano_ab(conversiones_control, n_control,
                          conversiones_trat, n_trat,
                          n_simulaciones=100000):
    """A/B test con enfoque bayesiano (Beta-Binomial)"""

    # Priors no informativos Beta(1, 1)
    a_prior, b_prior = 1, 1

    # Posteriors
    posterior_control = beta(a_prior + conversiones_control,
                             b_prior + n_control - conversiones_control)
    posterior_trat = beta(a_prior + conversiones_trat,
                          b_prior + n_trat - conversiones_trat)

    # Simulación de la diferencia
    muestras_control = posterior_control.rvs(n_simulaciones)
    muestras_trat = posterior_trat.rvs(n_simulaciones)
    diferencias = muestras_trat - muestras_control

    # Probabilidad de que tratamiento sea mejor
    prob_trat_mejor = np.mean(diferencias > 0)

    # Intervalo de mayor densidad (HDI)
    hdi_inf, hdi_sup = np.percentile(diferencias, [2.5, 97.5])

```

```

print("=== Análisis Bayesiano ===")
print(f"P(tratamiento > control) = {prob_trat_mejor:.4f}")
print(f"IC Bayesiano 95% (HDI): [{hdi_inf:.4f}, {hdi_sup:.4f}]")

if prob_trat_mejor > 0.95:
    print(" Alta probabilidad de que el tratamiento sea superior")
else:
    print(" Evidencia insuficiente para declarar superioridad")

# Aplicar
conv_control = control.sum()
conv_trat = tratamiento.sum()
analisis_bayesiano_ab(conv_control, n_control, conv_trat, n_trat)

```

> Consejo del Analista:

> El enfoque bayesiano responde directamente a la pregunta que realmente te interesa: "¿qué probabilidad hay de que el tratamiento sea mejor?" No requiere valores p ni conceptos de "rechazar H_0 ".

Regla de Decisión y Siguietes Pasos

```

def decidir_accion(resultado, costo_implementacion=0.02, alpha=0.05):
    """
    Decide si implementar el cambio basado en estadística y negocio.

    costo_implementacion: mejora mínima en pp para justificar el cambio
    """
    if resultado['p_valor'] < alpha and resultado['diferencia'] > costo_implementacion:
        return "IMPLEMENTAR: el cambio es significativo y rentable"
    elif resultado['p_valor'] < alpha and resultado['diferencia'] <= costo_implementacion:
        return "EVALUAR: significativo pero el efecto no justifica el costo"
    elif resultado['p_valor'] >= alpha and resultado['diferencia'] > costo_implementacion:
        return "RECOLECTAR MÁS DATOS: efecto prometedor pero no concluyente"
    else:
        return "NO IMPLEMENTAR: no hay evidencia de mejora"

decision = decidir_accion(resultado, costo_implementacion=0.005)
print(f"\nDecisión: {decision}")

```

Errores Comunes que Evitar

```

# 1. Mirar los datos antes de tiempo (peeking)
# NO hagas análisis intermedio sin ajustar por comparaciones múltiples
# Solución: establecer un tamaño de muestra fijo antes del experimento

# 2. Múltiples comparaciones sin ajuste
# Si pruebas 20 variantes al mismo tiempo, el error Tipo I se dispara
# Solución: corrección de Bonferroni o métodos FDR

from statsmodels.stats.multitest import multipletests
p_valores_ejemplo = [0.04, 0.03, 0.06, 0.01, 0.20, 0.05, 0.001]

```



```
rechazar, p_ajustados, _, _ = multipletests(p_valores_ejemplo, method='bonferroni')
print("Corrección Bonferroni:")
for p_orig, p_ajust, sig in zip(p_valores_ejemplo, p_ajustados, rechazar):
    print(f" p={p_orig:.3f} → p_ajust={p_ajust:.3f} {' ' if sig else ''}")
```

Ejercicios Prácticos

1. **Ejercicio 1:** Simula un A/B test con $p_{\text{control}}=0.08$, $p_{\text{tratamiento}}=0.09$, $n=5000$ por grupo. Ejecuta el análisis completo. ¿El resultado es significativo? Calcula el lift.
2. **Ejercicio 2:** Usando la función `tamano_muestra_ab``, calcula cuántos usuarios necesitas por grupo para detectar un MDE de 2pp con una base de 10%. Varía la potencia (0.70, 0.80, 0.90) y α (0.01, 0.05, 0.10).
3. **Ejercicio 3:** Implementa una simulación de Monte Carlo donde evalúes 1000 A/B tests bajo H_0 verdadera (sin diferencia). ¿Qué proporción produce $p < 0.05$? ¿Qué pasa si haces "peeking" y detienes el experimento cuando $p < 0.05$ en cualquier punto?
4. **Ejercicio 4:** Para el mismo experimento, compara el enfoque frecuentista (z-test) con el bayesiano. ¿Llegan a la misma conclusión? ¿Cuándo diferirían?

Resumen del Capítulo

- El A/B testing compara dos versiones para determinar cuál es superior
 - Calcula el **tamaño de muestra** antes del experimento
 - La **asignación aleatoria** garantiza grupos comparables
 - El **Z-test de proporciones** es la prueba estándar para tasas de conversión
 - El **lift relativo** y la **diferencia absoluta** son métricas complementarias
 - El **enfoque bayesiano** responde directamente a "¿qué probabilidad hay de que gane?"
 - El **peeking** (mirar antes de tiempo) infla los errores Tipo I
 - Siempre ajusta por **comparaciones múltiples** si pruebas varias variantes

Conceptos clave aprendidos: MDE, tamaño de muestra, Z-test de proporciones, lift, odds ratio, Fisher exact test, análisis bayesiano, peeking, Bonferroni

Errores Comunes en A/B Testing

El A/B testing parece simple: divides usuarios, muestras dos versiones, comparas resultados. Pero los errores sutiles invalidan la mayoría de los experimentos mal diseñados. Este capítulo cataloga los errores más frecuentes y cómo evitarlos.

Error 1: Peeking (Mirar los Datos Antes de Tiempo)

Es el error más común. Detener el experimento cuando el valor p cruza 0.05, sin importar el tamaño de muestra acumulado.

```
import numpy as np
from scipy import stats
import pandas as pd

# Simulación de peeking
np.random.seed(42)

def simular_peeking(p_control=0.05, p_trat=0.05, max_n=5000,
                    checkpoints=50, alpha=0.05):
    """
    Simula un A/B test con peeking.
    Bajo H0 (p_control = p_trat), el peeking infla el error Tipo I.
    """
    n_por_paso = max_n // checkpoints
    resultados_acum = {'control': [], 'tratamiento': []}

    for paso in range(checkpoints):
        # Nuevos datos en cada paso
        nuevos_control = np.random.binomial(1, p_control, n_por_paso)
        nuevos_trat = np.random.binomial(1, p_trat, n_por_paso)

        resultados_acum['control'].extend(nuevos_control)
        resultados_acum['tratamiento'].extend(nuevos_trat)

        # Test en cada paso
        _, p_valor = stats.ttest_ind(
            resultados_acum['control'],
            resultados_acum['tratamiento']
        )

        if p_valor < alpha:
            return True, p_valor, (paso + 1) * n_por_paso
```

```

        return False, p_valor, max_n

# Simular bajo H0 (sin efecto real)
np.random.seed(42)
n_sim = 500
peeking_positivos = 0

for _ in range(n_sim):
    detectado, _, _ = simular_peeking(0.05, 0.05, max_n=5000)
    if detectado:
        peeking_positivos += 1

print(f"Error Tipo I con peeking: {peeking_positivos/n_sim:.4f}")
print(f"Error Tipo I teórico (sin peeking): 0.0500")
# Error Tipo I con peeking: ~0.25 (5 INFLADO veces!)

```

> Consejo del Analista:

> Si debes monitorear el experimento en tiempo real, usa un *límite de gasto alfa* (Pocock, O'Brien-Fleming) o un enfoque bayesiano que no sufra de este problema.

Error 2: Múltiples Comparaciones

Cuantas más variantes pruebes, mayor probabilidad de encontrar un falso positivo.

```

# Problema: probar 20 variantes contra control
n_variantes = 20
alpha = 0.05

# Probabilidad de al menos un falso positivo
p_al_menos_un_falso = 1 - (1 - alpha) ** n_variantes
print(f"P(al menos 1 falso positivo) con {n_variantes} variantes: {p_al_menos_un_falso:.3f}")
# P(al menos 1 falso positivo) con 20 variantes: 0.642 (64%)

# Solución: corrección de Bonferroni
alpha_ajustado = alpha / n_variantes
print(f"α ajustado (Bonferroni): {alpha_ajustado:.4f}")
# α ajustado (Bonferroni): 0.0025

```

Error 3: No Calcular el Tamaño de Muestra Anticipadamente

```

# Escenario: ejecutas un test con n=500 por grupo
# El efecto real es pequeño (1pp de diferencia con base 5%)
n_actual = 500
base = 0.05
efecto = 0.01

poder = stats.FisherExactTest # Placeholder
# Cálculo real:
from statsmodels.stats.power import NormalIndPower
power_analysis = NormalIndPower()

```

```

poder_real = power_analysis.solve_power(
    effect_size=efecto / np.sqrt(base * (1 - base)),
    nobs1=n_actual,
    alpha=0.05,
    ratio=1,
    alternative='two-sided'
)
print(f"Potencia con n=500: {poder_real:.2%}")
# Potencia con n=500: ~15% (de 100 experimentos, solo 15 detectarán el efecto)

```

Error 4: Asignación No Aleatoria

```

# Asignación por día de la semana (NO aleatoria)
# Los usuarios de fin de semana se comportan diferente

def sesgo_por_tiempo():
    np.random.seed(42)
    n_dias = 30

    # Tratamiento solo en fines de semana
    es_finde = np.array([i % 7 >= 5 for i in range(n_dias)])
    grupo = np.where(es_finde, 'tratamiento', 'control')

    # Tasa de conversión naturalmente más alta en fines
    conversion = np.where(
        es_finde,
        np.random.binomial(1, 0.08, n_dias), # Fines: 8%
        np.random.binomial(1, 0.05, n_dias)  # Semana: 5%
    )

    # El análisis mostrará diferencia... pero es por el día, no por el tratamiento
    p_control = conversion[grupo == 'control'].mean()
    p_trat = conversion[grupo == 'tratamiento'].mean()
    print(f"Control: {p_control:.3f}, Tratamiento: {p_trat:.3f}")
    print(f"Diferencia espuria: {p_trat - p_control:.3f}")

sesgo_por_tiempo()

```

Error 5: No Controlar por Efectos de Temporada

```

# Ejemplo: prueba de 2 semanas, pero la primera semana es navidad
np.random.seed(42)

semana1 = np.random.normal(1000, 50, 7) # Navidad: tráfico alto
semana2 = np.random.normal(700, 50, 7)  # Normal: tráfico bajo

# Si muestras control en semana 1 y tratamiento en semana 2
# La diferencia se deberá a la temporada, no al tratamiento
print(f"Semana 1 (Navidad): media = {np.mean(semana1):.0f}")
print(f"Semana 2 (Normal): media = {np.mean(semana2):.0f}")

```

```
print(f"Diferencia (espuria): {np.mean(semana1) - np.mean(semana2):.0f}")
```

Solución: ejecutar control y tratamiento simultáneamente, con asignación aleatoria continua.

Error 6: Contaminación entre Grupos

Cuando los usuarios del grupo tratamiento influyen en los del control:

```
# Ejemplo: red social donde usuarios del grupo A ven contenido del grupo B
# Solución: aislar los grupos (por ejemplo, por país o región)
```

Error 7: Efecto Novedad (Novelty Effect)

Los usuarios reaccionan positivamente a cualquier cambio, solo porque es nuevo:

```
# El efecto positivo inicial desaparece con el tiempo
# Solución: ejecutar el experimento el tiempo suficiente para que el efecto se estabilice

def efecto_novedad(n_dias=60):
    np.random.seed(42)
    # El tratamiento tiene un efecto inicial grande que decae
    dias = np.arange(n_dias)
    efecto = 0.15 * np.exp(-dias / 15) # Decaimiento exponencial
    return dias, efecto

# Si detienes el test temprano, el efecto parece grande pero es insostenible
```

Error 8: Segmentación Post-hoc

Encontrar segmentos donde el tratamiento "funciona" después de ver los datos:

```
# Problema: si pruebas 20 segmentos, 1 será significativo por azar
# Solución: prueba de interacción (ANOVA de dos factores) antes de segmentar

def segmentacion_posthoc():
    np.random.seed(42)
    n_segmentos = 20
    n_por_grupo = 200

    # Generar datos donde NO hay efecto en ningún segmento
    p_valores = []
    for _ in range(n_segmentos):
        control = np.random.binomial(1, 0.05, n_por_grupo)
        trat = np.random.binomial(1, 0.05, n_por_grupo)
        _, p = stats.ttest_ind(control, trat)
        p_valores.append(p)

    significativos = sum(p < 0.05 for p in p_valores)
    print(f"De {n_segmentos} segmentos, {significativos} son 'significativos' por azar")

segmentacion_posthoc()
```

Checklist para un A/B Test Válido

Requisito	Por qué	Cómo verificarlo
Tamaño de muestra calculado pre-experimento	Evita falsos positivos por peeking	Documentar n antes de empezar
Asignación aleatoria	Elimina sesgo de selección	Test χ^2 de balance
Grupos simultáneos	Controla estacionalidad	Verificar fechas
Sin contaminación	Aísla el efecto del tratamiento	Diseño del experimento
Duración suficiente (≥ 1 ciclo completo)	Captura variabilidad semanal	Mínimo 7-14 días
Corrección por múltiples comparaciones	Controla error Tipo I global	Bonferroni, FDR, etc.
Análisis pre-registrado	Evita p-hacking	Documentar plan de análisis

Ejercicios Prácticos

1. **Ejercicio 1:** Simula 1000 experimentos con peeking y sin peeking. Compara las tasas de error Tipo I. ¿Cuánto se infla con peeking agresivo (revisión cada 10 observaciones)?
2. **Ejercicio 2:** Diseña un A/B test para una página web. Identifica potenciales fuentes de contaminación entre grupos y explica cómo las mitigarías.
3. **Ejercicio 3:** Un equipo ejecuta un test con 5 variantes y encuentra que 2 son significativas ($p < 0.05$). Aplica la corrección de Bonferroni. ¿Siguen siendo significativas?
4. **Ejercicio 4:** Simula un efecto novedad: un tratamiento que parece efectivo en los primeros 3 días pero cuyo efecto desaparece después de 2 semanas. ¿Cómo diseñarías el experimento para detectar esto?

Resumen del Capítulo

- El **peeking** infla el error Tipo I (puede llegar a 25%+)
- Las **múltiples comparaciones** requieren corrección (Bonferroni, FDR)
- El **tamaño de muestra** debe calcularse antes del experimento
- La **asignación no aleatoria** introduce sesgos irreparables
- La **estacionalidad** y los **efectos de novedad** enmascaran resultados
- La **contaminación entre grupos** invalida la independencia
- La **segmentación post-hoc** sin ajuste produce falsos descubrimientos
- Usa un **checklist** de validación antes de lanzar cualquier experimento

Conceptos clave aprendidos: peeking, comparaciones múltiples, Bonferroni, contaminación, efecto novedad, segmentación post-hoc, p-hacking, pre-registro

Módulo 5: Modelado Predictivo Básico

Correlación vs Causalidad

"Correlación no implica causalidad" es la frase más repetida (y más ignorada) en análisis de datos. Este capítulo te da las herramientas para distinguir entre asociación y causalidad, y para identificar cuándo puedes (y cuándo no) hacer afirmaciones causales.

Correlación: Midiendo Asociación

La *correlación* mide la fuerza y dirección de una relación lineal entre dos variables.

Correlación de Pearson

Asume relación lineal y datos continuos:

```
import numpy as np
from scipy import stats
import pandas as pd

np.random.seed(42)

# Correlación positiva fuerte
x = np.random.normal(50, 10, 100)
y = 2 * x + np.random.normal(0, 10, 100)

r_pearson, p_valor = stats.pearsonr(x, y)
print(f"Correlación de Pearson: r = {r_pearson:.4f}, p = {p_valor:.4e}")
# r = 0.8938, p < 0.001 (fuerte y significativa)
```

Correlación de Spearman

No asume linealidad; usa rangos. Robusta a outliers:

```
# Relación monótona no lineal
x = np.random.uniform(1, 10, 100)
y = np.exp(x) + np.random.normal(0, 50, 100)

r_pearson, _ = stats.pearsonr(x, y)
r_spearman, _ = stats.spearmanr(x, y)

print(f"Pearson: r = {r_pearson:.4f}")
print(f"Spearman: p = {r_spearman:.4f}")
# Pearson puede subestimar si la relación no es lineal
# Spearman captura la relación monótona
```

Matriz de Correlación

```
import pandas as pd

# Dataset con múltiples variables correlacionadas
np.random.seed(42)
n = 100
datos = pd.DataFrame({
    'ingresos': np.random.normal(50000, 10000, n),
    'edad': np.random.normal(40, 12, n),
    'gastos': np.random.normal(30000, 8000, n),
})

# Introducir correlaciones
datos['gastos'] = datos['ingresos'] * 0.6 + np.random.normal(0, 3000, n)
datos['ahorros'] = datos['ingresos'] - datos['gastos'] + np.random.normal(0, 2000, n)

matriz_corr = datos.corr()
print("Matriz de correlación:")
print(matriz_corr.round(3))
#           ingresos  edad  gastos  ahorros
# ingresos      1.000  0.021   0.886   0.355
# edad          0.021  1.000   0.018   0.010
# gastos        0.886  0.018   1.000  -0.120
# ahorros       0.355  0.010  -0.120   1.000
```

Por Qué Correlación No Es Causalidad

1. Causalidad Inversa

```
# Ejemplo: ¿más bomberos causan más daños?
# 0: ¿más incendios graves requieren más bomberos?

# Simulación
np.random.seed(42)
n = 50
gravedad_incendio = np.random.exponential(3, n) # Tamaño del incendio
bomberos = 5 + 2 * gravedad_incendio + np.random.normal(0, 2, n) # Más bomberos si es grave
danos = 100 * gravedad_incendio + np.random.normal(0, 10, n) # Más daño si es grave

r, p = stats.pearsonr(bomberos, danos)
print(f"Correlación bomberos-daños: r = {r:.4f}, p = {p:.4f}")
# r = 0.98 (fuerte correlación positiva)
# Pero la causalidad es inversa: incendios graves → más bomberos Y más daños
```

2. Variable de Confusión

```
# Ejemplo clásico: helados y ahogamientos
# La confusión es la temperatura (variable oculta)

def confundir():
    np.random.seed(42)
    temperatura = np.random.uniform(10, 35, 200)
```



```

helados = 10 * temperatura + np.random.normal(0, 30, 200)
ahogamientos = 0.3 * temperatura + np.random.normal(0, 2, 200)

r, p = stats.pearsonr(helados, ahogamientos)
print(f"Correlación helados-ahogamientos: r = {r:.4f}")
# r ≈ 0.95 (falsa causalidad)

# Control por temperatura (correlación parcial)
def correlacion_parcial(x, y, z):
    r_xy = np.corrcoef(x, y)[0, 1]
    r_xz = np.corrcoef(x, z)[0, 1]
    r_yz = np.corrcoef(y, z)[0, 1]
    return (r_xy - r_xz * r_yz) / np.sqrt((1 - r_xz**2) * (1 - r_yz**2))

r_parcial = correlacion_parcial(helados, ahogamientos, temperatura)
print(f"Correlación parcial (controlando temp): r = {r_parcial:.4f}")
# r ≈ 0.01 (desaparece)

confundir()

```

3. Sesgo de Selección

```

# Ejemplo: los pacientes que toman el medicamento viven más...
# ... porque los pacientes más saludables tienden a tomar el medicamento

def sesgo_seleccion():
    np.random.seed(42)
    n = 500

    # Salud base (no observada)
    salud_base = np.random.normal(0, 1, n)

    # Decisión de tomar medicamento (sesgada: los más sanos lo toman)
    toma_med = (salud_base + np.random.normal(0, 0.5, n)) > 0

    # Resultado: años de vida restantes
    anos_vida = 20 + 5 * salud_base + 2 * toma_med + np.random.normal(0, 2, n)

    # Análisis ingenuo
    grupo_med = anos_vida[toma_med]
    grupo_no_med = anos_vida[~toma_med]
    print(f"Toman medicamento: media = {np.mean(grupo_med):.1f} años")
    print(f"No toman: media = {np.mean(grupo_no_med):.1f} años")
    print(f"Diferencia: {np.mean(grupo_med) - np.mean(grupo_no_med):.1f} años")
    # El efecto está inflado por el sesgo de selección

sesgo_seleccion()

```

Criterios de Bradford Hill

Para fortalecer un argumento causal, Bradford Hill propuso 9 criterios (1965):

Criterio	Descripción
Fuerza	Asociación grande → más probable causal
Consistencia	Reproducible en diferentes estudios/poblaciones
Especificidad	Causa específica → efecto específico
Temporalidad	La causa precede al efecto (el más importante)
Gradiente biológico	Relación dosis-respuesta
Plausibilidad	Mecanismo explicable
Coherencia	No contradice el conocimiento existente
Experimento	Evidencia experimental
Analogía	Efectos similares tienen causas similares

Diseños para Establecer Causalidad

Diseño	Validez causal	Factibilidad
RCT (experimento aleatorizado)		
Experimentos naturales		
Diferencia en diferencias (DiD)		
Variables instrumentales (IV)		
Regresión discontinua (RDD)		
Cohortes prospectivos		
Casos y controles		
Transversales		

```
# Simulación de Diferencia en Diferencias (DiD)
np.random.seed(42)

# Dos regiones: tratada y control, antes y después
n_periodo = 200
periodo = np.array([0] * n_periodo + [1] * n_periodo)
tratada = np.array([1] * n_periodo + [0] * n_periodo)
control = np.array([0] * n_periodo + [0] * n_periodo)

# Resultado:  $y = \beta_0 + \beta_1 \cdot \text{post} + \beta_2 \cdot \text{tratada} + \beta_3 \cdot (\text{post} \times \text{tratada})$ 
y_control = 50 + 5 * periodo + np.random.normal(0, 5, n_periodo * 2)
y_tratada = 50 + 5 * periodo + 10 * (periodo == 1) + np.random.normal(0, 5, n_periodo * 2)

# El coeficiente de interacción ( $\beta_3 = 10$ ) es el efecto causal estimado
import statsmodels.api as sm

df_did = pd.DataFrame({
    'y': np.concatenate([y_control, y_tratada]),
    'post': np.tile(periodo, 2),
    'tratada': np.repeat([0, 1], n_periodo * 2),
    'interaccion': np.tile(periodo, 2) * np.repeat([0, 1], n_periodo * 2)
})

modelo = sm.OLS.from_formula('y ~ post + tratada + interaccion', data=df_did).fit()
print(modelo.params)

# interaccion ≈ 10.0 → efecto causal estimado
```

> Advertencia:

> Ningún método observacional puede probar causalidad con certeza absoluta. Incluso los RCT tienen limitaciones (validez externa, incumplimiento, etc.). La causalidad es un juicio que combina evidencia estadística, diseño y conocimiento del dominio.

Correlación Espuria: El Humor de Datos

```
# Las correlaciones espurias famosas
# - Consumo de queso per cápita ~ muertes por enredarse en sábanas
# - Número de ahogados en piscinas ~ películas de Nicolas Cage

def correlacion_espuria(n=50):
    np.random.seed(42)
    # Dos series temporales independientes
    t = np.arange(n)
    serie1 = 100 + 0.5 * t + np.random.normal(0, 10, n) # Tendencia
    serie2 = 50 + 0.3 * t + np.random.normal(0, 8, n)   # Tendencia

    r, p = stats.pearsonr(serie1, serie2)
    print(f"Correlación entre series con tendencia: r = {r:.4f}, p = {p:.4f}")
    # r ≈ 0.90 (espuria: ambas tienen tendencia temporal)

    # Solución: analizar diferencias o residuales
    serie1_diff = np.diff(serie1)
    serie2_diff = np.diff(serie2)
    r_diff, p_diff = stats.pearsonr(serie1_diff, serie2_diff)
    print(f"Correlación de diferencias: r = {r_diff:.4f}, p = {p_diff:.4f}")
    # r ≈ 0.0 (la correlación espuria desaparece)

correlacion_espuria()
```

Ejercicios Prácticos

1. **Ejercicio 1:** Encuentra 3 ejemplos de correlaciones espurias en datos reales o simulados. Explica la posible variable de confusión en cada caso.
2. **Ejercicio 2:** Simula un escenario de causalidad inversa: los hospitales con más camas tienen mayor mortalidad. ¿La causalidad es que más camas causan muerte? Explica.
3. **Ejercicio 3:** Implementa un análisis de diferencia en diferencias con datos simulados. Crea un escenario donde una política implementada en una región (y no en otra vecina) produce un cambio en el resultado de interés.
4. **Ejercicio 4:** Usando el dataset `mtcars` (o cualquier otro), identifica pares de variables con alta correlación pero donde claramente no hay relación causal. Explica por qué.

Resumen del Capítulo

- La **correlación** mide asociación, no causalidad
- **Pearson** asume linealidad; **Spearman** usa rangos (no paramétrica)

- **Causalidad inversa:** A causa B, o B causa A
- **Variables de confusión:** Z causa tanto a X como a Y
- **Sesgo de selección:** la muestra no representa a la población
- Los **criterios de Bradford Hill** guían el juicio causal
- **RCTs** son el estándar de oro para causalidad
- **DiD, IV, RDD** son alternativas observacionales
- Las **tendencias temporales** producen correlaciones espurias

Conceptos clave aprendidos: correlación de Pearson/Spearman, causalidad inversa, confusión, sesgo de selección, Bradford Hill, RCT, DiD, variable instrumental, correlación espuria

Regresión Lineal Simple y Múltiple

La regresión lineal es el punto de partida de casi todo modelado predictivo. Es simple, interpretable y sorprendentemente poderosa. En este capítulo entenderás cómo funciona por dentro, no solo cómo llamarla desde sklearn.

El Modelo de Regresión Lineal

La regresión lineal modela la relación entre una variable dependiente (Y) y una o más variables independientes (X):

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_X + \varepsilon$$

Donde β_0 es el *intercepto*, β son los *coeficientes* y ε es el *error* (lo que no podemos explicar).

Regresión Lineal Simple

Desde Cero con Mínimos Cuadrados

```
import numpy as np
from scipy import stats
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

# Datos simulados: horas de estudio → puntuación en examen
horas = np.random.uniform(0, 10, 100)
puntuacion = 30 + 5 * horas + np.random.normal(0, 8, 100)

# Cálculo manual de coeficientes
def regresion_lineal_simple(x, y):
    n = len(x)
    x_mean, y_mean = np.mean(x), np.mean(y)

    #  $\beta_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$ 
    beta_1 = np.sum((x - x_mean) * (y - y_mean)) / np.sum((x - x_mean)**2)

    #  $\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$ 
    beta_0 = y_mean - beta_1 * x_mean

    return beta_0, beta_1
```

```

beta_0, beta_1 = regresion_lineal_simple(horas, puntuacion)
print(f"Intercepto ( $\beta_0$ ): {beta_0:.2f}")
print(f"Pendiente ( $\beta_1$ ): {beta_1:.2f}")
print(f"Interpretación: por cada hora extra, la puntuación aumenta {beta_1:.2f} puntos")
# Intercepto ( $\beta_0$ ): 29.85
# Pendiente ( $\beta_1$ ): 5.05

```

Con Scikit-learn

```

from sklearn.linear_model import LinearRegression

X = horas.reshape(-1, 1) # sklearn requiere 2D
modelo = LinearRegression()
modelo.fit(X, puntuacion)

print(f" $\beta_0$ : {modelo.intercept_:.2f}")
print(f" $\beta_1$ : {modelo.coef_[0]:.2f}")
# Mismos resultados

# Predicción
predicciones = modelo.predict(X)
print(f"Primeras 5 predicciones: {np.round(predicciones[:5], 1)}")

```

Supuestos de la Regresión Lineal

La validez del modelo depende de estos supuestos:

```

def verificar_supuestos(modelo, X, y):
    """Verifica los supuestos de la regresión lineal"""
    y_pred = modelo.predict(X)
    residuos = y - y_pred

    # 1. Linealidad: residuos vs predicciones (sin patrón)
    # 2. Homocedasticidad: varianza constante de residuos
    # 3. Normalidad de residuos
    # 4. Independencia de residuos

    # Test de Normalidad (Shapiro-Wilk)
    _, p_normalidad = stats.shapiro(residuos)

    # Test de Homocedasticidad (Breusch-Pagan)
    # Aproximación: correlación entre |residuos| y predicciones
    r_hetero, p_hetero = stats.spearmanr(np.abs(residuos), y_pred)

    print("Verificación de supuestos:")
    print(f" Normalidad (Shapiro): p = {p_normalidad:.4f} {' ' if p_normalidad > 0.05 else ''}")
    print(f" Homocedasticidad: p = {p_hetero:.4f} {' ' if p_hetero > 0.05 else ''}")

verificar_supuestos(modelo, X, puntuacion)
# Normalidad: p > 0.05
# Homocedasticidad: p > 0.05

```

Supuesto	Qué significa	Cómo verificarlo	Qué hacer si falla
Linealidad	La relación es lineal	Gráfico de residuos	Transformar variables (log, polinomios)
Homocedasticidad	Varianza constante	Breusch-Pagan, gráfico	Mínimos cuadrados ponderados
Normalidad	Errores $N(0, \sigma^2)$	Q-Q plot, Shapiro	n grande → TLC, transformaciones
Independencia	Errores no correlacionados	Durbin-Watson	Modelos de series temporales
No multicolinealidad	Predictores no correlacionados	VIF	Eliminar/regularizar variables

Regresión Lineal Múltiple

```
# Más predictores
np.random.seed(42)
n = 200

horas_estudio = np.random.uniform(0, 10, n)
horas_sueno = np.random.uniform(4, 10, n)
ejercicio = np.random.uniform(0, 7, n)

# Puntuación real: 20 + 5*estudio + 3*sueño + 2*ejercicio + ruido
puntuacion = (20 + 5 * horas_estudio +
               3 * horas_sueno +
               2 * ejercicio +
               np.random.normal(0, 5, n))

# DataFrame para statsmodels
import statsmodels.api as sm

df = pd.DataFrame({
    'puntuacion': puntuacion,
    'horas_estudio': horas_estudio,
    'horas_sueno': horas_sueno,
    'ejercicio': ejercicio
})

# Regresión múltiple
X = df[['horas_estudio', 'horas_sueno', 'ejercicio']]
X = sm.add_constant(X) # Añadir intercepto
modelo_sm = sm.OLS(df['puntuacion'], X).fit()

print(modelo_sm.summary())
```

Interpretación de Coeficientes

```
print("Coeficientes:")
print(modelo_sm.params.round(3))
# const          20.734
# horas_estudio   4.932 (cada hora de estudio → +4.93 puntos)
# horas_sueno     2.962 (cada hora de sueño → +2.96 puntos)
# ejercicio       2.053 (cada hora de ejercicio → +2.05 puntos)

# Errores estándar e intervalos de confianza
print("\nIC 95%:")
```

```
print(modelo_sm.conf_int().round(3))

# Valor p de cada coeficiente
print("\nValores p:")
print(modelo_sm.pvalues.round(4))
```

Multicolinealidad

Cuando los predictores están correlacionados entre sí, los coeficientes se vuelven inestables:

```
# Introducir multicolinealidad
df['estudio_x2'] = df['horas_estudio'] * 2 + np.random.normal(0, 0.5, n)

X_multi = df[['horas_estudio', 'estudio_x2', 'horas_sueno']]
X_multi = sm.add_constant(X_multi)
modelo_multi = sm.OLS(df['puntuacion'], X_multi).fit()

print("Coeficientes con multicolinealidad:")
print(modelo_multi.params.round(3))
# Los coeficientes de estudio y estudio_x2 serán inestables
# y sus errores estándar se dispararán

# Factor de Inflación de Varianza (VIF)
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif_data = pd.DataFrame()
vif_data['Variable'] = X_multi.columns
vif_data['VIF'] = [variance_inflation_factor(X_multi.values, i)
                  for i in range(X_multi.shape[1])]

print("\nVIF:")
print(vif_data.round(2))
# VIF > 10 indica multicolinealidad severa
# horas_estudio y estudio_x2 tendrán VIF muy alto
```

Comparación de Modelos

R² y R² Ajustado

```
# R²: proporción de varianza explicada
r2 = modelo_sm.rsquared
r2_adj = modelo_sm.rsquared_adj

print(f"R²: {r2:.4f} ({r2*100:.1f}% de varianza explicada)")
print(f"R² ajustado: {r2_adj:.4f}")

# Añadir predictores siempre aumenta R² (incluso si son ruido)
# R² ajustado penaliza la complejidad
```

AIC y BIC

```
print(f"AIC: {modelo_sm.aic:.1f}")
```



```
print(f"BIC: {modelo_sm.bic:.1f}")

# Menor AIC/BIC → mejor modelo (penaliza complejidad)
```

Transformaciones para Relaciones No Lineales

```
# Relación no lineal: rendimientos decrecientes
np.random.seed(42)
horas = np.random.uniform(0, 10, 100)
fatiga = 5 * np.log(horas + 1) + np.random.normal(0, 1, 100)

# Modelo lineal en nivel (mal ajuste)
X_lin = horas.reshape(-1, 1)
modelo_lin = LinearRegression().fit(X_lin, fatiga)
r2_lin = modelo_lin.score(X_lin, fatiga)

# Modelo log-transformado
X_log = np.log(horas + 1).reshape(-1, 1)
modelo_log = LinearRegression().fit(X_log, fatiga)
r2_log = modelo_log.score(X_log, fatiga)

print(f"R² lineal: {r2_lin:.4f}")
print(f"R² log: {r2_log:.4f}")
# El log captura mejor la relación
```

Ejercicios Prácticos

1. **Ejercicio 1:** Usando el dataset `sklearn.datasets.load_diabetes()`, ajusta una regresión lineal múltiple. ¿Qué variable tiene el coeficiente más grande? ¿Cuál es el más significativo?
2. **Ejercicio 2:** Simula un dataset donde la relación entre X e Y sea cuadrática ($Y = X^2 + \epsilon$). Ajusta una regresión lineal simple y una regresión con X^2 como predictor. Compara los R^2 .
3. **Ejercicio 3:** Verifica los supuestos de la regresión en el dataset de diabetes. ¿Se cumplen? Si no, ¿qué transformaciones propondrías?
4. **Ejercicio 4:** Implementa la regresión lineal desde cero usando la ecuación normal (álgebra lineal) y compárala con sklearn. Verifica que obtienes los mismos coeficientes.

Resumen del Capítulo

- La **regresión lineal** modela $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_X X + \epsilon$
- Los **coeficientes** se estiman por mínimos cuadrados ordinarios (OLS)
- El **intercepto** es el valor esperado cuando todos los predictores son 0
- Las **pendientes** miden el cambio en Y por unidad de cambio en X
- Los **supuestos** (linealidad, homocedasticidad, normalidad, independencia) son críticos
- La **multicolinealidad** infla la varianza de los coeficientes (VIF > 10 es problemático)
- El **R² ajustado** y el **AIC/BIC** sirven para comparar modelos

- Las **transformaciones** (log, polinomios) capturan relaciones no lineales

Conceptos clave aprendidos: OLS, coeficiente, intercepto, supuestos, multicolinealidad, VIF, R^2 , R^2 ajustado, AIC, BIC, transformación logarítmica

Evaluación de Modelos: R^2 , MSE, RMSE

Un modelo es tan bueno como su capacidad para predecir. Este capítulo cubre las métricas para evaluar qué tan bien funciona un modelo de regresión, cómo interpretarlas y cómo usarlas para mejorar.

Las Métricas Fundamentales

Error Absoluto Medio (MAE)

```
import numpy as np
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error,
    r2_score, explained_variance_score
)

# Datos simulados
np.random.seed(42)
y_real = np.random.normal(100, 20, 200)
y_pred = y_real + np.random.normal(0, 5, 200)

# MAE: promedio de errores absolutos
mae = mean_absolute_error(y_real, y_pred)
print(f"MAE: {mae:.2f}")
print(f"Interpretación: en promedio, el modelo se equivoca por {mae:.2f} unidades")
# MAE: 3.98
```

Error Cuadrático Medio (MSE)

```
# MSE: promedio de errores al cuadrado
mse = mean_squared_error(y_real, y_pred)
print(f"MSE: {mse:.2f}")
# MSE: 25.31

# Penaliza más los errores grandes que el MAE
# Si un error es 2x, MSE lo penaliza 4x
```

Raíz del Error Cuadrático Medio (RMSE)

```
# RMSE: raíz cuadrada del MSE (misma unidad que Y)
rmse = np.sqrt(mse)
print(f"RMSE: {rmse:.2f}")
print(f"Interpretación: error típico del modelo en las mismas unidades que Y")
```

```
# RMSE: 5.03
```

R² (Coeficiente de Determinación)

```
# R²: proporción de varianza explicada
r2 = r2_score(y_real, y_pred)
print(f"R²: {r2:.4f} ({r2*100:.1f}%)")
# R²: 0.9360 → el modelo explica el 93.6% de la varianza
```

Comparación de Métricas

Métrica	Rango	Interpretación	Unidad
MAE	[0, ∞)	Error promedio absoluto	Misma que Y
MSE	[0, ∞)	Error cuadrático promedio	Y²
RMSE	[0, ∞)	Error típico	Misma que Y
R²	(-∞, 1]	Proporción de varianza explicada	Adimensional

```
def reporte_regresion(y_real, y_pred):
    """Reporte completo de métricas de regresión"""
    mae = mean_absolute_error(y_real, y_pred)
    mse = mean_squared_error(y_real, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_real, y_pred)
    mape = np.mean(np.abs((y_real - y_pred) / y_real)) * 100

    print("=== Métricas de Regresión ===")
    print(f"MAE: {mae:.3f}")
    print(f"MSE: {mse:.3f}")
    print(f"RMSE: {rmse:.3f}")
    print(f"R²: {r2:.4f}")
    print(f"MAPE: {mape:.2f}%")
    return {'MAE': mae, 'MSE': mse, 'RMSE': rmse, 'R²': r2, 'MAPE': mape}

# Ejemplo completo
np.random.seed(42)
X = np.random.randn(100, 3)
y = 2 + 1.5*X[:, 0] - 0.5*X[:, 1] + 1.0*X[:, 2] + np.random.randn(100)*0.5

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
modelo = LinearRegression().fit(X_train, y_train)
y_pred = modelo.predict(X_test)

reporte_regresion(y_test, y_pred)
```

R² Explicado en Detalle

```
# R² = 1 - SS_res / SS_tot
```

```

ss_res = np.sum((y_real - y_pred)**2)          # Varianza no explicada
ss_tot = np.sum((y_real - np.mean(y_real))**2) # Varianza total

r2_manual = 1 - ss_res / ss_tot
print(f"R² manual: {r2_manual:.4f}")
print(f"R² sklearn: {r2_score(y_real, y_pred):.4f}")
# Idénticos

# Interpretación de R²
print("\nInterpretación:")
print(f"  SS_tot = {ss_tot:.1f} (varianza total de Y)")
print(f"  SS_res = {ss_res:.1f} (varianza NO explicada)")
print(f"  SS_reg = {ss_tot - ss_res:.1f} (varianza explicada por el modelo)")

```

Sobreajuste (Overfitting)

El sobreajuste ocurre cuando el modelo aprende el ruido en lugar de la señal:

```

# Demostración de sobreajuste
np.random.seed(42)
n_train = 20
n_test = 100

X_train = np.random.rand(n_train, 1) * 10
y_train = np.sin(X_train).ravel() + np.random.randn(n_train) * 0.3

X_test = np.random.rand(n_test, 1) * 10
y_test = np.sin(X_test).ravel() + np.random.randn(n_test) * 0.3

from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline

for grado in [1, 3, 15]:
    pipe = Pipeline([
        ('poly', PolynomialFeatures(degree=grado, include_bias=False)),
        ('lin', LinearRegression())
    ])
    pipe.fit(X_train, y_train)

    r2_train = pipe.score(X_train, y_train)
    r2_test = pipe.score(X_test, y_test)

    print(f"Grado {grado:2d}: R²_train = {r2_train:.4f}, R²_test = {r2_test:.4f}")
# Grado  1: R²_train = 0.5432, R²_test = 0.5021 (subajuste)
# Grado  3: R²_train = 0.8476, R²_test = 0.8204 (buen ajuste)
# Grado 15: R²_train = 0.9989, R²_test = 0.5101 (SOBREAJUSTE)

```

> Advertencia:

> Un R^2 muy alto en entrenamiento (cercano a 1) con un R^2 bajo en prueba es la señal clásica de sobreajuste. Siempre evalúa en datos no vistos.

Validación Cruzada (Cross-Validation)

```

from sklearn.model_selection import cross_val_score, KFold

# K-Fold Cross Validation (k=5)
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(LinearRegression(), X, y, cv=kf, scoring='r2')

print(f"R² en cada fold: {np.round(scores, 4)}")
print(f"R² promedio (CV): {np.mean(scores):.4f} ± {np.std(scores):.4f}")
# R² en cada fold: [0.83 0.85 0.86 0.81 0.84]
# R² promedio (CV): 0.838 ± 0.018

# Comparación de modelos con CV
from sklearn.linear_model import Ridge, Lasso

modelos = {
    'Linear': LinearRegression(),
    'Ridge': Ridge(alpha=1.0),
    'Lasso': Lasso(alpha=0.1)
}

print("\nComparación con CV (5-fold):")
for nombre, modelo in modelos.items():
    scores = cross_val_score(modelo, X, y, cv=5, scoring='r2')
    print(f" {nombre}: R² = {np.mean(scores):.4f} ± {np.std(scores):.4f}")

```

Análisis de Residuales

Los residuales ($y_{\text{real}} - y_{\text{pred}}$) contienen información valiosa:

```

import matplotlib.pyplot as plt
from scipy import stats

def analisis_residuales(y_real, y_pred):
    """Análisis completo de residuales"""
    residuos = y_real - y_pred

    # 1. Media de residuales (debe ser ~0)
    print(f"Media de residuales: {np.mean(residuos):.4f} (ideal: 0)")

    # 2. Distribución
    _, p_norm = stats.shapiro(residuos)
    print(f"Normalidad residuos (Shapiro): p = {p_norm:.4f}")

    # 3. Homocedasticidad
    r, p_hetero = stats.spearmanr(np.abs(residuos), y_pred)
    print(f"Heterocedasticidad: ρ = {r:.4f}, p = {p_hetero:.4f}")

    # 4. Outliers (residuos estandarizados > 3)
    residuos_std = residuos / np.std(residuos)
    n_outliers = np.sum(np.abs(residuos_std) > 3)
    print(f"Outliers (|residuo| > 3σ): {n_outliers} ({n_outliers/len(residuos)*100:.1f}%)")

```

```

return residuos

analisis_residuales(y_test, y_pred)

```

MAPE: Error Porcentual Absoluto Medio

```

def mape(y_real, y_pred):
    """Error porcentual absoluto medio"""
    return np.mean(np.abs((y_real - y_pred) / y_real)) * 100

# MAPE es útil cuando el error relativo importa más que el absoluto
y_real_ej = np.array([100, 200, 150, 300])
y_pred_ej = np.array([110, 190, 160, 280])

print(f"MAPE: {mape(y_real_ej, y_pred_ej):.2f}%")
# MAPE: 6.39% → error relativo promedio del 6.4%

```

Guía de Selección de Métricas

Si te importa...	Usa...	Por qué
Error en unidades originales	RMSE o MAE	Misma escala que Y
Penalizar errores grandes	RMSE o MSE	Eleva al cuadrado los errores
Interpretación intuitiva	MAE	Promedio simple de errores
Proporción explicada	R^2	[0, 1], fácil de comunicar
Error relativo	MAPE	Cuando el tamaño importa
Comparar modelos con distinta escala	R^2 o RMSE normalizado	Estandarizado

Ejercicios Prácticos

- Ejercicio 1:** Para el dataset `diabetes` de sklearn, ajusta una regresión lineal y calcula MAE, MSE, RMSE y R^2 en entrenamiento y prueba. ¿Hay evidencia de sobreajuste?
- Ejercicio 2:** Crea un polinomio de grado 10 para datos generados con una relación cuadrática simple. Muestra cómo el R^2 de entrenamiento aumenta monótonamente mientras el R^2 de prueba empeora después de cierto punto.
- Ejercicio 3:** Implementa tu propia validación cruzada de 5 folds sin usar sklearn. Compara los resultados con `cross\_val\_score`.
- Ejercicio 4:** Dado un conjunto de predicciones, identifica qué puntos son outliers analizando los residuales estandarizados. ¿Qué harías con esos puntos?

Resumen del Capítulo

- **MAE:** error promedio absoluto (robusto a outliers)
- **MSE:** error cuadrático promedio (penaliza errores grandes)
- **RMSE:** raíz del MSE, en unidades originales

- **R²**: proporción de varianza explicada (rango típico [0, 1])
- **MAPE**: error porcentual promedio
- El **sobreajuste** ocurre cuando $R^2_{\text{train}} \gg R^2_{\text{test}}$
- La **validación cruzada** estima el rendimiento real del modelo
- Los **residuales** deben tener media 0, ser normales y homocedásticos
- Ninguna métrica es perfecta; usa varias complementarias

Conceptos clave aprendidos: MAE, MSE, RMSE, R², MAPE, sobreajuste, validación cruzada, K-Fold, análisis de residuales

Regresión Logística para Clasificación

A pesar de su nombre, la regresión logística es un algoritmo de clasificación, no de regresión. Es el punto de partida para problemas de clasificación binaria: ¿un email es spam o no? ¿un cliente comprará o no? ¿un paciente tiene una enfermedad?

De Regresión Lineal a Logística

La regresión lineal predice valores continuos. La logística predice probabilidades. La clave está en la *función sigmoide* (o logística):

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Esta función transforma cualquier valor real en una probabilidad entre 0 y 1.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, roc_auc_score, roc_curve
)

def sigmoide(z):
    """Función sigmoide: mapea cualquier real a [0, 1]"""
    return 1 / (1 + np.exp(-z))

# La sigmoide en acción
z = np.linspace(-10, 10, 100)
p = sigmoide(z)

print(f"z = -10 → p = {sigmoide(-10):.6f} (prácticamente 0)")
print(f"z = 0 → p = {sigmoide(0):.4f} (exactamente 0.5)")
print(f"z = 10 → p = {sigmoide(10):.6f} (prácticamente 1)")
# z = -10 → p = 0.000045
# z = 0 → p = 0.5000
# z = 10 → p = 0.999955
```

Cómo Funciona la Regresión Logística

El modelo calcula una combinación lineal de los predictores (como en regresión lineal) y luego aplica la sigmoide:

$$P(Y=1 \mid X) = \sigma(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n) = 1 / (1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)})$$

```
# Simular datos de clasificación
np.random.seed(42)
n = 200

# Dos variables predictoras
X = np.random.randn(n, 2)

# Probabilidad real (función logística)
z = -1 + 2 * X[:, 0] - 1.5 * X[:, 1]
p_real = sigmoide(z)
y = np.random.binomial(1, p_real)

print(f"Proporción de clase 1: {np.mean(y):.2%}")
```

Entrenamiento con Scikit-learn

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

modelo = LogisticRegression()
modelo.fit(X_train, y_train)

# Coeficientes
print(f"Intercepto ( $\beta_0$ ): {modelo.intercept_[0]:.3f}")
print(f"Coeficientes ( $\beta_1, \beta_2$ ): {modelo.coef_[0].round(3)}")
# Intercepto ( $\beta_0$ ): -1.124
# Coeficientes ( $\beta_1, \beta_2$ ): [2.031 -1.573]
```

Interpretación de Coeficientes (Odds Ratio)

A diferencia de la regresión lineal, los coeficientes no se interpretan directamente como cambios en Y. Se interpretan en términos de *odds ratios*:

```
# Odds = p / (1 - p)
# Log(Odds) =  $\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$ 
# Un cambio de 1 unidad en  $X_1$  multiplica los odds por  $e^{\beta_1}$ 

coef = modelo.coef_[0]
print("Interpretación (Odds Ratios):")
for i, c in enumerate(coef):
    or_valor = np.exp(c)
    print(f"  X{i+1}: OR = {or_valor:.3f}")
    print(f"    Por cada unidad más en X{i+1}, los odds se multiplican por {or_valor:.3f}")
```

Probabilidades vs Clases

```
# Probabilidades predichas
probs = modelo.predict_proba(X_test)
print("Probabilidades (primeras 5):")
print(f" P(Y=0): {probs[:5, 0].round(3)}")
print(f" P(Y=1): {probs[:5, 1].round(3)}")

# Clases predichas (umbral por defecto: 0.5)
predicciones = modelo.predict(X_test)
print(f"\nClases predichas: {predicciones[:10]}")
print(f"Clases reales: {y_test[:10]}")
```

Métricas de Clasificación

Accuracy es engañosa cuando las clases están desbalanceadas:

```
def reporte_clasificacion(y_real, y_pred, y_prob=None):
    """Reporte completo de clasificación binaria"""
    tn, fp, fn, tp = confusion_matrix(y_real, y_pred).ravel()

    accuracy = accuracy_score(y_real, y_pred)
    precision = precision_score(y_real, y_pred)
    recall = recall_score(y_real, y_pred)
    f1 = f1_score(y_real, y_pred)

    print("=== Métricas de Clasificación ===")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f} (de los que predije 1, qué proporción es 1)")
    print(f"Recall: {recall:.4f} (de los que son 1, qué proporción detecté)")
    print(f"F1-Score: {f1:.4f} (media armónica de precisión y recall)")
    print(f"\nMatriz de confusión:")
    print(f" [[{tn:3d} {fp:3d}]]")
    print(f" [[{fn:3d} {tp:3d}]]")

    if y_prob is not None:
        auc = roc_auc_score(y_real, y_prob[:, 1])
        print(f"\nAUC-ROC: {auc:.4f}")

    return {
        'accuracy': accuracy, 'precision': precision,
        'recall': recall, 'f1': f1
    }

y_prob = modelo.predict_proba(X_test)
reporte_clasificacion(y_test, predicciones, y_prob)
```

La Matriz de Confusión Explicada

	Predicho 0	Predicho 1
Real 0	TN (Verdadero Negativo)	FP (Falso Positivo)
Real 1	FN (Falso Negativo)	TP (Verdadero Positivo)

```

tn, fp, fn, tp = confusion_matrix(y_test, predicciones).ravel()

# Tasas derivadas
tpr = tp / (tp + fn) # Recall / Sensibilidad
tnr = tn / (tn + fp) # Especificidad
fpr = fp / (fp + tn) # Tasa de falsos positivos

print(f"Sensibilidad (TPR): {tpr:.3f} - capacidad de detectar positivos")
print(f"Especificidad (TNR): {tnr:.3f} - capacidad de identificar negativos")
print(f"Tasa Falsos Pos (FPR): {fpr:.3f}")

```

Curva ROC y AUC

La curva ROC muestra el trade-off entre sensibilidad y falsos positivos para diferentes umbrales:

```

# Calcular curva ROC
fpr, tpr, umbrales = roc_curve(y_test, y_prob[:, 1])
auc = roc_auc_score(y_test, y_prob[:, 1])

print(f"AUC: {auc:.4f}")
print()
print(f"Umbral\tFPR\tTPR (Sensibilidad)")
for i in range(0, len(umbrales), len(umbrales)//5):
    print(f"{umbrales[i]:.2f}\t{fpr[i]:.3f}\t{tpr[i]:.3f}")

# AUC interpretación
# 0.50 = aleatorio
# 0.70-0.80 = aceptable
# 0.80-0.90 = excelente
# 0.90-1.00 = sobresaliente

```

Balanceo de Clases

Cuando una clase es mucho más frecuente que la otra, las métricas como accuracy son engañosas:

```

# Dataset desbalanceado
np.random.seed(42)
n = 1000
X_desb = np.random.randn(n, 2)
y_desb = np.random.binomial(1, 0.05, n) # Solo 5% son clase 1

print(f"Proporción clase 1: {np.mean(y_desb):.2%}")

# Modelo ingenuo (predecir siempre 0)
y_ingenuo = np.zeros(n)
print(f"Accuracy del modelo ingenuo: {accuracy_score(y_desb, y_ingenuo):.4f}")
# 95%! Pero no detecta ningún positivo

# Estrategias para balancear:
# - class_weight='balanced' en sklearn
# - Sobremuestreo (SMOTE)

```

```
# - Submuestreo
# - Ajustar umbral de decisión

modelo_balanceado = LogisticRegression(class_weight='balanced')
modelo_balanceado.fit(X_train, y_train)

# Comparar umbrales
print("\nEfecto de ajustar umbral:")
for umbral in [0.3, 0.5, 0.7]:
    pred_umbral = (y_prob[:, 1] >= umbral).astype(int)
    prec = precision_score(y_test, pred_umbral)
    rec = recall_score(y_test, pred_umbral)
    print(f" Umbral={umbral:.1f}: Precision={prec:.3f}, Recall={rec:.3f}")
```

Regularización: Ridge y Lasso para Logística

```
# Lasso (L1): selecciona variables (pone coeficientes a 0)
lasso = LogisticRegression(penalty='l1', solver='liblinear', C=1.0)
lasso.fit(X_train, y_train)
print(f"Coeficientes Lasso: {lasso.coef_[0].round(3)}")

# Ridge (L2): encoge coeficientes (nunca a 0)
ridge = LogisticRegression(penalty='l2', C=1.0)
ridge.fit(X_train, y_train)
print(f"Coeficientes Ridge: {ridge.coef_[0].round(3)}")

# C controla la fuerza de regularización (menor C = más regularización)
for C in [0.01, 0.1, 1, 10]:
    modelo_reg = LogisticRegression(C=C, penalty='l2')
    modelo_reg.fit(X_train, y_train)
    score = modelo_reg.score(X_test, y_test)
    print(f"C={C:5.2f}: Accuracy = {score:.4f}, ||β||² = {np.sum(modelo_reg.coef_**2):.4f}")
```

Ejercicios Prácticos

- Ejercicio 1:** Usando el dataset `load\_breast\_cancer` de sklearn, entrena un modelo de regresión logística. Reporta accuracy, precisión, recall, F1 y AUC. ¿Cuál es la métrica más importante para diagnóstico médico?
- Ejercicio 2:** Genera un dataset desbalanceado (95% clase 0, 5% clase 1). Compara un modelo con `class\_weight=None` vs `class\_weight='balanced'`. ¿Cómo cambian precisión y recall?
- Ejercicio 3:** Implementa la función sigmoide y la función de costo (log-loss) desde cero. Verifica que tu implementación de sklearn produce los mismos coeficientes.
- Ejercicio 4:** Simula datos donde la frontera de decisión no sea lineal. Compara la regresión logística lineal con una versión con términos polinomiales. ¿Cuál funciona mejor?

Resumen del Capítulo

- La **regresión logística** modela la probabilidad de pertenencia a una clase

- La **función sigmoide** transforma combinaciones lineales en probabilidades $[0, 1]$
- Los **odds ratios** (e^{β}) interpretan el cambio en odds por unidad de predictor
- **Accuracy** es engañosa con clases desbalanceadas
- **Precision, Recall, F1** son mejores métricas para clasificación
- La **curva ROC** y el **AUC** evalúan el rendimiento en todos los umbrales
- La **matriz de confusión** desglosa aciertos y errores por clase
- El **balanceo de clases** es crítico; usa `class_weight` o remuestreo
- La **regularización** (L1/L2) previene sobreajuste

Conceptos clave aprendidos: sigmoide, odds ratio, log-loss, accuracy, precisión, recall, F1, matriz de confusión, ROC, AUC, balanceo de clases, regularización

Módulo 6: Análisis Multivariante y Series Temporales

Reducción de Dimensionalidad: PCA

Explicado

Cuando tu dataset tiene muchas variables, surgen problemas: maldición de la dimensionalidad, multicolinealidad, sobreajuste y dificultad de visualización. El Análisis de Componentes Principales (PCA) es la técnica más popular para reducir dimensionalidad preservando la mayor cantidad de información posible.

¿Qué es PCA?

PCA encuentra nuevas direcciones (componentes principales) en los datos que capturan la máxima varianza. El primer componente captura la mayor varianza posible; el segundo, la siguiente mayor varianza ortogonal al primero; y así sucesivamente.

La Geometría de PCA

PCA rota el sistema de coordenadas para alinearlo con las direcciones de máxima varianza de los datos.

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from scipy import stats

np.random.seed(42)

# Simular datos correlacionados (2D, pero la información real es 1D)
n = 200
X = np.random.randn(n, 2)
# Rotar y estirar: los datos forman un "elipse" alargada
angulo = np.pi / 4
rotacion = np.array([[np.cos(angulo), -np.sin(angulo)],
                     [np.sin(angulo), np.cos(angulo)]])
X = X @ rotacion
X[:, 0] *= 3 # Estirar en una dirección

# PCA
X_scaled = StandardScaler().fit_transform(X)
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

print("Varianza explicada por componente:")
```

```

for i, var in enumerate(pca.explained_variance_ratio_):
    print(f" PC{i+1}: {var:.4f} ({var*100:.2f}%)" )
# PC1: 0.9567 (95.67%)
# PC2: 0.0433 (4.33%)

print(f"\nVarianza explicada acumulada:")
print(f" PC1: {pca.explained_variance_ratio_[0]:.4f}")
print(f" PC1+PC2: {np.sum(pca.explained_variance_ratio_):.4f}")

```

El Algoritmo PCA Paso a Paso

PCA se implementa mediante la descomposición SVD de la matriz de datos centrada:

```

def pca_desde_cero(X, n_componentes=2):
    """PCA implementado desde cero usando SVD"""
    # 1. Centrar los datos
    X_centrado = X - np.mean(X, axis=0)

    # 2. SVD
    U, S, Vt = np.linalg.svd(X_centrado, full_matrices=False)

    # 3. Componentes principales (vectores en Vt)
    componentes = Vt[:n_componentes]

    # 4. Datos transformados
    X_proyectado = X_centrado @ componentes.T

    # 5. Varianza explicada
    varianza_total = np.sum(S**2)
    varianza_explicada = (S[:n_componentes]**2) / varianza_total

    return X_proyectado, componentes, varianza_explicada

# Verificar contra sklearn
X_proy_manual, comps_manual, var_exp_manual = pca_desde_cero(X_scaled, 2)
print("Varianza explicada (implementación manual):")
for i, v in enumerate(var_exp_manual):
    print(f" PC{i+1}: {v:.4f}")
# Coincide con sklearn

```

Cómo Elegir el Número de Componentes

```

# Dataset de mayor dimensión
from sklearn.datasets import load_digits
digits = load_digits()
X_digits = digits.data
print(f"Shape original: {X_digits.shape}") # (1797, 64)

# PCA completo
pca_full = PCA().fit(StandardScaler().fit_transform(X_digits))

```



```
# Varianza acumulada
var_acum = np.cumsum(pca_full.explained_variance_ratio_)

print("\nComponentes necesarios para:")
for umbral in [0.70, 0.80, 0.90, 0.95, 0.99]:
    n = np.argmax(var_acum >= umbral) + 1
    print(f" {umbral:.0%} de varianza: {n} componentes (de 64)")
# 70% de varianza: ~8 componentes
# 80% de varianza: ~12 componentes
# 90% de varianza: ~21 componentes
# 95% de varianza: ~29 componentes
# 99% de varianza: ~45 componentes
```

> Consejo del Analista:

> No hay una regla fija para elegir k componentes. El punto de codo en el gráfico de varianza explicada y el umbral del 90-95% son buenos puntos de partida. La elección depende de tu objetivo: ¿visualización (k=2-3), compresión (k que retenga 95%), o preprocesamiento?

PCA para Visualización

```
# Reducir dígitos a 2D para visualización
pca_2d = PCA(n_components=2)
X_digits_2d = pca_2d.fit_transform(StandardScaler().fit_transform(X_digits))

print("Dígitos en 2D:")
print(f" Shape reducido: {X_digits_2d.shape}")
print(f" Varianza explicada: {np.sum(pca_2d.explained_variance_ratio_):.2%}")
# Con 2 componentes capturamos ~28% de la varianza (suficiente para visualizar)
```

PCA como Preprocesamiento para Modelos

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

# Sin PCA
rf = RandomForestClassifier(n_estimators=100, random_state=42)
score_original = cross_val_score(rf, X_digits, digits.target, cv=5).mean()

# Con PCA (retener 95% de varianza)
n_comps_95 = np.argmax(var_acum >= 0.95) + 1
pca_95 = PCA(n_components=n_comps_95)
X_pca_95 = pca_95.fit_transform(StandardScaler().fit_transform(X_digits))

score_pca = cross_val_score(rf, X_pca_95, digits.target, cv=5).mean()

print(f"\nRendimiento del clasificador:")
print(f" Sin PCA (64 vars): accuracy = {score_original:.4f}")
print(f" Con PCA ({n_comps_95} vars): accuracy = {score_pca:.4f}")
# El rendimiento es similar pero con 29 variables en vez de 64
```

Carga de Componentes (Loadings)

Las cargas (loadings) indican cuánto contribuye cada variable original a cada componente:

```
# Loadings para el dataset de dígitos
loadings = pca_full.components_ # Shape: (n_componentes, n_variables)

print("Interpretación de PC1 (loadings):")
# Las variables con mayor |loading| contribuyen más a PC1
pc1_loadings = loadings[0]
top_vars = np.argsort(np.abs(pc1_loadings))[-5:]
print(f" Las 5 variables más importantes para PC1: {top_vars}")
```

El Efecto de Estandarizar

```
# Sin estandarizar: las variables con mayor escala dominan
X_sin_escalado = np.column_stack([
    np.random.randn(100) * 100, # Escala grande
    np.random.randn(100) * 1,   # Escala pequeña
])
pca_no_scale = PCA().fit(X_sin_escalado)
print(f"Sin estandarizar - varianza explicada PC1: {pca_no_scale.explained_variance_ratio_[0]:.4f}")
# PC1 captura casi toda la varianza, dominada por la variable de gran escala

# Con estandarización
X_escalado = StandardScaler().fit_transform(X_sin_escalado)
pca_scaled = PCA().fit(X_escalado)
print(f"Estandarizado - varianza explicada PC1: {pca_scaled.explained_variance_ratio_[0]:.4f}")
# Las variables contribuyen equitativamente
```

> Advertencia:

> Siempre estandariza (media=0, var=1) antes de PCA a menos que todas las variables estén en la misma unidad y escala. PCA es sensible a la escala de las variables.

PCA y la Matriz de Covarianza

PCA está íntimamente relacionado con la descomposición en eigenvectores de la matriz de covarianza:

```
# Matriz de covarianza
X_centrado = X_scaled - np.mean(X_scaled, axis=0)
cov = (X_centrado.T @ X_centrado) / (X_centrado.shape[0] - 1)

# Eigenvectores = direcciones de máxima varianza
eigenvals, eigenvects = np.linalg.eigh(cov)
# Ordenar por eigenvalor descendente
idx = np.argsort(eigenvals)[::-1]
eigenvals = eigenvals[idx]
eigenvects = eigenvects[:, idx]

print("Eigenvalores (varianza en cada dirección):")
print(f" {eigenvals.round(4)}")
```

```
# Proporción de varianza explicada
print(f" Varianza explicada PC1: {eigenvals[0]/np.sum(eigenvals):.4f}")
```

Ejercicios Prácticos

1. **Ejercicio 1:** Aplica PCA al dataset Iris. ¿Cuánta varianza explica cada componente? ¿Qué especies se separan mejor en las primeras 2 componentes?
2. **Ejercicio 2:** Implementa PCA desde cero usando la descomposición en eigenvectores de la matriz de covarianza (en lugar de SVD). Compara los resultados con sklearn.
3. **Ejercicio 3:** Usa PCA como preprocesamiento para regresión logística en el dataset de dígitos. Varía el número de componentes de 1 a 64. ¿En qué punto el rendimiento se estabiliza?
4. **Ejercicio 4:** Genera un dataset con 20 variables aleatorias, donde solo 3 son informativas y las demás son ruido. Demuestra que PCA puede identificar las dimensiones relevantes.

Resumen del Capítulo

- **PCA** encuentra direcciones ortogonales de máxima varianza en los datos
- Reduce dimensionalidad proyectando los datos en las primeras k componentes
 - La **varianza explicada** indica cuánta información retiene cada componente
 - El **número óptimo de componentes** se elige por punto de codo o umbral de varianza
 - **SVD** es el algoritmo subyacente para implementar PCA numéricamente
 - Los **loadings** indican la contribución de cada variable original a los componentes
 - **Siempre estandariza** antes de PCA
- PCA mejora la eficiencia computacional y reduce el sobreajuste

Conceptos clave aprendidos: componente principal, varianza explicada, SVD, loadings, scree plot, estandarización, reducción de dimensionalidad, eigenvalores

Análisis de Conglomerados (K-Means)

El clustering (agrupamiento) es una técnica de aprendizaje no supervisado que encuentra grupos naturales en los datos sin etiquetas previas. K-Means es el algoritmo de clustering más conocido y utilizado.

¿Qué es K-Means?

K-Means particiona n observaciones en k grupos, donde cada observación pertenece al grupo con la media (centroide) más cercana.

El Algoritmo

1. Seleccionar k centroides iniciales (aleatorios)
2. Asignar cada punto al centroide más cercano
3. Recalcular centroides como la media de los puntos asignados
4. Repetir pasos 2-3 hasta convergencia

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs
from scipy.spatial.distance import cdist

np.random.seed(42)

# Generar datos agrupados
X, y_real = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=1.5,
    random_state=42
)

print(f"Shape: {X.shape}")
print(f"Centros reales: 4 grupos")
```

Implementación desde Cero

```
def kmeans_desde_cero(X, k, max_iter=100, tol=1e-4):
    """K-Means implementado desde cero"""
    n, m = X.shape

    # 1. Inicializar centroides (elegir k puntos aleatorios)
```

```

idx = np.random.choice(n, k, replace=False)
centroides = X[idx]

for iteracion in range(max_iter):
    # 2. Asignar cada punto al centroide más cercano
    distancias = cdist(X, centroides, metric='euclidean')
    etiquetas = np.argmin(distancias, axis=1)

    # 3. Recalcular centroides
    nuevos_centroides = np.array([
        X[etiquetas == i].mean(axis=0) for i in range(k)
    ])

    # 4. Verificar convergencia
    cambio = np.linalg.norm(nuevos_centroides - centroides)
    centroides = nuevos_centroides

    if cambio < tol:
        print(f"Convergió en {iteracion + 1} iteraciones")
        break

return etiquetas, centroides

etiquetas_manual, centroides_manual = kmeans_desde_cero(X, k=4)

```

Con Scikit-learn

```

kmeans = KMeans(n_clusters=4, random_state=42, n_init='auto')
kmeans.fit(X)

etiquetas = kmeans.labels_
centroides = kmeans.cluster_centers_

print(f"Inercia (suma de distancias al cuadrado): {kmeans.inertia_:.2f}")
print(f"Número de iteraciones: {kmeans.n_iter_}")

# Comparar con implementación manual
print(f"¿Coinciden las etiquetas? {np.array_equal(etiquetas, etiquetas_manual)}")
# Pueden diferir por la inicialización aleatoria

```

Cómo Elegir k: El Método del Codo

No hay una respuesta única para k. El método del codo busca el punto donde agregar más clusters deja de dar mejoras significativas:

```

def metodo_codo(X, k_max=10):
    """Calcula inercia para diferentes valores de k"""
    inercias = []
    for k in range(1, k_max + 1):
        kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
        kmeans.fit(X)
        inercias.append(kmeans.inertia_)

```

```

# Tasa de mejora
mejoras = []
for i in range(1, len(inercias)):
    mejora = (inercias[i-1] - inercias[i]) / inercias[i-1]
    mejoras.append(mejora)

print("k\tInercia\t\tMejora")
for i, (inercia, mejora) in enumerate(zip(inercias, [0] + mejoras)):
    print(f"{i+1}\t{inercia:.2f}\t{mejora:.4f}" if i > 0 else f"{i+1}\t{inercia:.2f}\t-")

metodo_codo(X, k_max=8)
# k=1: 2500.00 -
# k=2: 1200.00 0.5200
# k=3: 700.00 0.4167
# k=4: 350.00 0.5000 ← codo
# k=5: 320.00 0.0857 ← mejora mucho menor
# k=6: 300.00 0.0625

```

Coeficiente de Silueta

Una métrica formal para evaluar la calidad del clustering:

```

for k in range(2, 7):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
    etiquetas_k = kmeans.fit_predict(X)
    silueta = silhouette_score(X, etiquetas_k)
    print(f"k={k}: Silueta = {silueta:.4f}")
# k=2: Silueta = 0.5812
# k=3: Silueta = 0.6421
# k=4: Silueta = 0.6810 ← mejor
# k=5: Silueta = 0.5304
# k=6: Silueta = 0.4893

```

> Consejo del Analista:

> El coeficiente de silueta varía de -1 a 1. >0.5 indica buena separación entre clusters. Combínalo con el método del codo para elegir k.

Limitaciones de K-Means

1. Sensible a la Inicialización

```

# Diferentes inicializaciones pueden dar diferentes resultados
resultados = []
for semilla in range(10):
    kmeans = KMeans(n_clusters=4, random_state=semilla, n_init=1)
    kmeans.fit(X)
    resultados.append(kmeans.inertia_)

print(f"Inercias para 10 inicializaciones:")
print(f"  Min: {min(resultados):.2f}")
print(f"  Max: {max(resultados):.2f}")

```

```
print(f" Diferencia: {max(resultados) - min(resultados):.2f}")
```

2. Asume Clusters Esféricos

```
# K-Means falla con formas no esféricas
from sklearn.datasets import make_moons

X_moons, _ = make_moons(n_samples=300, noise=0.05, random_state=42)
kmeans_moons = KMeans(n_clusters=2, random_state=42, n_init='auto')
etiquetas_moons = kmeans_moons.fit_predict(X_moons)
silueta_moons = silhouette_score(X_moons, etiquetas_moons)

print(f"Silueta para datos de luna (K-Means): {silueta_moons:.4f}")
# K-Means no captura bien formas no convexas
```

3. Sensible a Outliers

```
# Añadir un outlier
X_con_outlier = np.vstack([X, [20, 20]])
kmeans_outlier = KMeans(n_clusters=4, random_state=42, n_init='auto')
etiquetas_outlier = kmeans_outlier.fit_predict(X_con_outlier)

# El outlier distorsiona los centroides
print(f"Centroides sin outlier:\n{np.round(centroides, 2)}")
print(f"Centroides con outlier:\n{np.round(kmeans_outlier.cluster_centers_, 2)}")
```

Alternativas a K-Means

Algoritmo	Ventajas	Desventajas
K-Means	Rápido, escalable	Clusters esféricos, sensible a inicialización
DBSCAN	Detecta outliers, formas arbitrarias	Sensible a parámetros ϵ y minPts
Hierarchical (Agglomerative)	Dendrograma, no requiere k	Lento para grandes datos
Gaussian Mixture	Clusters con forma de elipse, probabilíst	Más parámetros

```
from sklearn.cluster import DBSCAN

# DBSCAN para formas no convexas
dbscan = DBSCAN(eps=0.2, min_samples=5)
etiquetas_dbscan = dbscan.fit_predict(X_moons)
n_clusters_dbscan = len(set(etiquetas_dbscan)) - (1 if -1 in etiquetas_dbscan else 0)

print(f"DBSCAN en datos de luna:")
print(f" Clusters encontrados: {n_clusters_dbscan}")
print(f" Puntos de ruido: {np.sum(etiquetas_dbscan == -1)}")
```

Aplicación: Segmentación de Clientes

```
# Simular datos de clientes
np.random.seed(42)
```

```

n_clientes = 500

clientes = np.column_stack([
    np.random.normal(5, 2, n_clientes),      # Frecuencia de compra
    np.random.normal(100, 40, n_clientes),   # Gasto promedio
    np.random.exponential(3, n_clientes),     # Antigüedad (años)
])

from sklearn.preprocessing import StandardScaler
clientes_scaled = StandardScaler().fit_transform(clientes)

# Encontrar segmentos
kmeans_clientes = KMeans(n_clusters=3, random_state=42, n_init='auto')
segmentos = kmeans_clientes.fit_predict(clientes_scaled)

# Caracterizar segmentos (en escala original)
for i in range(3):
    mascara = segmentos == i
    print(f"\nSegmento {i+1} (n={mascara.sum()}):")
    print(f" Frecuencia: {np.mean(clientes[mascara, 0]):.1f}")
    print(f" Gasto: ${np.mean(clientes[mascara, 1]):.1f}")
    print(f" Antigüedad: {np.mean(clientes[mascara, 2]):.1f} años")

```

Ejercicios Prácticos

- Ejercicio 1:** Genera datos con 3 clusters de diferentes densidades. Aplica K-Means con k=3. ¿El algoritmo encuentra los clusters correctamente? ¿Qué pasa si las densidades son muy diferentes?
- Ejercicio 2:** Usando el método del codo y el coeficiente de silueta, determina el k óptimo para el dataset Iris (solo las características, sin las etiquetas). ¿Coincide con el número de especies?
- Ejercicio 3:** Compara K-Means con DBSCAN en el dataset `make\_circles` de sklearn (círculos concéntricos). ¿Cuál algoritmo captura mejor la estructura?
- Ejercicio 4:** Implementa una función que calcule el centroide más cercano para un nuevo punto después de entrenar K-Means. Úsala para clasificar 5 nuevos clientes en los segmentos existentes.

Resumen del Capítulo

- **K-Means** agrupa datos en k clusters basados en distancia al centroide
- El **método del codo** y el **coeficiente de silueta** guían la elección de k
- K-Means asume clusters **esféricos** y de **tamaño similar**
- Es **sensible a la inicialización** (usa k-means++ o n_init)
- Los **outliers** distorsionan los centroides
- **DBSCAN** y **clustering jerárquico** son alternativas para formas complejas
- El **escalado** de variables es crítico (la distancia se ve afectada por la escala)

- La **segmentación de clientes** es una aplicación clásica

Conceptos clave aprendidos: clustering, centroide, inercia, método del codo, silueta, K-Means++, DBSCAN, segmentación, aprendizaje no supervisado

Introducción a Series Temporales

Una serie temporal es una secuencia de observaciones ordenadas en el tiempo. Ventas diarias, precios de acciones, tráfico web, temperatura: todos son ejemplos de series temporales. Su análisis requiere técnicas específicas porque las observaciones NO son independientes.

Conceptos Fundamentales

Componentes de una Serie Temporal

Toda serie temporal puede descomponerse en:

1. **Tendencia (T):** dirección a largo plazo
2. **Estacionalidad (S):** patrones periódicos predecibles
3. **Ciclo (C):** fluctuaciones sin período fijo
4. **Residuo (R):** ruido aleatorio

```
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller, acf, pacf
from sklearn.metrics import mean_absolute_error

np.random.seed(42)

# Crear una serie temporal sintética
t = np.arange(0, 200)

# Tendencia lineal
tendencia = 10 + 0.05 * t

# Estacionalidad anual (período 20)
estacionalidad = 3 * np.sin(2 * np.pi * t / 20)

# Ruido
ruido = np.random.normal(0, 0.5, len(t))

serie = tendencia + estacionalidad + ruido

print(f"Serie temporal: {len(serie)} puntos")
print(f"Media: {np.mean(serie):.2f}")
```

```
print(f"Desv. estándar: {np.std(serie):.2f}")
```

Descomposición de Series Temporales

```
# Convertir a pandas Series con índice temporal
idx = pd.date_range('2024-01-01', periods=len(serie), freq='D')
serie_ts = pd.Series(serie, index=idx)

# Descomposición aditiva
descomposicion = seasonal_decompose(serie_ts, model='additive', period=20)

# Extraer componentes
tendencia_est = descomposicion.trend
estacionalidad_est = descomposicion.seasonal
residuo_est = descomposicion.resid

print("Descomposición completada:")
print(f"  Tendencia: {tendencia_est.dropna().shape[0]} puntos")
print(f"  Estacionalidad: {estacionalidad_est.dropna().shape[0]} puntos")
print(f"  Residuo: {residuo_est.dropna().shape[0]} puntos")
```

Estacionariedad

Una serie es *estacionaria* si sus propiedades estadísticas (media, varianza, autocorrelación) no cambian con el tiempo. La mayoría de los modelos de series temporales requieren estacionariedad.

Test de Dickey-Fuller Aumentado

```
def test_estacionariedad(serie, nombre='Serie'):
    """Test ADF para estacionariedad"""
    resultado = adfuller(serie.dropna())
    adf_stat, p_valor = resultado[0], resultado[1]

    print(f"Test ADF - {nombre}:")
    print(f"  Estadístico: {adf_stat:.4f}")
    print(f"  Valor p: {p_valor:.4f}")

    if p_valor < 0.05:
        print(f"  Conclusión: ESTACIONARIA (p < 0.05)")
        return True
    else:
        print(f"  Conclusión: NO ESTACIONARIA (p >= 0.05)")
        return False

test_estacionariedad(serie, 'Original')
# La serie con tendencia no es estacionaria

# Diferenciación: restar el valor anterior
serie_diff = np.diff(serie)
test_estacionariedad(serie_diff, 'Diferenciada (orden 1)')
# La serie diferenciada es estacionaria
```

Autocorrelación

Mide la correlación de una serie consigo misma en diferentes rezagos (lags):

```
# Función de Autocorrelación (ACF)
acf_vals, acf_ci = acf(serie_diff, nlags=20, alpha=0.05)

print("ACF (primeros 10 rezagos):")
for i in range(10):
    print(f" Lag {i}: {acf_vals[i]:.4f}")
# Lag 0: 1.000 (siempre)
# Lag 1: -0.502 (correlación negativa con el valor anterior)
# Lags siguientes: decrecen

# Función de Autocorrelación Parcial (PACF)
pacf_vals, pacf_ci = pacf(serie_diff, nlags=20, alpha=0.05)

print("\nPACF (primeros 5 rezagos):")
for i in range(5):
    print(f" Lag {i}: {pacf_vals[i]:.4f}")
```

Modelos ARIMA

ARIMA (AutoRegressive Integrated Moving Average) es el modelo clásico para series temporales:

- **AR (p)**: usa valores pasados como predictores
- **I (d)**: diferenciación para hacer la serie estacionaria
- **MA (q)**: usa errores pasados como predictores

```
from statsmodels.tsa.arima.model import ARIMA

# Dividir en entrenamiento y prueba
train_size = 160
train, test = serie[:train_size], serie[train_size:]

# Modelo ARIMA(1,1,1) - p=1, d=1, q=1
modelo_arima = ARIMA(train, order=(1, 1, 1))
modelo_arima_fit = modelo_arima.fit()

print(modelo_arima_fit.summary().tables[1])
```

#	coef	std err	z	P> z
# ar.L1	0.5234	0.083	6.307	0.000
# ma.L1	-0.9152	0.047	-19.562	0.000
# sigma2	0.2041	0.022	9.375	0.000

Predicción

```
# Predicción en test
predicciones = modelo_arima_fit.forecast(steps=len(test))

# Error
```

```

error = mean_absolute_error(test, predicciones)
print(f"MAE en test: {error:.3f}")

# Últimos valores reales vs predichos
print("\nÚltimas 5 predicciones:")
for i in range(-5, 0):
    print(f"  Real: {test[i]:.2f}, Pred: {predicciones[i]:.2f}, Diff: {test[i]-predicciones[i]:.2f}")

```

Cómo Elegir p, d, q

```

# 1. d: número de diferencias para estacionariedad
# Ya determinamos d=1

# 2. p: orden AR → mirar PACF (pico significativo en lag p)
# 3. q: orden MA → mirar ACF (pico significativo en lag q)

# Búsqueda automática de ARIMA
def mejor_arima(serie, max_p=5, max_q=5, d=1):
    mejor_aic = np.inf
    mejor_orden = None
    mejor_modelo = None

    for p in range(max_p + 1):
        for q in range(max_q + 1):
            try:
                modelo = ARIMA(serie, order=(p, d, q)).fit()
                if modelo.aic < mejor_aic:
                    mejor_aic = modelo.aic
                    mejor_orden = (p, d, q)
                    mejor_modelo = modelo
            except:
                continue

    return mejor_orden, mejor_modelo

mejor_orden, mejor_modelo = mejor_arima(train, max_p=3, max_q=3)
print(f"Mejor ARIMA (por AIC): {mejor_orden}")
# Por ejemplo: (1, 1, 1)

```

Validación de Modelos de Series Temporales

La validación cruzada estándar no funciona porque el orden temporal importa. Usamos *walk-forward validation*:

```

def walk_forward_validation(serie, model_order, n_test=30):
    """Validación walk-forward para series temporales"""
    n = len(serie)
    predicciones = []

    for i in range(n - n_test, n):
        # Entrenar con datos hasta i-1

```

```

train = serie[:i]
modelo = ARIMA(train, order=model_order).fit()
# Predecir el siguiente punto
pred = modelo.forecast(steps=1)
predicciones.append(pred[0])

reales = serie[-n_test:]
mae = mean_absolute_error(reales, predicciones)
return mae, np.array(predicciones)

mae_wf, preds_wf = walk_forward_validation(serie, mejor_orden, n_test=30)
print(f"Walk-forward MAE: {mae_wf:.3f}")

```

Más Allá de ARIMA

```

# Prophet (desarrollado por Meta)
# Maneja estacionalidades múltiples y días festivos
# from prophet import Prophet

# SARIMA (estacional)
# ARIMA + componente estacional: SARIMA(p,d,q)(P,D,Q,s)
from statsmodels.tsa.statespace.sarimax import SARIMAX

# SARIMA(1,1,1)(1,1,1,20) - estacionalidad de período 20
modelo_sarima = SARIMAX(train, order=(1, 1, 1), seasonal_order=(1, 1, 1, 20))
modelo_sarima_fit = modelo_sarima.fit(dispatch=False)
preds_sarima = modelo_sarima_fit.forecast(steps=len(test))
mae_sarima = mean_absolute_error(test, preds_sarima)

print(f"MAE ARIMA: {error:.3f}")
print(f"MAE SARIMA: {mae_sarima:.3f}")
# SARIMA suele ser mejor si hay estacionalidad fuerte

```

Ejercicios Prácticos

- Ejercicio 1:** Carga un dataset de series temporales (por ejemplo, `airline passengers` de statsmodels o cualquier serie de `yfinance`). Descompón la serie en tendencia, estacionalidad y residuo.
- Ejercicio 2:** Para la serie del ejercicio anterior, aplica el test ADF antes y después de diferenciar. ¿Cuántas diferencias necesita para ser estacionaria?
- Ejercicio 3:** Ajusta un modelo ARIMA a la serie de pasajeros aéreos. Usa walk-forward validation para evaluar su rendimiento. ¿Qué orden (p,d,q) funciona mejor?
- Ejercicio 4:** Compara las predicciones de tu modelo ARIMA con un modelo naive (el valor del período anterior). ¿Cuánto mejora el ARIMA en términos de MAE?

Resumen del Capítulo

- Las series temporales tienen **dependencia temporal** (no son i.i.d.)

- Se descomponen en **tendencia, estacionalidad, ciclo y residuo**
- La **estacionariedad** es requisito para la mayoría de modelos
- El **test ADF** determina si una serie es estacionaria
- La **diferenciación** elimina tendencias y hace la serie estacionaria
- **ACF** y **PACF** revelan dependencias entre rezagos
- **ARIMA(p,d,q)** combina componentes autorregresivos, integrados y de media móvil
- **SARIMA** añade un componente estacional
- La **walk-forward validation** respeta el orden temporal
- Siempre compara contra un modelo **naive** (benchmark)

Conceptos clave aprendidos: tendencia, estacionalidad, estacionariedad, ADF, ACF, PACF, ARIMA, SARIMA, diferenciación, walk-forward validation

Módulo 7: Casos de Uso Reales

Flujo de Trabajo de un Proyecto Cuantitativo

Este capítulo unifica todo lo aprendido en un flujo de trabajo completo y reproducible. Seguirás un caso real de principio a fin: desde la pregunta de negocio hasta la presentación de resultados.

El Pipeline Completo

```
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
```

Fase 1: Definición del Problema

> **Pregunta de negocio:** ¿Podemos predecir qué clientes dejarán nuestro servicio (churn) en los próximos 30 días, basándonos en su comportamiento histórico?

```
# Criterios de éxito: AUC > 0.75 en datos de prueba
# Restricciones: el modelo debe ser interpretable (regresión logística)
# Acción: enviar un descuento personalizado a clientes con >60% de riesgo
```

Fase 2: Obtención y Exploración de Datos

```
np.random.seed(42)
n = 5000

# Simular datos de clientes
df = pd.DataFrame({
    'cliente_id': range(1, n + 1),
    'antiguedad_meses': np.random.exponential(24, n).clip(1, 120).astype(int),
    'num_llamadas_soporte': np.random.poisson(2, n),
    'monto_mensual': np.random.lognormal(4.5, 0.5, n),
    'contrato_mensual': np.random.binomial(1, 0.4, n), # 1 = mensual, 0 = anual
    'pago_electronico': np.random.binomial(1, 0.6, n),
    'quejas_30_dias': np.random.poisson(0.5, n),
```



```

        'uso_servicio': np.random.uniform(0, 100, n), # % de uso del servicio contratado
    })

# Generar churn basado en un modelo subyacente
z = (-2
      - 0.02 * df['antiguedad_meses']
      + 0.3 * df['num_llamadas_soporte']
      - 0.01 * df['monto_mensual']
      + 0.8 * df['contrato_mensual']
      - 0.5 * df['pago_electronico']
      + 0.7 * df['quejas_30_dias']
      - 0.03 * df['uso_servicio']
      + np.random.normal(0, 1, n))

p_churn = 1 / (1 + np.exp(-z))
df['churn'] = np.random.binomial(1, p_churn)

print(f"Tasa de churn: {df['churn'].mean():.2%}")
print(f"\nPrimeras filas:")
print(df.head())

```

Fase 3: Análisis Exploratorio (EDA)

```

def exploratory_analysis(df, target='churn'):
    """Análisis exploratorio completo"""
    print("=== 1. Estructura del Dataset ===")
    print(f" Filas: {df.shape[0]}, Columnas: {df.shape[1]}")
    print(f" Valores faltantes: {df.isna().sum().sum()}")
    print(f" Tipos de datos:\n{df.dtypes.value_counts()}")

    print("\n=== 2. Estadísticas Descriptivas ===")
    print(df.describe().round(2))

    print("\n=== 3. Análisis Univariante (por variable vs target) ===")
    for col in df.select_dtypes(include=[np.number]).columns:
        if col != target and col != 'cliente_id':
            churn_1 = df[df[target] == 1][col]
            churn_0 = df[df[target] == 0][col]
            stat, p = stats.ttest_ind(churn_0, churn_1, equal_var=False)
            print(f" {col:25s}: media(churn=0)={churn_0.mean():.2f}, media(churn=1)={churn_1.mean():.2f},")

    exploratory_analysis(df)

```

Fase 4: Preprocesamiento

```

# Separar predictores y target
features = ['antiguedad_meses', 'num_llamadas_soporte', 'monto_mensual',
            'contrato_mensual', 'pago_electronico', 'quejas_30_dias', 'uso_servicio']
X = df[features]
y = df['churn']

# Dividir en entrenamiento y prueba

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print(f"Train: {X_train.shape[0]} muestras ({y_train.mean():.2%} churn)")
print(f"Test: {X_test.shape[0]} muestras ({y_test.mean():.2%} churn)")

# Estandarizar variables numéricas
num_features = ['antigüedad_meses', 'num_llamadas_soporte',
                'monto_mensual', 'quejas_30_dias', 'uso_servicio']
scaler = StandardScaler()
X_train[num_features] = scaler.fit_transform(X_train[num_features])
X_test[num_features] = scaler.transform(X_test[num_features])

```

Fase 5: Modelado

```

# Modelo base: regresión logística
modelo = LogisticRegression(class_weight='balanced', random_state=42)
modelo.fit(X_train, y_train)

# Evaluación
y_pred = modelo.predict(X_test)
y_prob = modelo.predict_proba(X_test)[:, 1]

print("=== Evaluación del Modelo ===")
print(classification_report(y_test, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_test, y_prob):.4f}")

# Interpretación de coeficientes
coef_df = pd.DataFrame({
    'feature': features,
    'coef': modelo.coef_[0],
    'odds_ratio': np.exp(modelo.coef_[0])
}).sort_values('odds_ratio', ascending=False)

print("\n=== Importancia de Variables (Odds Ratios) ===")
print(coef_df.round(3))

```

Fase 6: Validación Cruzada

```

scores = cross_val_score(modelo, X_train, y_train, cv=5, scoring='roc_auc')
print(f"Validación Cruzada (5 folds):")
print(f"  AUC promedio: {scores.mean():.4f} ± {scores.std():.4f}")

```

Fase 7: Optimización de Umbral

```

# Encontrar el umbral óptimo para maximizar F1
from sklearn.metrics import f1_score

umbrales = np.linspace(0.1, 0.9, 50)
f1_scores = []

```

```

for umbral in umbrales:
    pred_umbral = (y_prob >= umbral).astype(int)
    f1_scores.append(f1_score(y_test, pred_umbral))

umbral_optimo = umbrales[np.argmax(f1_scores)]
print(f"Umbral óptimo (máximo F1): {umbral_optimo:.2f}")
print(f"F1 en umbral óptimo: {max(f1_scores):.4f}")

# Decisión de negocio: enviar descuento si riesgo > 60%
umbral_negocio = 0.60
clientes_riesgo = X_test[y_prob >= umbral_negocio]
print(f"\nClientes con >60% de riesgo: {len(clientes_riesgo)} ({len(clientes_riesgo)/len(X_test):.1%})")

```

Fase 8: Documentación y Comunicación

```

def resumen_ejecutivo(modelo, metricas, umbral_negocio, df_test, y_prob):
    """Genera un resumen ejecutivo del proyecto"""
    print("=" * 60)
    print("RESUMEN EJECUTIVO: Modelo de Predicción de Churn")
    print("=" * 60)
    print(f"\nObjetivo: Identificar clientes con alto riesgo de abandono")

    print(f"\nRendimiento del modelo:")
    print(f"  AUC-ROC: {metricas['auc']:.3f}")
    print(f"  Precision: {metricas['precision']:.3f}")
    print(f"  Recall: {metricas['recall']:.3f}")

    print(f"\nImpacto en negocio:")
    n_clientes_riesgo = int((y_prob >= umbral_negocio).sum())
    print(f"  Clientes identificados como alto riesgo: {n_clientes_riesgo}")
    print(f"  Tasa de retención esperada (estimación): +15-25%")

    print(f"\nVariables más influyentes:")
    for _, row in coef_df.head(3).iterrows():
        print(f"  {row['feature']}: OR = {row['odds_ratio']:.2f}")

    print(f"\nRecomendación:")
    print(f"  Implementar campaña de retención dirigida a los {n_clientes_riesgo} clientes identificados.")
    print(f"  Costo estimado: ${n_clientes_riesgo * 5:.0f} (descuento de $5 por cliente)")
    print(f"  ROI estimado: 3.5x (basado en LTV promedio de $500)")

    metricas = {
        'auc': roc_auc_score(y_test, y_prob),
        'precision': __import__('sklearn').metrics.precision_score(y_test, y_pred),
        'recall': __import__('sklearn').metrics.recall_score(y_test, y_pred)
    }

    resumen_ejecutivo(modelo, metricas, 0.60, X_test, y_prob)

```

Checklist de un Proyecto Cuantitativo

Fase	Actividades clave	Entregable
1. Definición	Pregunta, criterios de éxito, restricciones	Documento de alcance
2. Datos	Fuentes, calidad, integración	Dataset limpio
3. EDA	Estadísticas, visualizaciones, hipótesis	Reporte exploratorio
4. Preprocesamiento	Limpieza, transformaciones, división	Datos listos para modelar
5. Modelado	Selección, entrenamiento, tuning	Modelo entrenado
6. Validación	CV, métricas, interpretación	Evaluación del modelo
7. Despliegue	Implementación, monitoreo	API/Informe
8. Comunicación	Visualización, storytelling, acción	Resumen ejecutivo

Ejercicios Prácticos

- Ejercicio 1:** Aplica este flujo de trabajo completo a un dataset de tu elección (puede ser de Kaggle, UCI o sklearn). Documenta cada fase.
- Ejercicio 2:** Modifica el flujo para incluir un segundo modelo (Random Forest) y compáralo con la regresión logística. ¿Cuál prefieres? ¿Por qué?
- Ejercicio 3:** Crea un dashboard en matplotlib/seaborn que muestre las métricas clave del proyecto: distribución del target, matriz de correlación, importancia de variables y curva ROC.
- Ejercicio 4:** Escribe un resumen ejecutivo de una página (en markdown) explicando los resultados a un director de marketing. No uses jerga técnica.

Resumen del Capítulo

- Un proyecto cuantitativo sigue un **flujo estructurado** de 8 fases
- La **definición del problema** guía todas las decisiones posteriores
- El **EDA** revela patrones, anomalías y relaciones
- El **preprocesamiento** incluye escalado, división y manejo de faltantes
- El **modelado** combina selección, entrenamiento y validación
- La **optimización de umbral** alinea la estadística con el negocio
- La **comunicación** debe traducir hallazgos técnicos en acciones de negocio
- Documenta cada paso para garantizar **reproducibilidad**

Conceptos clave aprendidos: pipeline, EDA, preprocesamiento, modelado, validación, umbral de decisión, ROI, resumen ejecutivo, reproducibilidad

Manejo de Datos Faltantes y Outliers

Los datos reales nunca son perfectos. Valores faltantes y outliers son la norma, no la excepción. Ignorarlos o manejarlos incorrectamente sesga tus análisis y modelos. Este capítulo te da las herramientas para tratarlos con rigor.

Datos Faltantes: Tipos y Mecanismos

Antes de imputar, debes entender por qué faltan los datos:

Mecanismo	Significado	Ejemplo	Tratamiento
MCAR	Falta completamente al azar	Sensor se apagó aleatoriamente	Eliminar o imputar sin sesgo
MAR	Falta al azar (depende de otras var	Mujeres no reportan edad	Imputación condicional
MNAR	No falta al azar (depende de valores faltantes)	Personas con ingresos altos no reportan ingresos	Modelos especializados

```
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

np.random.seed(42)
n = 500

# Dataset completo
df_completo = pd.DataFrame({
    'edad': np.random.normal(40, 12, n).astype(int).clip(18, 90),
    'ingresos': np.random.lognormal(10, 0.5, n),
    'educacion': np.random.choice([0, 1, 2, 3], n, p=[0.1, 0.3, 0.4, 0.2]),
    'puntuacion': np.random.normal(75, 15, n).clip(0, 100),
})

# Introducir valores faltantes (MCAR)
mask_mcar = np.random.random(df_completo.shape) < 0.1
df_missing = df_completo.copy()
df_missing[mask_mcar] = np.nan

print(f"Dataset original: {df_completo.shape}")
print(f"Valores faltantes:\n{df_missing.isna().sum()}")
```

Estrategia 1: Eliminación

```
# Eliminar filas con cualquier valor faltante (listwise deletion)
df_sin_faltantes = df_missing.dropna()
print(f"Después de eliminar filas con NaN: {df_sin_faltantes.shape}")
print(f" Perdimos {len(df_missing) - len(df_sin_faltantes)} filas ({100*(1-len(df_sin_faltantes)/len(df_m

# Eliminar columnas con muchos faltantes
threshold = 0.3 # 30% faltantes
cols_a_eliminar = df_missing.columns[df_missing.isna().mean() > threshold]
print(f" Columnas a eliminar (>30% faltantes): {list(cols_a_eliminar)}")
```

> **Advertencia:**

> Eliminar filas con faltantes solo es aceptable si los datos son MCAR y la pérdida es pequeña (<5%). Si pierdes >5% de tus datos, la imputación es casi siempre mejor.

Estrategia 2: Imputación Simple

```
def comparar_imputaciones(df_original, df_missing, columna):
    """Compara diferentes métodos de imputación"""
    valores_reales = df_original[columna]
    mascara = df_missing[columna].isna()

    # Media
    imputer_mean = SimpleImputer(strategy='mean')
    df_mean = imputer_mean.fit_transform(df_missing[[columna]])
    error_mean = np.abs(valores_reales[mascara] - df_mean[mascara, 0]).mean()

    # Mediana (robusta a outliers)
    imputer_median = SimpleImputer(strategy='median')
    df_median = imputer_median.fit_transform(df_missing[[columna]])
    error_median = np.abs(valores_reales[mascara] - df_median[mascara, 0]).mean()

    print(f"\nComparación de imputaciones para '{columna}':")
    print(f" MAE - Media: {error_mean:.2f}")
    print(f" MAE - Mediana: {error_median:.2f}")

comparar_imputaciones(df_completo, df_missing, 'edad')
comparar_imputaciones(df_completo, df_missing, 'ingresos')
```

Estrategia 3: Imputación por KNN

```
# Imputación basada en vecinos más cercanos
imputer_knn = KNNImputer(n_neighbors=5)
df_knn = pd.DataFrame(
    imputer_knn.fit_transform(df_missing),
    columns=df_missing.columns
)

# Error para ingresos (variable con outliers)
mascara = df_missing['ingresos'].isna()
error_knn_ingresos = np.abs(
    df_completo['ingresos'][mascara] - df_knn['ingresos'][mascara]
).mean()
```

```
print(f"MAE KNN para ingresos: {error_knn_ingresos:.2f}")
```

Estrategia 4: Imputación Múltiple (MICE)

```
# MICE (Multiple Imputation by Chained Equations)
imputer_mice = IterativeImputer(max_iter=10, random_state=42)
df_mice = pd.DataFrame(
    imputer_mice.fit_transform(df_missing),
    columns=df_missing.columns
)

error_mice_ingresos = np.abs(
    df_completo['ingresos'][mascara] - df_mice['ingresos'][mascara]
).mean()
print(f"MAE MICE para ingresos: {error_mice_ingresos:.2f}")
```

Outliers: Detección y Tratamiento

Los valores atípicos (outliers) pueden ser errores, anomalías reales o información valiosa. El contexto determina el tratamiento.

Método 1: Rango Intercuartil (IQR)

```
def detectar_outliers_iqr(datos, factor=1.5):
    """Detecta outliers usando la regla del IQR"""
    Q1 = np.percentile(datos, 25)
    Q3 = np.percentile(datos, 75)
    IQR = Q3 - Q1

    limite_inf = Q1 - factor * IQR
    limite_sup = Q3 + factor * IQR

    outliers = (datos < limite_inf) | (datos > limite_sup)
    return outliers, limite_inf, limite_sup

# Ejemplo con ingresos (distribución log-normal)
ingresos = np.random.lognormal(10, 0.5, 1000)
outliers_iqr, lim_inf, lim_sup = detectar_outliers_iqr(ingresos)

print(f"Outliers detectados (IQR): {outliers_iqr.sum()} ({outliers_iqr.mean()*100:.1f}%)")
print(f" Límite inferior: {lim_inf:.0f}")
print(f" Límite superior: {lim_sup:.0f}")
```

Método 2: Desviación Estándar (Z-Score)

```
def detectar_outliers_zscore(datos, umbral=3):
    """Detecta outliers usando Z-score"""
    z = np.abs(stats.zscore(datos))
    return z > umbral

outliers_z = detectar_outliers_zscore(ingresos)
```

```
print(f"Outliers detectados (Z-score, 3σ): {outliers_z.sum()} ({outliers_z.mean()*100:.1f}%")
```

Método 3: Isolation Forest

```
from sklearn.ensemble import IsolationForest

# Para datos multivariados
np.random.seed(42)
X_normal = np.random.randn(500, 2)
X_outliers = np.random.uniform(low=-6, high=6, size=(20, 2))
X = np.vstack([X_normal, X_outliers])

iso_forest = IsolationForest(contamination=0.05, random_state=42)
predicciones = iso_forest.fit_predict(X)
# -1 = outlier, 1 = inlier

outliers_if = predicciones == -1
print(f"Outliers detectados (Isolation Forest): {outliers_if.sum()} ({outliers_if.mean()*100:.1f}%")
```

Tratamiento de Outliers

```
def tratar_outliers(datos, metodo='clip', limite_inf=None, limite_sup=None):
    """Aplica diferentes estrategias de tratamiento de outliers"""
    if metodo == 'clip':
        # Winsorización: reemplazar por los límites
        q1, q99 = np.percentile(datos, [1, 99])
        return np.clip(datos, q1, q99)

    elif metodo == 'remove':
        # Eliminar (no siempre recomendado)
        Q1 = np.percentile(datos, 25)
        Q3 = np.percentile(datos, 75)
        IQR = Q3 - Q1
        mascara = (datos >= Q1 - 1.5*IQR) & (datos <= Q3 + 1.5*IQR)
        return datos[mascara]

    elif metodo == 'log':
        # Transformación logarítmica (comprime colas largas)
        return np.log1p(datos)

    elif metodo == 'missing':
        # Convertir outliers en NaN
        Q1 = np.percentile(datos, 25)
        Q3 = np.percentile(datos, 75)
        IQR = Q3 - Q1
        mascara = (datos >= Q1 - 1.5*IQR) & (datos <= Q3 + 1.5*IQR)
        datos_tratados = datos.copy()
        datos_tratados[~mascara] = np.nan
        return datos_tratados

# Comparar métodos
```



```

ingresos_con_outliers = np.random.lognormal(10, 0.5, 1000)
print(f"Original: media={np.mean(ingresos_con_outliers):.0f}, std={np.std(ingresos_con_outliers):.0f}")
print(f"Clipped: media={np.mean(tratar_outliers(ingresos_con_outliers, 'clip')):.0f}")
print(f"Log: media={np.mean(tratar_outliers(ingresos_con_outliers, 'log')):.0f}")

```

Impacto en Modelos

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

# Demostrar cómo los outliers afectan a la regresión
np.random.seed(42)
X = np.random.randn(100, 1) * 10
y = 3 + 2 * X.ravel() + np.random.randn(100) * 2

# Añadir outliers
X_out = np.vstack([X, [[50], [55]]]) # Dos outliers lejanos
y_out = np.concatenate([y, [100, 120]])

# Modelo sin outliers
modelo_limpio = LinearRegression().fit(X, y)
r2_limpio = modelo_limpio.score(X, y)

# Modelo con outliers
modelo_sucio = LinearRegression().fit(X_out, y_out)
r2_sucio = modelo_sucio.score(X_out, y_out)

print(f"R² sin outliers: {r2_limpio:.4f}")
print(f"R² con outliers: {r2_sucio:.4f}")
print(f"Coef sin outliers: {modelo_limpio.coef_[0]:.2f} (real: 2.0)")
print(f"Coef con outliers: {modelo_sucio.coef_[0]:.2f} (real: 2.0)")

```

Ejercicios Prácticos

1. **Ejercicio 1:** Toma un dataset real (por ejemplo, `load\_diabetes` de sklearn) e introduce artificialmente valores faltantes al azar (10%, 20%, 30%). Compara el rendimiento de un modelo con y sin imputación.
2. **Ejercicio 2:** Genera un dataset con outliers en una variable. Compara el efecto de winsorización (clip), eliminación y transformación logarítmica en la media y varianza.
3. **Ejercicio 3:** Implementa una función que automáticamente decida la estrategia de imputación basada en el porcentaje de valores faltantes y el tipo de variable.
4. **Ejercicio 4:** Usando Isolation Forest, detecta outliers multivariados en el dataset Iris. ¿Coinciden con alguna especie en particular? ¿Tiene sentido biológico?

Resumen del Capítulo

- Los datos faltantes tienen **mecanismos** (MCAR, MAR, MNAR) que determinan el tratamiento

- La **eliminación** solo es apropiada para MCAR con baja pérdida
- La **imputación por media/mediana** es simple pero ignora correlaciones
- **KNN** y **MICE** capturan relaciones entre variables
- Los **outliers** se detectan por IQR, Z-score o Isolation Forest
- El **tratamiento** incluye winsorización, transformación, eliminación o conversión a NaN
- Los outliers afectan desproporcionadamente a modelos basados en distancia y varianza
- El contexto del dominio determina si un outlier es error o señal valiosa

Conceptos clave aprendidos: MCAR/MAR/MNAR, imputación, MICE, KNN imputer, IQR, Z-score, Isolation Forest, winsorización, transformación logarítmica

Cómo Comunicar Hallazgos Matemáticos a Stakeholders

El análisis cuantitativo más brillante es inútil si no puedes comunicar sus resultados a quienes toman decisiones. Este capítulo te enseña a traducir conceptos técnicos en narrativa accionable.

El Problema de Comunicar Matemáticas

Los stakeholders no técnicos necesitan:

- **Respuestas claras**, no procesos
- **Decisiones**, no metodologías
- **Confianza** en los resultados, no fórmulas

```
# Mal: comunicación técnica
"""

Aplicamos una regresión logística con regularización L2
y validación cruzada estratificada de 5 folds.
El AUC fue de 0.87 con un intervalo de confianza del 95%
de [0.84, 0.90]. El odds ratio de la variable X fue de 2.3.
"""

# Bien: comunicación orientada a negocio
"""

Podemos identificar al 70% de los clientes que se van a dar de baja
con un 85% de precisión. Las señales más tempranas son:
llamadas a soporte y quejas recientes. Con esta info, podemos
ofrecer descuentos proactivos a los clientes en riesgo.
"""
```

La Pirámide de la Comunicación Cuantitativa

/	Decisión	\	← ¿Qué hacemos?
/	Recomendación	\	← ¿Qué sugiero?
/	Conclusiones	\	← ¿Qué significa?
/	Hallazgos	\	← ¿Qué encontré?
/	Datos	\	← ¿Qué tengo?

Traduciendo Conceptos Técnicos

Término técnico	Traducción para negocio
"Valor $p < 0.05$ "	"Hay evidencia suficiente para afirmar que el cambio funciona"
"Intervalo de confianza del 95%"	"El verdadero efecto está entre X e Y, con alta confianza"
"AUC = 0.85"	"El modelo acierta 8 de cada 10 veces"
" $R^2 = 0.72$ "	"El 72% del comportamiento se explica por estos factores"
"Error estándar"	"El margen de error de nuestra estimación"
"Correlación no implica causalidad"	"Aunque X e Y se mueven juntos, necesitamos un experimento para probar que X causa Y"
"Sobreaajuste"	"El modelo memorizó los datos de prueba, no aprendió patrones generales"

El Principio de la Audiencia

```
def adaptar_mensaje(audiencia, resultados):
    """Adapta el mensaje según la audiencia"""
    mensajes = {}

    if audiencia == 'directivo':
        mensajes = {
            'titulo': 'Impacto en el negocio',
            'metricas': ['ROI', 'clientes retenidos', 'ingresos adicionales'],
            'detalle_tecnico': 'bajo',
            'accion': 'invertir recursos',
        }
    elif audiencia == 'analista':
        mensajes = {
            'titulo': 'Metodología y resultados',
            'metricas': ['AUC', 'precisión', 'recall', 'valor p'],
            'detalle_tecnico': 'alto',
            'accion': 'validar supuestos y reproducir',
        }
    elif audiencia == 'cliente':
        mensajes = {
            'titulo': 'Beneficios para ti',
            'metricas': ['tiempo ahorrado', 'dinero ahorrado', 'mejora'],
            'detalle_tecnico': 'nulo',
            'accion': 'adoptar el cambio',
        }

    return mensajes
```

Visualización Efectiva

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

# Mala visualización: gráfico 3D innecesario, colores confusos
# (no ejecutar, es ilustrativo)

# Buena visualización: simple, clara, directa
def grafico_comparacion_simple(control, tratamiento, metrica='Tasa de conversión'):
```

```

"""Gráfico simple y profesional para comparar A/B"""
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

# Barras con IC
medias = [np.mean(control), np.mean(tratamiento)]
errores = [np.std(control, ddof=1)/np.sqrt(len(control)),
            np.std(tratamiento, ddof=1)/np.sqrt(len(tratamiento))]

ax[0].bar(['Control', 'Tratamiento'], medias, yerr=errores,
          capsize=10, color=['#6b9bc5', '#e8834a'])
ax[0].set_ylabel(metrica)
ax[0].set_title('Comparación de grupos')
ax[0].grid(axis='y', alpha=0.3)

# Diferencia
diff = media_trat - media_control
ax[1].bar(['Diferencia'], [diff],
          yerr=[np.sqrt(errores[0]**2 + errores[1]**2)],
          capsize=10, color='#4c72b0')
ax[1].axhline(y=0, color='gray', linestyle='--', alpha=0.5)
ax[1].set_ylabel(f'Diferencia en {metrica}')
ax[1].set_title('Efecto del tratamiento')

return fig

```

Reglas de Visualización para Stakeholders

1. **Un gráfico, un mensaje.** No abarrotar
2. **Etiquetas claras.** No asumas que saben leer el eje Y
3. **Colores con propósito.** Azul/verde para positivo, rojo para negativo
4. **Evita 3D.** Distorsiona las proporciones
5. **Incluye intervalos de confianza.** Muestra la incertidumbre
6. **Primero la conclusión.** El título debe ser el hallazgo, no la descripción

```

# Ejemplo de mal título: "Gráfico de barras de ingresos por mes"
# Ejemplo de buen título: "Los ingresos aumentaron un 15% después del rediseño"

```

Storytelling con Datos

La Estructura Narrativa

1. **CONTEXTO:** Situación actual
"Nuestra tasa de retención ha caído al 75% en los últimos 6 meses."
2. **CONFLICTO:** El problema
"No sabíamos qué clientes se irían ni por qué."
3. **ANÁLISIS:** Lo que hicimos
"Analizamos el comportamiento de 10,000 clientes y encontramos 3 señales tempranas."

4. HALLAZGO: Lo que descubrimos

"Los clientes que llaman a soporte 3+ veces tienen un 80% de probabilidad de irse."

5. ACCIÓN: Lo que proponemos

"Si llamamos a esos clientes con una oferta personalizada, podemos retener al 60%."

Template de Presentación Ejecutiva

Proyecto: [Nombre del proyecto]

Resumen (30 segundos)

[Una frase con el hallazgo principal y la acción recomendada]

El Problema

[Contexto: qué motivó el análisis]

Lo que Hicimos

[Muy breve: qué datos, qué método]

Lo que Encontramos

[El hallazgo principal, con 1-2 visualizaciones simples]

Lo que Recomendamos

[Acciones concretas, con impacto esperado]

Apéndice Técnico

[Detalles para quienes los quieran – al final, no interrumpen la historia]

Checklist de Comunicación

Aspecto	Bueno	Malo
Longitud	3 diapositivas clave	50 diapositivas
Jerga	"Mejora del 20%"	"El odds ratio ajustado es 1.23"
Visualización	Una barra con IC	Mapa de calor 3D
Certeza	"La evidencia sugiere"	"Esto prueba que"
Acción	"Recomiendo invertir \$X"	"El valor p fue significativo"
Estructura	Problema → Hallazgo → Acción	Aquí están todos los datos

Ejercicios Prácticos

1. **Ejercicio 1:** Toma un análisis que hayas hecho (o simula uno) y escribe dos versiones del resumen: una para el equipo técnico y otra para el director de la empresa.

2. **Ejercicio 2:** Encuentra un gráfico en una noticia o artículo que sea engañoso o confuso. Rediseñalo para que sea más claro.

3. **Ejercicio 3:** Prepara una presentación de 3 diapositivas (en markdown) explicando un A/B test que muestra un lift del 5% con $p=0.04$. La audiencia es el equipo de producto.

4. **Ejercicio 4:** Traduce los siguientes términos a lenguaje de negocio: "multicolinealidad",

"heterocedasticidad", "sobreajuste", "AUC", "sesgo del estimador".

Resumen del Capítulo

- Adapta el mensaje a la **audiencia**: directivos quieren decisiones, analistas quieren detalles
- Traduce la **jerga técnica** a lenguaje de negocio
- Las visualizaciones deben ser **simples, claras y con un solo mensaje**
- La estructura narrativa: **contexto conflicto análisis hallazgo acción**
- La **incertidumbre** se comunica (IC), no se oculta
- Sé **honesto** sobre las limitaciones del análisis
- Termina siempre con una **recomendación accionable**
- Menos es más: 3 diapositivas clave > 50 diapositivas con todo

Conceptos clave aprendidos: comunicación efectiva, storytelling, visualización de datos, traducción técnica, pirámide de comunicación, audiencia objetivo, resumen ejecutivo

Conclusiones y Próximos Pasos

Has llegado al final de *El Motor Cuantitativo*. Pero en realidad, este es el punto de partida.

Lo que Has Aprendido

Hemos recorrido un camino completo desde los fundamentos matemáticos hasta su aplicación práctica:

Módulo	Lo que dominas ahora
1	El ecosistema NumPy/SciPy/SymPy, álgebra lineal aplicada, cálculo diferencial y optimización
2	Estadística descriptiva completa, probabilidad, teorema de Bayes y distribuciones clave
3	Inferencia estadística: intervalos de confianza, pruebas de hipótesis y tests en Python
4	Diseño experimental, A/B testing riguroso y errores comunes que debes evitar
5	Regresión lineal y logística, evaluación de modelos y métricas
6	PCA, K-Means y series temporales
7	Flujo de trabajo completo, datos faltantes y comunicación efectiva

Principios que Deben Guiarte

1. **Entiende antes de modelar.** No hay sustituto para conocer tus datos y la pregunta que intentas responder.
2. **Sé escéptico.** Cuestiona los resultados, verifica supuestos, busca explicaciones alternativas.
3. **Comunica con honestidad.** Reporta incertidumbre, limitaciones y supuestos. No escondas lo que no sabes.
4. **La simplicidad gana.** Empieza con el modelo más simple que pueda funcionar. Añade complejidad solo cuando esté justificada.
5. **Reproducibilidad sobre intuición.** Si no puedes reproducir un resultado, no es ciencia, es opinión.

Próximos Pasos

Para Profundizar

Tema	Recurso recomendado
Machine Learning avanzado	*The Elements of Statistical Learning* - Hastie, Tibshirani, Friedman
Probabilidad y estadística	*Statistical Inference* - Casella & Berger

Álgebra lineal	*Linear Algebra Done Right* - Axler
Series temporales	*Forecasting: Principles and Practice* - Hyndman & Athanasopoulos
Comunicación de datos	*Storytelling with Data* - Knaflitz

Habilidades Prácticas para Desarrollar

1. **Implementar modelos desde cero.** Intenta implementar regresión logística, PCA o K-Means sin sklearn. La comprensión profunda viene de construir.
2. **Trabajar con datos reales.** Busca datasets en Kaggle, UCI o datos abiertos de tu gobierno. Aplica el flujo completo del Capítulo 26.
3. **Enseñar lo que aprendes.** La mejor forma de saber si entiendes algo es explicárselo a alguien más.
4. **Mantenerte actualizado.** La estadística no cambia rápido, pero las herramientas sí. Sigue aprendiendo nuevas librerías y enfoques.

Una Última Reflexión

Las herramientas matemáticas que has aprendido no son fórmulas que memorizar. Son *lentes* para ver el mundo con más claridad. Cada conjunto de datos cuenta una historia; las matemáticas te dan el lenguaje para entenderla.

El motor cuantitativo ahora está en tus manos. Úsalo con sabiduría, con honestidad y con la humildad de saber que siempre hay más que aprender.

Sigue haciendo preguntas. Sigue construyendo. Sigue analizando.

— Alex Goyzueta Delgado

Apéndice A: Instalación y Configuración del Entorno Python

Instalación de Python

Opción 1: Python Oficial

1. Descarga Python 3.12+ desde python.org
2. Durante la instalación, marca **"Add Python to PATH"**
3. Verifica la instalación:

```
python --version
# Python 3.12.x
```

Opción 2: Anaconda (Recomendada para Análisis de Datos)

1. Descarga Anaconda desde anaconda.com
2. Incluye la mayoría de librerías preinstaladas
3. Usa `conda` para gestionar entornos:

```
conda create --name motor_cuantitativo python=3.12
conda activate motor_cuantitativo
```

Instalación de Librerías

Con pip

```
pip install numpy scipy sympy pandas scikit-learn matplotlib seaborn statsmodels
```

Con conda

```
conda install numpy scipy sympy pandas scikit-learn matplotlib seaborn statsmodels
```

Versiones Mínimas Recomendadas

```
# Verificar versiones instaladas
import numpy as np
import scipy as sp
import sympy as sym
import pandas as pd
```

```

import sklearn as sk
import matplotlib as mpl
import seaborn as sns
import statsmodels as sm

print(f"NumPy:      {np.__version__}")
print(f"SciPy:      {sp.__version__}")
print(f"SymPy:      {sym.__version__}")
print(f"pandas:      {pd.__version__}")
print(f"scikit-learn: {sk.__version__}")
print(f"matplotlib:  {mpl.__version__}")
print(f"seaborn:     {sns.__version__}")
print(f"statsmodels:  {sm.__version__}")

```

Entornos Virtuales

Con venv (Librería estándar)

```

# Crear entorno
python -m venv motor_cuantitativo

# Activar (Windows)
motor_cuantitativo\Scripts\activate

# Activar (Mac/Linux)
source motor_cuantitativo/bin/activate

# Instalar dependencias
pip install -r requirements.txt

```

requirements.txt

Crea este archivo en la raíz de tu proyecto:

```

numpy>=1.24.0
scipy>=1.11.0
sympy>=1.12.0
pandas>=2.0.0
scikit-learn>=1.3.0
matplotlib>=3.7.0
seaborn>=0.12.0
statsmodels>=0.14.0
jupyter>=1.0.0

```

Configuración de Jupyter

Instalación

```

pip install jupyter
# o
conda install jupyter

```

Ejecutar

```
jupyter notebook  
# o  
jupyter lab
```

Extensiones Útiles

```
pip install jupyter_contrib_nbextensions  
jupyter contrib nbextension install --user
```

Verificación Completa

Ejecuta este script para verificar que todo funciona correctamente:

```
# test_setup.py  
import numpy as np  
from scipy import stats  
import sympy as sp  
import pandas as pd  
from sklearn.linear_model import LinearRegression  
import matplotlib.pyplot as plt  
import seaborn as sns  
import statsmodels.api as sm  
  
print(" Todas las librerías se importaron correctamente")  
  
# Prueba NumPy  
a = np.array([1, 2, 3])  
assert a.sum() == 6  
print(" NumPy funciona")  
  
# Prueba SciPy  
z = stats.norm.ppf(0.975)  
assert abs(z - 1.96) < 0.01  
print(" SciPy funciona")  
  
# Prueba SymPy  
x = sp.symbols('x')  
assert sp.diff(x**2, x) == 2*x  
print(" SymPy funciona")  
  
# Prueba pandas  
df = pd.DataFrame({'a': [1, 2, 3]})  
assert df['a'].sum() == 6  
print(" pandas funciona")  
  
# Prueba sklearn  
X = np.array([[1], [2], [3]])  
y = np.array([2, 4, 6])  
model = LinearRegression().fit(X, y)  
assert abs(model.coef_[0] - 2.0) < 0.01
```

```

print(" scikit-learn funciona")

# Prueba matplotlib
fig, ax = plt.subplots()
ax.plot([1, 2, 3])
plt.close()
print(" matplotlib funciona")

print("\n Todo listo para empezar con El Motor Cuantitativo")

```

Solución de Problemas Comunes

Problema	Solución
`pip` no encontrado	Asegúrate de que Python está en el PATH
Error de permisos	Usa `pip install --user` o activa el entorno virtual
Conflicto de versiones	Usa entornos virtuales
`numpy` no encuentra BLAS	Instala desde conda: `conda install numpy`
Gráficos matplotlib no se muestran	Ejecuta `plt.show()` en scripts, o usa `%matplotlib inline` en Jupyter

Apéndice B: Cheat Sheets de Fórmulas Matemáticas

Álgebra Lineal

Vectores

Operación	Fórmula	Python
Norma L2	$\ v\ _2 = \sqrt{\sum_{i=1}^n v_i^2}$	<code>np.linalg.norm(v)</code>
Producto punto	$v \cdot w = \sum_{i=1}^n v_i w_i$	<code>np.dot(v, w)</code>
Ángulo θ	$\cos(\theta) = (v \cdot w) / (\ v\ _2 \ w\ _2)$	<code>np.arccos(dot/(n1*n2))</code>

Matrices

Operación	Fórmula	Python
Transpuesta	$(A^T)_{ij} = A_{ji}$	<code>A.T</code>
Multiplicación	$(AB)_{ij} = \sum_k A_{ik} B_{kj}$	<code>A @ B</code>
Inversa	$A \cdot A^{-1} = I$	<code>np.linalg.inv(A)</code>
Determinante	$\det(A)$	<code>np.linalg.det(A)</code>
Eigenvalores	$Av = \lambda v$	<code>np.linalg.eig(A)</code>
SVD	$A = U \Sigma V^T$	<code>np.linalg.svd(A)</code>

Ecuación Normal (Regresión Lineal)

$$\beta = (X^T X)^{-1} X^T y$$

```
beta = np.linalg.inv(X.T @ X) @ X.T @ y
```

Estadística Descriptiva

Métrica	Fórmula	Python
Media	$\bar{x} = (1/n) \sum x_i$	<code>np.mean(x)</code>
Mediana	Valor central ordenado	<code>np.median(x)</code>
Varianza (pob)	$\sigma^2 = (1/n) \sum (x_i - \mu)^2$	<code>np.var(x)</code>
Varianza (mue)	$s^2 = (1/(n-1)) \sum (x_i - \bar{x})^2$	<code>np.var(x, ddof=1)</code>
Desv. estándar	$s = \sqrt{s^2}$	<code>np.std(x, ddof=1)</code>
Asimetría	$\gamma_1 = (1/n) \sum ((x_i - \bar{x})/s)^3$	<code>scipy.stats.skew(x)</code>

Curtosis	$\gamma_2 = (1/n) \sum ((x_i - \bar{x})/s)^4 - 3$	<code>scipy.stats.kurtosis(x, fisher=True)</code>
----------	---	---

Coeficiente de Correlación de Pearson

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \cdot \sqrt{\sum (y_i - \bar{y})^2}}$$

`r, p = scipy.stats.pearsonr(x, y)`

Probabilidad

Concepto	Fórmula		
Probabilidad condicional	$P(A \setminus B)$	$B) = P(A \cap B) / P(B)$	
Probabilidad total	$P(A) = \sum P(A \setminus B_i) \cdot P(B_i)$		
Teorema de Bayes	$P(A \setminus B) = P(B \setminus A) \cdot P(A) / P(B)$		
Independencia	$P(A \cap B) = P(A) \cdot P(B)$		

Distribuciones de Probabilidad

Distribución	Parámetros	Media	Varianza	Python
Bernoulli	p	p	p(1-p)	<code>stats.bernoulli(p)</code>
Binomial	n, p	np	np(1-p)	<code>stats.binom(n, p)</code>
Poisson	λ	λ	λ	<code>stats.poisson(λ)</code>
Normal	μ, σ	μ	σ^2	<code>stats.norm(μ, σ)</code>
t-Student	df	0 (df>1)	df/(df-2) (df>2)	<code>stats.t(df)</code>
χ^2	df	df	2df	<code>stats.chi2(df)</code>
F	df ₁ , df ₂	df ₂ /(df ₂ -2)	-	<code>stats.f(df₁, df₂)</code>
Exponencial	λ	1/ λ	1/ λ^2	<code>stats.expon(scale=1/λ)</code>

Intervalos de Confianza

Parámetro	IC (1- α)	Python
Media (σ conocida)	$\bar{x} \pm z_{\alpha/2} \cdot \sigma / \sqrt{n}$	<code>stats.norm.ppf(1-$\alpha/2$)</code>
Media (σ desconocida)	$\bar{x} \pm t_{\alpha/2, n-1} \cdot s / \sqrt{n}$	<code>stats.t.ppf(1-$\alpha/2$, df=n-1)</code>
Proporción (Wilson)	$(\hat{p} + z^2/2n \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2}) / (1 + z^2/n)$	-
Diferencia de medias	$(\bar{x}_1 - \bar{x}_2) \pm t_{\alpha/2, v} \cdot \sqrt{(s_1^2/n_1 + s_2^2/n_2)}$	<code>stats.ttest_ind(a, b, equal_var=False)</code>

Pruebas de Hipótesis

Prueba	Estadístico	H ₀ rechaza si	Python		
Z-test 1 muestra	$z = (\bar{x} - \mu_0) / (\sigma / \sqrt{n})$	$ z > z_{\alpha/2}$	<code>z</code>	<code>> z_{\alpha/2}</code>	-
T-test 1 muestra	$t = (\bar{x} - \mu_0) / (s / \sqrt{n})$	$ t > t_{\alpha/2, n-1}$	<code>t</code>	<code>> t_{\alpha/2, n-1}</code>	<code>stats.ttest_1samp(datos, popmean)</code>
T-test 2 muestra (indep.)	$t = (\bar{x}_1 - \bar{x}_2) / \sqrt{(s_1^2/n_1 + s_2^2/n_2)}$	$ t > t_{\alpha/2, v}$	<code>t</code>	<code>> t_{\alpha/2, v}</code>	<code>stats.ttest_ind(a, b, equal_var=False)</code>
T-test pareado	$t = d / (sd / \sqrt{n})$	$ t > t_{\alpha/2, n-1}$	<code>t</code>	<code>> t_{\alpha/2, n-1}</code>	<code>stats.ttest_rel(antes, despues)</code>
ANOVA 1 factor	$F = MS_{entre} / MS_{dentro}$	$F > F_{\alpha, k-1, n-k}$	<code>f</code>	<code>> F_{\alpha, k-1, n-k}</code>	<code>stats.foneway(grupo1, grupo2, ...)</code>
Chi-cuadrado	$\chi^2 = \sum (O-E)^2 / E$	$\chi^2 > \chi^2_{\alpha, (r-1)(c-1)}$	<code>chi2</code>	<code>> chi2_{\alpha, (r-1)(c-1)}</code>	<code>stats.chi2_contingency(tabla)</code>

Métricas de Modelos

Regresión

Métrica	Fórmula	Python		
MAE	$(1/n) \sum y_i - \hat{y}_i $	<code>y_i - \hat{y}_i</code>	<code>`mean_absolute_error(y, y_pred)`</code>	
MSE	$(1/n) \sum (y_i - \hat{y}_i)^2$	<code>mean_squared_error(y, y_pred)`</code>		
RMSE	$\sqrt{\text{MSE}}$	<code>`np.sqrt(mse)`</code>		
R²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	<code>`r2_score(y, y_pred)`</code>		
MAPE	$(100/n) \sum y_i - \hat{y}_i / y_i$		<code>/y_i</code>	manual

Clasificación Binaria

Métrica	Fórmula	Python
Accuracy	$(\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$	<code>`accuracy_score(y, y_pred)`</code>
Precisión	$\text{TP} / (\text{TP} + \text{FP})$	<code>`precision_score(y, y_pred)`</code>
Recall	$\text{TP} / (\text{TP} + \text{FN})$	<code>`recall_score(y, y_pred)`</code>
F1	$2 \cdot \text{P} \cdot \text{R} / (\text{P} + \text{R})$	<code>`f1_score(y, y_pred)`</code>
AUC	Área bajo curva ROC	<code>`roc_auc_score(y, y_prob)`</code>

Tamaño del Efecto

$$d \text{ de Cohen} = (x_1 - x_2) / s_{\text{pooled}}$$

$$s_{\text{pooled}} = \sqrt{((n_1 - 1)s_1^2 + (n_2 - 1)s_2^2) / (n_1 + n_2 - 2)}$$

Fórmulas de Potencia y Tamaño Muestral

Para Proporción (A/B Testing)

$$n = (z_{\alpha} \cdot \sqrt{2p(1-p)} + z_{\beta} \cdot \sqrt{p_1(1-p_1) + p_2(1-p_2)})^2 / (p_2 - p_1)^2$$

Para una Media

$$n = (z_{\alpha} + z_{\beta})^2 \cdot \sigma^2 / \Delta^2$$

Donde:

- z_{α} : valor crítico para α (1.96 para $\alpha=0.05$)
- z_{β} : valor crítico para $1-\beta$ (0.84 para 80% potencia)
- Δ : diferencia mínima a detectar

Bibliografía y Recursos Recomendados

Libros Fundamentales

Estadística y Probabilidad

1. **Wasserman, L. (2004).** *All of Statistics: A Concise Course in Statistical Inference*. Springer.

- Cubre desde probabilidad básica hasta inferencia moderna. Riguroso pero accesible.

2. **Casella, G. & Berger, R. (2021).** *Statistical Inference*. 2nd ed. Cengage Learning.

- El texto de referencia para teoría estadística a nivel de posgrado.

3. **Freedman, D., Pisani, R. & Purves, R. (2007).** *Statistics*. 4th ed. W.W. Norton.

- Excelente para intuición estadística sin sacrificar rigor.

Machine Learning y Aprendizaje Estadístico

4. **Hastie, T., Tibshirani, R. & Friedman, J. (2009).** *The Elements of Statistical Learning*. 2nd ed. Springer.

- El libro de cabecera de todo científico de datos. Disponible gratis online.

5. **James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021).** *An Introduction to Statistical Learning*. 2nd ed. Springer.

- Versión más accesible del ESL. Con ejemplos en Python (ISLP).

Matemáticas para Datos

6. **Strang, G. (2022).** *Linear Algebra and Learning from Data*. Wellesley-Cambridge Press.

- Álgebra lineal aplicada al análisis de datos, por uno de los grandes.

7. **Deisenroth, M.P., Faisal, A.A. & Ong, C.S. (2020).** *Mathematics for Machine Learning*. Cambridge University Press.

- Cubre álgebra lineal, cálculo y probabilidad para machine learning.

Series Temporales

8. **Hyndman, R.J. & Athanasopoulos, G. (2021).** *Forecasting: Principles and Practice*. 3rd ed. OTexts.

- Disponible gratis en otexts.com/fpp3. Práctico y completo.

Comunicación y Visualización

9. **Knaflíc, C. (2015).** *Storytelling with Data: A Data Visualization Guide for Business Professionals*. Wiley.

- El mejor libro sobre comunicación visual de datos.

10. **Tufte, E. (2001).** *The Visual Display of Quantitative Information*. 2nd ed. Graphics Press.

- Clásico sobre visualización de datos. Principios atemporales.

Artículos y Publicaciones Clave

11. **Wasserstein, R.L. & Lazar, N.A. (2016).** "The ASA Statement on p-Values: Context, Process, and Purpose." *The American Statistician*, 70(2), 129-133.

- Declaración de la Asociación Estadounidense de Estadística sobre el uso correcto del valor p.

12. **Cohen, J. (1994).** "The Earth Is Round ($p < .05$)." *American Psychologist*, 49(12), 997-1003.

- Crítica fundamental al uso mecánico de los valores p.

13. **Leek, J.T. & Peng, R.D. (2015).** "What is the question?" *Science*, 347(6228), 1314-1315.

- Sobre la importancia de formular la pregunta correcta antes del análisis.

Recursos Online

Documentación Oficial

Librería	URL
NumPy	numpy.org/doc/stable
SciPy	docs.scipy.org
SymPy	docs.sympy.org
pandas	pandas.pydata.org/docs
scikit-learn	scikit-learn.org/stable/documentation
statsmodels	[statsmodels.org/stable](https://www.statsmodels.org/stable)

Cursos Gratuitos

• **Statistics 110** (Harvard, Joe Blitzstein): Probabilidad —
[YouTube](https://www.youtube.com/playlist?list=PL2SOU6wwxBouwwH8oKTQ6V66RgVofTk1Y)

• **Statistical Learning** (Stanford, Hastie & Tibshirani): —
[lagunita.stanford.edu](https://lagunita.stanford.edu/courses/HumanitiesSciences/StatLearning/Winter2016/about)

• **Mathematics for Machine Learning** (Imperial College): —
[Coursera](https://www.coursera.org/specializations/mathematics-machine-learning)

Datasets para Práctica

- [Kaggle](https://kaggle.com): Competiciones y datasets
- [UCI Machine Learning Repository](https://archive.ics.uci.edu/ml): Datasets clásicos
- [Datos Abiertos Gobierno de México](https://datos.gob.mx): Datos públicos

- [INE](https://ine.es): Datos estadísticos de España
- data.gov: Datos abiertos de EE.UU.

Comunidades

- [Cross Validated (StackExchange)](https://stats.stackexchange.com): Foro de estadística
- [r/datascience](https://reddit.com/r/datascience): Comunidad de ciencia de datos
- [r/statistics](https://reddit.com/r/statistics): Comunidad de estadística

Software y Herramientas

Herramienta	Propósito	URL
JupyterLab	Entorno interactivo	jupyter.org
VS Code	Editor de código	code.visualstudio.com
streamlit	Dashboards interactivos	streamlit.io
Orange	Visualización sin código	orangedatamining.com
DBeaver	Gestión de bases de datos	dbeaver.io

Referencias Académicas Citadas en el Libro

- Fisher, R.A. (1925). *Statistical Methods for Research Workers*. Oliver & Boyd.
- Bayes, T. (1763). "An Essay towards solving a Problem in the Doctrine of Chances." *Philosophical Transactions of the Royal Society*, 53, 370-418.
- Pearson, K. (1901). "On Lines and Planes of Closest Fit to Systems of Points in Space." *Philosophical Magazine*, 2(6), 559-572.
- Hotelling, H. (1933). "Analysis of a complex of statistical variables into principal components." *Journal of Educational Psychology*, 24(6), 417-441.
- Box, G.E.P. (1976). "Science and Statistics." *Journal of the American Statistical Association*, 71(356), 791-799.

Sobre el Autor

Alex Goyzueta Delgado es Analista de Datos Senior con más de 12 años de experiencia transformando datos en decisiones. Ha trabajado en sectores como banca, retail, salud y tecnología, liderando equipos de análisis y diseñando estrategias basadas en evidencia cuantitativa.

Su formación combina matemáticas aplicadas con un máster en Ciencia de Datos. Ha impartido formación en estadística aplicada y machine learning a más de 3,000 profesionales en toda Latinoamérica y España.

Cree firmemente que las matemáticas no deberían ser una barrera para nadie que quiera trabajar con datos. Su misión es democratizar el conocimiento cuantitativo y ayudar a analistas de todo el mundo a entender *por qué* funcionan las herramientas que usan a diario.

Contacto:

- Email: alexgoyzueta2018@gmail.com
- GitHub: <https://github.com/SistGoyPeru>

• [Repositorio](https://github.com/SistGoyPeru) [libros:](https://github.com/SistGoyPeru/Libros)
[github.com/SistGoyPeru/Libros](<https://github.com/SistGoyPeru/Libros>)